MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI Generated Interpretation

LICENSE THESIS

Graduate: **Ioan-Iulian TODOR**

Supervisor: **Conf. Dr. Eng. Bianca Cristina POP**

2026

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA
FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Graduate: **Ioan-Iulian TODOR**

HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI Generated Interpretation

1. **Project proposal:** HospitAI is a hospital analytics platform that clusters patients into phenotypes with K-Prototypes, mines their care pathways with PrefixSpan, and uses a large language model to summarize the results and answer natural language questions about the database. The initial data is a Synthea generated synthetic export of 22,284 patients with full clinical histories.
2. **Project contents:** Introduction, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, System Design and Implementation, Testing and Validation, User's Manual, Conclusions, Bibliography, Appendices.
3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:** Conf. Dr. Eng. Viorica CHIFU
5. **Date of issue of the proposal:** October 1, 2025
6. **Date of delivery:** July 10, 2026

Graduate: Ioan-Iulian Todor Todor

Supervisor: Conf. Dr. Eng. Bianca Cristina POP

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA
FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Declarație pe proprie răspundere privind autenticitatea lucrării de licență

Subsemnatul(a) **TODOR Ioan-Iulian** legitimat(ă) cu **CI** seria **XB** nr. **667298 CNP 5030517060617**, autorul lucrării **HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI Generated Interpretation** elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea Calculatoare din cadrul Universității Tehnice din Cluj-Napoca, sesiunea **vara** a anului universitar **2026**, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență.*

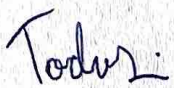
Data

10.07.2026

Nume, Prenume

TODOR Ioan-Iulian

Semnătura



Contents

Chapter 1 Introduction **1**

Chapter 2 Project Objectives **3**
2.1 Main Objective . . . 3
2.2 Secondary Objectives . . . 4

Chapter 3 Bibliographic Research **6**
3.1 Clustering and Sequence Mining in the Context of Patient Data . . . 6
3.1.1 Patient Clustering Methods . . . 6
3.1.2 Sequential Pattern Mining Methods . . . 9
3.2 Large Language Models and Prompt Engineering in Health Care . . . 11
3.3 Conclusions . . . 12

Chapter 4 Analysis and Theoretical Foundation **14**
4.1 Analysis . . . 14
4.1.1 Use Case Analysis . . . 14
4.1.2 Functional and Non-functional Requirements . . . 18
4.2 Theoretical Foundation . . . 19
4.2.1 Clustering Algorithms and Mixed-Type Data Handling . . . 20
4.2.2 Sequential Patterns Mining . . . 26
4.3 AI-Driven Patient Phenotyping and Pathway Discovery . . . 28
4.3.1 Formal Problem Definition . . . 28
4.3.2 Conceptual Pipeline Architecture . . . 29
4.3.3 Feature Scalers and Outlier Handling . . . 30
4.3.4 Pattern Reduction, Ranking, and Selection . . . 31
4.3.5 Bridging the Semantic Gap via Large Language Models . . . 32

Chapter 5 System Design and Implementation **35**
5.1 System Design . . . 35
5.1.1 System Architecture . . . 35
5.1.2 Database Design . . . 36
5.1.3 Frontend Design . . . 39
5.1.4 Backend Design . . . 40
5.2 System Implementation . . . 41
5.2.1 Frontend Implementation . . . 42
5.2.2 Backend Implementation . . . 45
5.2.2.1 ETL Pipeline . . . 48
5.2.2.2 K-Prototypes Clustering . . . 50
5.2.2.3 PrefixSpan Customizations . . . 57
5.2.2.4 Pipeline Orchestration . . . 64
5.2.2.5 AI Results Generation . . . 66
5.2.2.6 LLM SQL Agent (LangGraph) . . . 68

Chapter 6 Testing and Validation **72**
6.1 Used Dataset . . . 72
6.2 K-Prototypes Results . . . 74
6.3 Sequence Mining Results . . . 84
6.3.1 Statistical Bar: Lift over Chance . . . 86
6.3.2 Clinical Face-Validity and Per-Cluster Relevance . . . 87
6.3.3 Order-Sensitivity Null Test . . . 87
6.4 LLM Validation . . . 88
6.4.1 Cluster and Sequence Interpretation . . . 88
6.4.2 SQL Agent . . . 89

Chapter 7 User's Manual **91**
7.1 Installation . . . 91
7.1.1 Software Requirements . . . 91
7.1.2 Step-by-Step Installation . . . 91
7.2 User Manual . . . 92
7.2.1 Signing In . . . 93
7.2.2 Patient . . . 93
7.2.3 Doctor . . . 94
7.2.4 Manager . . . 96
7.2.5 Natural-Language Query Agent . . . 101

Chapter 8 Conclusions **102**

Bibliography **104**

Appendix A Relevant Code sections **107**

Chapter 1. Introduction

The medical field is one of the domains where huge amount of data can be collected. This data includes both general patient information (age, weight, height, blood type) and clinical records such as treatment plans (prescribed medications, surgical interventions, vaccines), consultation parameters, hospitalizations, allergies and so on. Through proper processing, this data can help doctors identify **patterns that repeat across patients**. In this way, patient groups that require similar treatment plans can be determined, and the treatments that worked best for each group can also be observed.

In the last 15 to 20 years, to keep up with digitalization, more and more hospitals have choosen digital solutions for storing data and patient history. The classic paper records and files have gradually been replaced by Electronic Health Records (EHR), a system that stores the complete history of a patient in digital form. Because digital records can be accessed much more easily than paper archives, analyses and reports made on patients or groups of patients have become easier to do, and their findings can feed back into daily medical practice.

Despite the growing volume of medical data, many hospitals around the world still lack systems to efficiently store and process patient records. The data either remains written on stacks of paper, or is stored in hospital databases but remains untouched and unprocessed. This potential is largely wasted, as Yadav et al. [1] point out.

Implementing a system like the one described in this thesis raises four challenges. The first is getting and preparing the data that needs to be processed. The second involves grouping patients using an algorithm that can combine both numerical data (age, body mass index, number of hospitalizations, results of the medical tests) and categorical data (gender, dominant care plan, dominant encounter class, condition burden). The third is finding and analyzing clinical event sequences through a sequential pattern mining algorithm. The fourth relates to interpreting the results and querying the clinical database, so non-technical users can find and understand the information they are searching for.

The second and third challenges are complex, because clinical data does not behave like ordinary business data. The record of a single patient mixes values of different kinds: integers such as blood pressure or cholesterol, real numbers such as the body mass index, and plain text such as diagnoses or allergy descriptions. These values also move on very different scales. A thyroid hormone result stays inside a narrow interval, while cholesterol can cover an interval a hundred times wider. On top of that, the timeline of a patient is irregular: a healthy person may have no records for years, while another patient collects dozens of events in a single week. No single mathematical model handles all of this at once, which is why the platform combines several specialized algorithms instead of relying on only one.

Building this pipeline requires a complete, complex dataset. Real hospital records, however, are hard to obtain: they contain sensitive information such as names, birth dates, and full medical histories, so GDPR regulations restrict their use to specific insti-

tutional approvals, and even then the sensitive fields must be anonymized. For this thesis, the opensource project **Synthea** was used instead, through which a CSV dataset of 22,284 patients was generated. Each patient has their entire medical history simulated in depth, with detailed data on organizations, providers, encounters, conditions, allergies, observations, immunizations, care plans, procedures, and medications. Because of the depth of the relationships between tables, the dataset is a suitable base for the system that we decided to build. The choice of synthetic data does not weaken the analysis: Walonoski et al. [2] show that, as the simulated Synthea population grows, its demographic and disease statistics come close to the official ones, so the conclusions of this work remain meaningful for real patients.

Since the dataset contains both numerical and categorical data, this thesis proposes the use of **K-Prototypes** for implementing the clustering process, as this algorithm allows the simultaneous use of both data types. K-Prototypes, introduced by Huang [3], extends the well-known K-Means algorithm by adding support for categorical attributes through a combined distance function that uses squared Euclidean distance for numerical features and a simple matching measure for categorical ones.

To find patterns of clinical sequences (particular diagnosis → medicine → surgery → care plan), the thesis adopts the **PrefixSpan** approach [4], a sequential pattern mining method that searches efficiently for frequent subsequences of events. Applied to the medical event sequences extracted from the Synthea data, it identifies the common clinical sequences of the patients.

In addition to the clustering and sequential analysis, the system uses a **Large Language Model (LLM)** that performs two functions. First, an LLM generates automatic, easily readable summaries for each patient cluster, converting numeric statistical outputs into something that a physician would be able to understand without specialized training. The second function lets users query the database in natural language through a text-to-SQL agent (for instance, "Show me the most common disease among patients under 15 in the last month"), with results returned in natural language as well.

This thesis introduces **HospitAI**, a hospital analytics platform built around a single end-to-end pipeline in which the output of each stage becomes the input of the next. An ETL stage first ingests the raw records and builds each patient's clinical features and event history. K-Prototypes then splits the patient population into groups, PrefixSpan mines the frequent clinical sequences inside each discovered group, and an LLM finally turns both results into readable clinical text and answers natural language questions about the data. The platform was developed on synthetic EHRs generated via Synthea and offers three role-based interfaces: Patients consult their own medical record, Doctors explore patient profiles and enter consultation data, and Managers launch the pipeline, analyze its results, and view hospital statistics.

The reminder of this document is organized as follows. Chapter 2 defines the project objectives. Chapter 3 reviews the related literature and positions HospitAI within the current state of the art. Chapter 4 analyzes the system through its actors, use cases, and requirements, and then presents the theoretical foundation of the algorithms used, together with the conceptual architecture of the analytical pipeline. Chapter 5 describes the detailed design and implementation. Chapter 6 presents the testing and validation results. Chapter 7 provides the user manual. Chapter 8 concludes the thesis and discusses potential future work.

Chapter 2. Project Objectives

As discussed in Chapter 1, modern healthcare organizations store huge volumes of EHR data that can either not be analyzed at all or, if it is, is rather difficult for interpretation for the majority of people, such as physicians, hospital managers or even patients. This thesis addresses this issue and aims to *develop and validate an analytics solution called HospitAI that converts EHR data into meaningful clinical insights.*

Such an analytics framework, although not yet realized in literature, could build on technologies that currently operate independently from each other. The clustering of *mixed-type* clinical data can be performed using algorithms like K-Prototypes, while the temporal aspect of it, whose clinical significance cannot be represented in a static fashion, is achievable through sequential pattern mining. What is missing today, and is discussed in detail in Chapter 3, is a framework that can combine all these pieces. In particular, it should cluster patients into phenotypes, mine trajectories of care in separate clusters, and interpret the mined patterns automatically. The last step is necessary, since the outputs of algorithms used in the previous steps, namely prototype vectors, support counts and event sequences, are completely opaque to physicians and healthcare decision makers, which causes a *semantic gap* between the data mining phase and the target user group. Stratification, per-cluster trajectory mining, and automatic interpretation are the three key concepts of the proposed framework. Since experimentation with those concepts is not possible with real-world data due to legal restrictions, the proposed framework uses the *Synthea* synthetic dataset in its design and evaluation process, while the costly data analysis is performed offline in batch mode, making it possible to focus on the analytics pipeline rather than production-grade application infrastructure. The rest of this chapter formally sets out the objective of this thesis and six subobjectives that form the underlying structure of this work.

2.1. Main Objective

The primary goal of the present work is to **design, implement, and evaluate the HospitAI platform**, which is an end-to-end framework thattransforms raw EHRs into practical clincial knowledge. This primary goal will be reached by creating a software pipeline that chains clustering, sequence mining, and language-model interpretation. The pipeline ingests and processes data (synthetically generated patient data) using unsupervised clustering algorithms, which allows the creation of phenotypes based on complex medical backgrounds. Then, sequential pattern mining algorithms will be applied for care path and treatment trajectory identification. To connect the purely statistical processing mentioned above with clinical decision-making, the platform use Large Language Models (LLMs) integrated into the constrained interpretation pipelines for understanding the output. Success is defined as the system's ability to autonomously process raw EHR data, perform phenotyping, and visualize care pathways via a user interface.

2.2. Secondary Objectives

The primary goal discussed above is reached through the accomplishment of the following secondary goals, corresponding to the stages of development of the platform and further chapters of this thesis.

I. **Study the state-of-the-art literature and existing algorithms.** Perform an in-depth study of existing methods of clustering, sequence mining and LLMs, to theoretically justify the chosen architecture of the platform. The literature survey should emphasize the limits of the use of individual approaches in processing complex healthcare datasets, revealing the gap which is covered by the current work. Success is measured by the critical comparison of each work studied in the literature survey to the proposed platform, provided in Chapter 3.

II. **Develop an integrated, functional prototype of the platform.** Consolidate the analytical building blocks into a single runnable system rather than disconnected scripts. The prototype couples a web-based, role-aware interface for the Manager, the Doctor, and the Patient with a backend application and a relational database, whose data foundation is established in the following objective. Success is measured by a running end-to-end system that loads the dataset, executes the full pipeline on demand, and presents the resulting phenotypes and care pathways.

III. **Design the end-to-end data flow (ETL pipeline) and database architecture.** Extract, transform and load the synthetic raw data into a normalized relational database to filter the noise, establish the relational links between the entities, and generate patient-level clinical features used by the downstream models. This will create a standardized and reproducible data environment that decouples machine learning tasks from the format of the raw data, forming the data backbone of the functional prototype defined above.

IV. **Implement the proposed methodology for patient stratification and clinical trajectory extraction.** Use unsupervised learning techniques to generate homogeneous phenotypes from patients using attributes of a mixed type natively, and use a pattern-mining approach to find frequent and clinically meaningful sequences of events in the history of each individual cluster. Given the generated phenotypes, the combination of two different approaches will produce one analytical pipeline where the output of the clustering part will feed into the input of the mining part.

V. **Develop an LLM-based conversational and decision-support interface.** Combine an intelligent agent that has two tasks: secure conversion of natural language questions into read-only SQL queries so that non-expert users could query the database, and creation of a clear and constraint-driven clinical summary for each cluster and pathway found. In both tasks, the language model is intentionally constrained so that it speaks about and queries the available information without ever making up numbers and updating the database. The interface succeeds if a user who lacks technical expertise can get the correct and readable answer backed up by the data.

VI. **Rigorously test, validate, and evaluate the system outcomes.** Evaluate the quality of the architecture design based on the internal validation criteria for the clusters and statistical criteria for the sequential patterns, because there is no way to know the ground truth labels for either type of problem. At the same time, strictly verify the safety and accuracy of the LLM-generated responses to prevent clinical hallucinations, which is a prerequisite for deploying the model on the clinical data. These results are presented in Chapter 6.

Having established these objectives, Chapter 3 reviews the literature relevant to each of these analytical techniques and shows where HospitAI stands in the current state of the art.

Chapter 3. Bibliographic Research

This chapter provides a review of the existing state of the art related to the three pillars that HospitAI relies on for its analysis: the stratification of the patient population using clustering, the discovery of clinical pathways using sequential pattern mining, and the interpretation of the resulting models using large language models (LLMs). For every work we first explain, in plain terms, the problem the authors solve and how their method works, then compare it to the solution of this thesis. Each subsection closes with a concise comparison table. Every reviewed study succeeds at its own objective. This chapter reads them through one specific lens, that of building a single integrated analytical platform, and points out where each stops short of that particular goal. Those gaps are what HospitAI sets out to fill.

3.1. Clustering and Sequence Mining in the Context of Patient Data

Both of these methods are discussed together in this section, not only because they are often combined in the existing literature, but also because in the context of this thesis one is used as an input for the other.

3.1.1. Patient Clustering Methods

Unsupervised stratification of patients into groups with similar clinical phenotypes is an old and extensively researched problem in healthcare analytics. The five sources presented below address the issues that are relevant for HospitAI, including an algorithm selection survey, an application for decision support, a couple of generalizations for heterogeneous data, and an application for mixed-type data on a specific disease.

Aljohani (2024). Aljohani [5] treats the choice of a clustering algorithm for patient stratification from electronic health records (EHRs) as a decision with many competing criteria, not a single accuracy score. A panel of experts scores eight of the usual algorithms, from K-Means and DBSCAN to Gaussian Mixture Models and Self-Organizing Maps. They are judged on eight practical qualities, cluster quality, scalability, robustness to noise, interpretability, and stability among them. The scores are then combined with Fuzzy TOPSIS, a multi-criteria method that folds many weighted expert judgments into a single ranking. That ranking is cross-checked with a second method, the Analytic Hierarchy Process, and K-Means comes out on top, ahead of hierarchical clustering and DBSCAN. The study is a careful map of how mainstream algorithms behave on EHR data. But every one of the eight works only on purely numerical inputs. Clinical records are not purely numerical. Attributes such as diagnostic codes, gender, or careplan type are categorical, and handing them to these algorithms means encoding them as numbers first. That inflates the dimensionality and introduces geometric distortions discussed in

Chapter 4. HospitAI avoids that step by clustering with K-Prototypes. It measures numerical and categorical attributes each with its own native distance and then combines them, so a patient's diagnoses and vitals are compared together without either being forced onto a single artificial numeric scale.

Yang et al. (2024). Yang et al. [6] build an intelligent healthcare system that clusters medical records to track patient risks and trends over time. The clusters are meant to support hospital-management decisions, and they are paired with the idea of evidence-based medicine: groups of patients are compared across diseases, time periods, and patient counts, so managers can weigh options against documented outcomes. The aim of making analytics useful to hospital managers, and not only to statisticians, is close in spirit to HospitAI. The difference lies in how the patients are grouped. Yang et al. cluster on data that has been encoded into numbers beforehand, so the categorical side of a record is flattened before any grouping happens. HospitAI keeps categorical attributes as categories and clusters them directly with K-Prototypes. This is what lets a group be defined collectively by, say, a dominant careplan category and an average number of active medications, without either being converted into the other.

Novidianto et al. (2021). Novidianto et al. [7] use the same algorithm as this thesis, K-Prototypes, to group heart-failure patients by attributes linked to mortality. Their cohort is real but small: 299 patients described by eleven mixed attributes. These combine numeric measurements, such as ejection fraction and serum creatinine, with categorical flags, such as diabetes and hypertension. They pick the number of clusters with the silhouette coefficient, which is highest at $k = 2$, giving two groups that they read as low and high mortality risk. The study is useful confirmation that K-Prototypes handles clinical mixed data well and that a simple internal index can choose the number of groups. HospitAI validates its own clustering the same way. The scope, however, is deliberately narrow: one disease, eleven curated attributes, two clusters. HospitAI applies the same native mixed-type clustering to a far broader problem. It covers more than 22,000 patients from the whole population rather than one pathology, described by dozens of engineered numeric and categorical features. It also adds an explicit outlier pass that sets atypical patients aside as a separate group, instead of letting them distort the cluster centers.

Najjar and Gagné (2014). Najjar and Gagné [8] generalize K-Prototypes to a kind of data the standard algorithm cannot handle: *multivalued* categorical attributes. These are single fields that hold a whole set of values at once, such as all the diagnostic codes recorded for one admission. Collapsing such a set to a single code would throw away most of it, so the authors borrow the bag-of-words idea from text analysis. For each multivalued attribute, the Apriori algorithm finds the frequent combinations of codes, and those combinations become the vocabulary. Each patient's codes are then weighted by tf–idf, so common codes count for less and distinctive ones for more, and a cosine comparison between these weight vectors and the cluster centers is added to the K-Prototypes distance. The extension keeps the algorithm's linear running time. It also makes the case for handling clinical attributes in their natural form rather than through reductive encodings. HospitAI shares that philosophy but draws the line differently. Instead of feeding raw sets of codes into the clustering distance, it engineers a patient's multivalued history into compact single-valued features first: disease flags such as *has-diabetes*, counts such as the number of active conditions, and one dominant careplan

category. The standard K-Prototypes distance then applies directly, and the full ordered set of clinical events is reserved for the separate mining stage described in the next section.

José-García et al. (2022). José-García et al. [9] attack the mixed-data problem from the opposite direction. Instead of finding one distance that fits every attribute type, they avoid committing to any single distance at all. Their method, MVMC, treats each kind of data as a separate "view" with its own dissimilarity measure. It then runs a multi-objective evolutionary search over a systemic-sclerosis cohort to find a consensus grouping that holds up across all the views at once. Because the search optimizes several objectives together, it returns not one clustering but a Pareto set, a family of trade-off solutions in which no member is strictly better than the others. The strength of this design is that it never merges the different attribute types early, so no view can dominate the rest by accident. The cost is that the evolutionary search is heavy to run, and its output is a set of trade-off partitions described by their members rather than by compact, readable cluster centers. HospitAI makes the opposite trade. It accepts a single distance, the K-Prototypes mix of numeric and categorical measures, in exchange for one definite partition whose clusters are summarized by prototype centers. That partition is the concrete, inspectable grouping the later stages of the platform build on.

Table 3.1 summarizes the five clustering methods along the aspects that matter for HospitAI.

Reference (approach)	**Data types**	**Advantages**	**How HospitAI clusters differently**
Aljohani [5] (8-algorithm Fuzzy TOPSIS comparison)	Only numerical	Multi-criteria rigorous approach to EHR clustering	All eight algorithms are numeric-only; categoricals need lossy encoding that distorts distances. HospitAI clusters mixed types natively with K-Prototypes
Yang et al. [6] (Applied clustering DSS)	Numerical (encoded)	Management-oriented. Integrates clustering with evidence-based medicine	Clusters on pre-encoded numbers, flattening categoricals first. HospitAI keeps categories as categories
Novidianto et al. [7] (ClusterMix K-Prototypes)	Mixed (numeric + categorical)	Natural mixed-type distance measure. Silhouette-based k optimization on real data	Small single-pathology cohort (n=299), k=2, few curated attributes. HospitAI scales the same distance to 22k patients and dozens of engineered features with outlier handling
Najjar and Gagné [8] (multivalued K-Prototypes)	Mixed + multivalued categorical	Handles sets of codes via bag-of-words. Linear time	Multivalued codes handled inside the distance via Apriori/tf–idf. HospitAI engineers such history into single valued flags and counts instead

continued on next page

Reference (approach)	Data types	Advantages	How HospitAI clusters differently
José-García et al. [9] (multi-view MVMC)	Mixed (per view)	Avoids early integration. Consensus clusters across views	Costly evolutionary search yielding a Pareto set of partitions. HospitAI produces one prototype-based partition from a single combined distance

Table 3.1: Comparison of the reviewed patient-clustering methods.

Across these papers the same lesson recurs: clinical data is heterogeneous, and native mixed-type clustering, which HospitAI performs with K-Prototypes, fits it better than forcing every attribute onto one numeric scale. What a clustering cannot show, though, is the order in which a patient's clinical events unfold. A cluster is a snapshot of who a patient is, not a timeline of what happened to them. Reading care as a sequence over time is the job of the methods in the next section.

3.1.2. Sequential Pattern Mining Methods

While clustering helps us understand *who* our patients are at a particular time point, it is incapable of conveying the *sequence* in which these clinical events happen. The following five papers have been selected, ranging from visual analytics and efficiency studies to a pipeline that actually couples sequence mining with clustering.

Perer et al. (2015). Perer et al. [10] build Care Pathway Explorer, a system that mines frequent sequences of clinical events from electronic medical records and lays them out in an interactive visual interface. Their miner is adapted to the awkward realities of EMR data: events that happen on the same day, events recorded at different levels of detail, and outcomes attached to a pathway. The discovered patterns are then shown in a combined overview-and-flow view. Applied to a group of hyperlipidemia patients, it surfaces care patterns that are useful and occasionally new to clinicians. HospitAI mines sequences for the same reason but differs in two ways that matter for this thesis. First, Care Pathway Explorer hands the discovered patterns to an experienced analyst, who reads them off the visualization by hand. HospitAI instead turns each pattern into a written clinical narrative automatically, so it does not need a trained analyst in front of a chart to become readable. Second, Care Pathway Explorer mines the whole cohort at once. HospitAI mines each patient cluster separately, and the outlier group on its own, so a rare but expensive pathway confined to one small group is not drowned out by the majority.

Edman (2018). Edman's work [11] is about efficiency: can clinical-pathway sequences be mined at a lower support threshold, so that rarer patterns come into view? In earlier work the author had used PrefixSpan, the same family of algorithm as HospitAI. But PrefixSpan was too slow to run at the low support levels needed, so the expected pathways could only be partly confirmed. The remedy here is CSpan, reported to be about two orders of magnitude faster, which does reach those lower thresholds. The important finding is a negative one: pushing the threshold down and mining many more patterns

did not improve their quality by much. That result is part of the motivation behind HospitAI's design. Mining more sequences is not the goal in itself. What decides whether the output is useful is what happens to the patterns afterwards: how they are ranked, filtered, scoped to the right patients, and explained. HospitAI therefore spends its effort not on squeezing out ever-rarer patterns, but on ranking the ones it finds by a statistical interestingness score and turning them into readable narratives.

Botas (2019). Botas [12] mines a national e-prescription database, using a case study on anticoagulants. The work combines two families of pattern mining. Association rules, via Apriori, find drugs that co-occur. Sequential pattern mining, via PrefixSpan, finds the order in which prescriptions follow one another. The raw patterns are not taken at face value. They are scored with a battery of measures for how interesting, novel, and semantically meaningful they are, and the results are broken down along fixed axes such as age group, gender, and health district. HospitAI is close to this work in two respects: it also uses PrefixSpan, and it also ranks patterns by interestingness rather than raw frequency. But it differs in structure. Botas mines a single type of event, drug presciptions. HospitAI mines four types together, conditions, medications, procedures, and careplans, so a pattern can cross from a diagnosys to a treatment to a procedure. Botas also slices results along predefined demographic strata. HospitAI instead mines within clusters it has learned from the data. It then applies closed-pattern selection, a geometric-mean-lift interestingness score, and a Jaccard overlap filter, all described in Chapter 4, to keep the surviving patterns distinct rather than near-duplicates.

Baudrier et al. (2022). Baudrier et al. [13] take the step this thesis cares about most: they explicitly combine clustering with sequence mining. They work on a French medico-administrative dataset of kidney-cancer patients on oral anticancer therapy. First they mine each patient's healthcare-utilization sequence with the CP-SPAM algorithm. Then they run hierarchical clustering on the principal components of those mined sequences, ending with three groups that differ markedly in mortality and resource use. Of all the works reviewed here, this pipeline is the closest to HospitAI's idea of cluster-scoped pathway analytics. The differences are in the order of operations and in how the patterns are handled. Baudrier et al. mine first and cluster afterwards, so their clusters are groups of similar sequences. HospitAI clusters patients first and then mines each cluster, so the sequences are read within an already-established patient phenotype. Baudrier et al. also keep every mined pattern without a statistical interestingness ranking, and their sequences are read by domain experts by hand. HospitAI ranks patterns by geometric mean lift and generates their interpretation automatically.

Najjar et al. (2018) The pipeline by Najjar et al. [14] is, in overall shape, the closest of all to HospitAI. It clusters heterogeneous clinical visits with K-Prototypes, sharing this thesis's commitment to native mixed-type clustering. It then assembles the resulting clusters into treatment pathways and clusters those pathways again to find common trajectories, all evaluated on a real cohort of elderly heart-failure patients. Two differences remain central. Najjar et al. model the time dimension with Hidden Markov Models, which produce probabilistic transition matrices between hidden states. HospitAI instead produces explicit sequences of actual clinical events, ranked by support and interestingness. A clinician can read one directly as "this, then this, then this" rather

than as a state-transition probability. Najjar et al. also stop at the mined models, leaving a clinician to interpret them. HospitAI passes each pattern through a constraint based language model verbalizer that writes the interpretation. These two gaps, explicit ranked sequences and automatic verbalization, are what the mining and language-model components of HospitAI are built to fill.

Table 3.2 contrasts the five sequence-mining works along the algorithm they use, the scope over which they mine, and their limitations relative to HospitAI.

Reference (algorithm)	**Mining scope**	**Advantages**	**Limitations (vs. HospitAI)**
Perer et al. [10] (frequent sequence mining)	Whole cohort	EMR-aware mining. Rich interactive visual analytics	Visual reading needed. No phenotype scope. No automation of narrative
Edman [11] (CSpan vs. PrefixSpan)	Whole cohort	More efficient mining. Achieves lower support limits	Efficiency only. Lower support improved output little. No re-ranking, scope or interpretation
Botas [12] (Apriori, PrefixSpan)	Pre-defined demographic segments	Combines rules and sequences. Thorough interestingness assessment	One event type (medications). Segments fixed, not learned. No cluster conditioning or LLM
Baudrier et al. [13] (CP-SPAM + HCPC)	Post-analysis trajectory clusters	Couples clustering with mining. Outcome-related groups	Clustering follows mining. No interestingness re-ranking. Manual interpretation
Najjar et al. [14] (K-Prototypes + HMM)	Per visit-cluster	Full two-step pipeline on mixed data. Closest overall shape	Probabilistic HMM rather than explicit ranked sequences. Interpretation left to clinicians

Table 3.2: Comparison of the reviewed sequential-pattern-mining methods.

All components of HospitAI's second pillar already exist in the literature, though not combined in the form this thesis proposes: mining based on learned phenotypes, ranking patterns by their statistical interestingnes, and, above all, a way to connect mined sequences automatically into a story a clinician can read. That is the topic of the third pillar.

3.2. Large Language Models and Prompt Engineering in Health Care

The third pillar of HospitAI is about translation. The clustering and mining stages output raw numbers, and those numbers must become language a clinician can read. In the other direction, a clinician's plain-language question must become a safe database query. The two references here cover, in turn, prompt engineering and a responsible-deployment framework.

Liu et al. (2025). Liu et al. [15] offer a tutorial on prompt engineering aimed at clinicians who are not NLP specialists. Their premise is that with models like the GPT series the quality of the answer depends heavily on the quality of the prompt. Small changes in

wording can produce very different outputs, and in a medical context that is a safety problem, not just an inconvenience. The tutorial works through the main techniques: zero-, one-, and few-shot prompting, chain-of-thought, self-consistency, and meta-prompting. It adds practical advice on stating objectives clearly and refining a prompt step by step, always with an eye on limiting hallucinations and protecting patients. HospitAI puts these principles to work directly. Its cluster- and sequence-interpretation prompts state an explicit objective and require every claim to be grounded in the numbers provided. They anchor risks and recommendations to recognized clinical guidelines, for example ADA for diabetes or ACC/AHA for hypertension. They also carry bias-mitigation instructions and reason through the case in internal chain-of-thought steps that stay out of the final answer. Where the tutorial stops is at free-text clinical interaction. It does not cover how to interpret structured machine-learning output, or how to turn a natural-language question into a safe database query. That is exactly what HospitAI adds, through deterministic control graphs and read-only SQL filters.

Al-Garadi et al. (2024). Al-Garadi et al. [16] give a broad review of how large language models are being used in healthcare and, more useful for this thesis, a framework for deploying them responsibly. They survey the ways an LLM is adapted to a specific task: fine-tuning, prompt engineering, and multimodal adaptation to EHR data. The whole effort is organized into a five-stage lifecycle, running from planning to maintenance. The framework rests on general principles: accountability, privacy, fairness, interpretability, keeping a clinician in the loop, and preventing hallucinations. Several of these map onto concrete mechanisms in HospitAI. Hallucination prevention corresponds to the deterministic, read-only SQL filter that every generated query must pass through. Keeping a clinician in the loop corresponds to HospitAI's framing of its output as decision support to be verified, not a diagnosys to be trusted. Being a review, though, the article stops at principles. It does not provide a working pipeline that connects the output of an unsupervised model to an LLM and a natural-language query interface. That is the gap HospitAI's third pillar fills.

Table 3.3 compares the two LLM-related works along their focus and their treatment of structured data and safety guardrails.

Reference	Focus	Advantages	Limitations (vs. HospitAI)
Liu et al. [15]	Tutorial on prompting for clinicians	Unified methods. Focus on hallucinations and safety	Controls free text only. Does not address structured ML output. No query control
Al-Garadi et al. [16]	Review and responsibility framework	End-to-end lifecycle. Accountability, fairness, clinician involvement	Guidance only. No structured-ML-to-LLM pipeline or NL querying

Table 3.3: Comparison of the reviewed LLM and prompt-engineering works.

3.3. Conclusions

The discussion of the three pillars leads to one observation. The present body of literature solves each task well on its own, but none of the frameworks under review

connect structured data mining to automated, guardrail-based interpretation of the resulting medical conclusions. The clustering papers end with the clustering itself and the interpretation of results is left for human specialists. The pathway-mining papers provide sequences either without considering learned phenotypes or without re-ranking the results using a statistical interestingnes metric. And the LLM papers control the generation of free-form interactions or predict outcomes without linking to the results of unsupervised interpretable pipelines. A single overall comparison matrix is deliberately avoided, since a clustering algorithm and a prompt-engineering paper cannot be judged by the same criteria. Still, read together, the per-section tables all point to the same conclusion: the gap is not inside each pillar, but between them.

That gap is what HospitAI addresses, and it is why the platform exists. The thesis does not introduce a new clustering algorithm or sequence miner. Its contribution is integration. It brings native mixed-type clustering, constrained sequence mining, and statistically re-ranked, constraint-based LLM interpretation into one platform, where each component feeds the next. The final patient-facing layer is generated automatically and is safe by design. The theoretical framework of these components is established in Chapter 4, while their practical implementation is detailed in Chapter 5.

Chapter 4. Analysis and Theoretical Foundation

This chapter presents the HospitAI analytics platform from two different points of view. The first is a user-focused view: the actors, the use cases, and the requirements of the system. The second is the mathematical background behind patient segmentation and sequence mining, the algorithms and the models they rely on. It also sketches the whole pipeline, including how the numeric results are turned into natural language with large language models (LLMs). Therefore, the aim is to justify the choice of methods, conceptually and mathematically. The technical details come later, in Chapter 5.

4.1. Analysis

This section describes how the system behaves. It uses use case decomposition first, then lists the requirements the platform satisfies.

4.1.1. Use Case Analysis

This subsection describes the use cases identified in the process of analysis. As seen in Figure 4.1, there are three actors who interact with the platform, each of them having their own access rights and specific interests in it:

- **Manager**: corresponds to the role of hospital management and data administration and is the primary beneficiary of the analytical layer. The Manager actor controls the analytics workflow, accesses the statistical dashboards, maintains and updates provider records, and accesses the patient data in read-only mode. The Manager also collaborate with clinical personnel to examine the identified patient phenotypes, care pathways, and the database through natural-language queries.
- **Doctor**: focuses on everyday clinical practice, by managing the records of patients and adding new clinical encounters data. For clinical decision making purposes, the Doctor can also investigate patient clusters and sequential patterns, and query the database using natural-language questions limited only to their own patients.
- **Patient**: has the smallest scope of interests, checking his or her own medical record and natural language queries for its personal health data.

As stated in Chapter 1, the three actors mentioned above form the demonstration layer illustarting how the interface and data scope vary from one user to another.

The analysis found eleven use cases. Four of them carry the contribution of this thesis and are detailed below, together with their flows. The other seven are listed briefly.

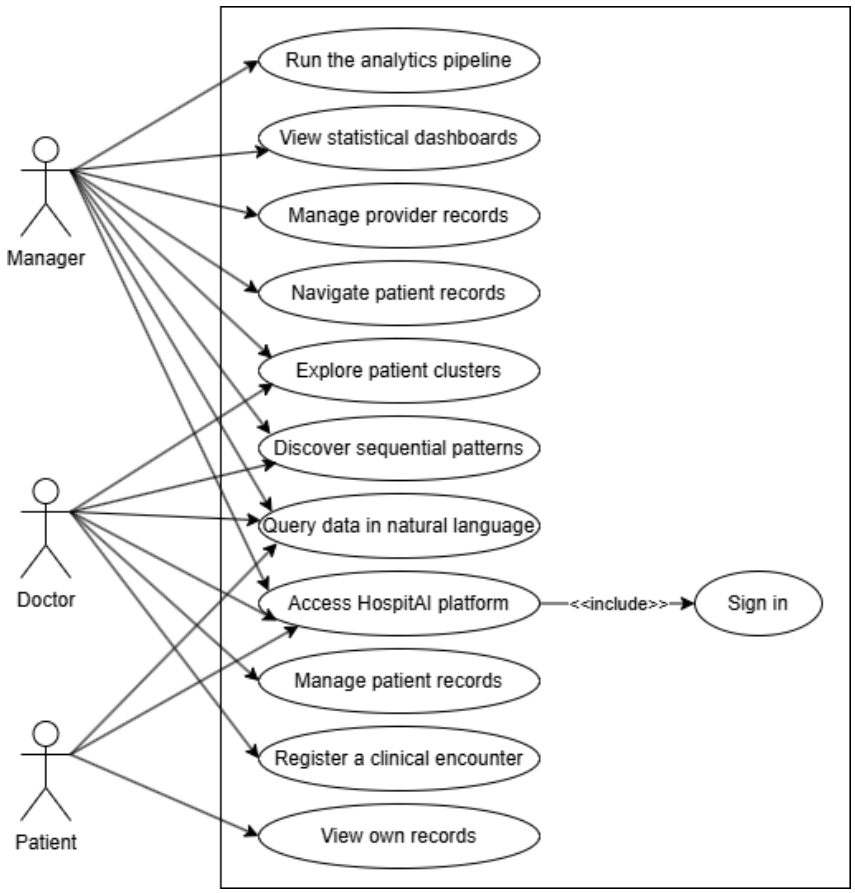


Figure 4.1: Use case diagram

1. **Use Case Title: Run the analytics pipeline**
   - **Brief description**: The Manager starts the end-to-end analytics pipeline. It loads the dataset on the first run, executes the K-Prototypes clustering, and mines sequential patterns both globally and within every cluster. The Manager can watch start the pipeline anytime, but it is by default automatically started once a month.
   - **Primary actor**: Manager
   - **Pre-conditions**: The actor is logged into the application as the Manager and there are no other pipeline executions running.
   - **Post-conditions**: The updated results of clustering and sequential pattern mining get available in the corresponding views.
   - **Basic flow**:
     (a) The actor visits the Pipeline page.
     (b) The system displays the current status of the pipeline along with the number of data, clusters, and mined sequences available.
     (c) The actor starts a new pipeline execution.
     (d) The system executes the stages seuqentially: data loading (only if the database is empty), clustering, global sequence mining, and sequence mining per cluster.
     (e) The system reports the stage of the pipeline execution until the execution completes.
     (f) Upon completion, the system completes the pipeline execution success-

fully.

- **Alternate flows**:
  3a. *A pipeline run is already in progress.*
    (a) The system rejects the request and keeps showing the progress of the ongoing run.
  4a. *A stage fails.*
    (a) The system stops the run and records the failure together with the stage that caused it.

2. **Use Case Title: Discover sequential patterns**
   - **Brief description**: The Manager or Doctor access the sequential care patterns identified through the PrefixSpan algorithm applied to the longitudinal histories of the patients in each cluster.
   - **Primary actor**: Manager, Doctor
   - **Pre-conditions**: The actor is logged in under the Manager or Doctor user role. The analytics pipeline has been run, which means that the clustering and sequential patterns mining per cluster has been done.
   - **Post-conditions**: The closed sequential patterns are presented, both globally and divided per clusters.
   - **Basic flow**:
     (a) The actor goes to the Sequences page.
     (b) The actor can select specific cluster and minimum support.
     (c) The system presents the closed patterns left after mining, sorted according to the support (descending order). The selected cluster, the support and the sorting order can be changed in the UI.
     (d) The actor picks a sequence pattern to see the clinical interpretation provided by the AI.
   - **Alternate flows**:
     3a. *No pattern with support above the threshold is found for the chosen cluster.*
       (a) The system presents an empty state message.

3. **Use Case Title: Explore patient clusters**
   - **Brief description**: The Manager or Doctor reviews the clustering analysis result by K-Prototypes algorithm: a cluster list, cluster prototypes, feature distributions, comparative heatmaps, PCA scatter plots, and summaries provided by AI.
   - **Primary actor**: Manager, Doctor
   - **Pre-conditions**: The actor is logged into the system as a Manager or Doctor and the analytics process is finished with storing the clustering result into the database.
   - **Post-conditions**: Cluster views appear.
   - **Basic flow**:
     (a) The actor moves to the Clusters tab.
     (b) The system displays the list of clusters and the cluster general info.
     (c) The actor selects one of the clusters.
     (d) The system displays the selected cluster prototype with feature distributions and the AI summary of the cluster (+ insights and recommendations).
     (e) The actor opens cluster comparison plots (heatmap and PCA scatter plot)

or feature-specific plots.

- **Alternate flows**:
  1a. *There are no cluster results yet.*
    (a) The system displays the empty state message.

4. **Use Case Title: Query data in natural language**
   - **Brief description**: The actor types a natural-language question. It is turned into a read-only SQL query, run against the database, and the result comes back as a natural-language answer.
   - **Primary actor**: Manager, Doctor, Patient
   - **Pre-conditions**: The actor is logged in and has the Manager, Doctor or Patient role.
   - **Post-conditions**: The natural-language answer extracted from the query result is delivered to the UI. The created SQL query is logged server-side and is not shown to the actor.
   - **Basic flow**:
     (a) The actor goes to the Ask AI page by pressing the button from the right side.
     (b) The actor inputs a natural-language question.
     (c) The actor submits the question.
     (d) The system creates an SQL query out of the provided question using the LLM agent.
     (e) The system checks the generated query against the read-only fragment and database schema. In the case of a Doctor actor, the query is limited to his/her provider records, and in the case of Patient actors, it is limited to his/her medical record only.
     (f) The system executes the query.
     (g) The system outputs the natural-language answer based on the result.
   - **Alternate flows**:
     5a. *Query is rejected by validation.*
       (a) Query is requested to be created again by the system up to three times from the agent.
       (b) When the query can't be validated after three attempts, the system notifies the user that the question cannot be answered safely.

The other seven use cases are create/read/update/delete/access use cases which have no inherent analysis logic behind them, so they are presented here briefly.

- **Access HospitAI platform** (Manager, Doctor, Patient). The user gains access to the platform, a process which implicitly includes the **Sign in** process using one of the three pre-defined demo accounts. Since the role of the user only shows the behavior of the UI, any password with lenght >8 will be accepted and no session token will be provided.
- **View statistical dashboards** (Manager). The Manager views aggregate dashboards covering all of the key clinical dimensions (patients, encounters, conditions, medications, observations, procedures, allergies, immunizations, care plans, and providers). An empty message is displayed for those dimensions having no data at all.
- **Manage provider records** (Manager). The Manager can create, edit delete or view providers using the paginated list. A confirmation message is asked before deletion,

inline validation is done.

- **Navigate patient records** (Manager). The Manager navigates through the paginated and filterable list of patients in the read-only mode and views the patient's longitudinal record.
- **Manage patient records** (Doctor). The Doctor performs operations of browsing, filtering and also adding, modifying or deleting of patient records, where there is confirmation during deletion and inline validation.
- **Register a clinical encounter** (Doctor). The Doctor adds, modifies or deletes a clinical encounter along with its sub-records (test results, condition, medication, procedures, immunizations, care plans, and allergies), where the patient and the doctor should be registered first.
- **View own records** (Patient). The Patient performs read-only viewing of his/her own records and all the sub-records that belong to each record, including procedures, conditions, observations, allergies and so on.

4.1.2. Functional and Non-functional Requirements

The requirements specified below refer to the functionalities of the platform. The main purpose of this thesis is not the development of a commercial quality web application, but rather the design and testing of the target analytical pipeline. The web application is needed to optionally run the pipeline and display its output. Also, it is used as a management platform, where EHR can be modified at any time. Because of this, the requirements were kept realistic on purpose, and all of them are met by the system described in Chapter 5.

Functional requirements

Functional requirements cover both the analysis from this thesis, as well as the support services needed to operate within the web application. These define what needs to be done by the system, and how that is accomplished.

- **Execution of the Analytics Pipeline.** The solution should allow Managers to run and automate the whole analytics pipeline. This is done by introducing a user interface for manual triggers and a scheduler that will move data loading, K-Prototypes clustering, and sequential pattern mining to asynchronous tasks that are executed automatically each 1st of the month, at 2am.
- **Cluster exploration.** The solution should allow the Managers and Doctors to visually understand the patient segmentation. It is accomplished through a separate page showing the results of the clusterization algorithm (individual prototypes, distribution of the features (means of the most important cluster features) and a PCA scatter plot) as well as AI-generated descriptions, insights and recommendations.
- **Exploration of the Sequential Patterns.** The solution should provide the opportunity to analyze frequent care patterns among patients for clinical staff and management. It is achieved by creating an interactive panel that will show closed sequences detected by the PrefixSpan algorithm and sorted by the support value, with additional AI explanations.
- **Interrogation of Medical Information in Natural Language.** It should be possible for authorized users (Manager, Doctor or Patient) to ask questions regarding medical information in natural language. This is done by integrating a Large Language

Model (LLM) agent that can translate natural text into a read-only SQL query and run it on the database.

- **Visualization of Statistics.** It should be possible for Managers to get aggregated graphical representations of the main clinical aspects. This is achieved through rendering dashboards with visualization of information related to patients, encounters, conditions, medications and procedures. These statistics can be queried per week, month or per year, depending the timeline managers want to consult.
- **Clinical Data Management.** It should be possible for Doctors to manage clinical history of their patients. This is achieved by having a full set of CRUD interfaces that allow registering and editing encounters, observations (test results), immunizations, allergies, care plans and medications
- **Provider Management.** It should be possible for Managers to keep the hospital's provider directory up to date. This feature is implemented through a management module with the ability to create, edit, delete or query providers using pagination and filtering.

Non-functional requirements

The non-functional requirements will cover the attributes of the system's quality that will ensure its stability, responsiveness, and ease of maintenance in real-life conditions.

- **Performance.** The system must stay fast even with millions of clinical records. Several mechanisms make that possible. Retrieval GET operations use server-side pagination, with a default page size that varies by resource (20 rows for patient sub-resources and provider encounters, 25 for the top-level patient, provider, and encounter listings, 8 for sequences). The dashboard's aggregate endpoints return precomputed totals rather than paginated rows. Expensive cluster information is cached in `JSONB` columns (`cluster_patient.features`, `cluster.detail_cache`), so it is never recomputed on the fly. Statistics come from PostgreSQL materialized views, pre-computed over more than 14 million observation rows. Startup indexes on the large tables (encounters, procedures) keep retrieval quick and prevent bottlenecks during complex queries.
- **Usability.** The interface must stay responsive and communicative even during heavy computation. To achieve this, all expensive operations, K-Prototypes on ≈22,000 patients and PrefixSpan on ≈4.9 million events, run in the background, outside the request handler. A two-second polling interval keeps the user informed of the pipeline's progress.
- **Maintainability.** The codebase must be modular, so it is easy to update, debug, and run per environment. The backend is split into four strict top-down layers (`api` → `service` → `repository` → `domain`), with no cross-layer imports. The frontend lives in one package alongside its own styles, and environment variables are used only for database and API connections.

4.2. Theoretical Foundation

Chapter 1 described the shift from paper to Electronic Health Records and the analytical value locked inside them. This section approaches that value from the technical side. Setting aside the legal barrier of data protection, treated later in this chapter, the

core obstacle is mathematical: a single patient's record mixes several kinds of events that no single model handles well. Clinical data is:

1. **Of mixed types:** integer (blood pressure, blood sugar), float (body mass index, hemoglobin, quality-adjusted life years), and string (diagnoses, diagnosis codes, allergies) values coexist in the same record.
2. **On very different scales:** variables span narrow ranges (e.g., $[0.4, 4]$ mIU/L for TSH) or wide ones (e.g., $[50, 500]$ mg/dL for cholesterol), so they must be rescaled before processing.
3. **Irregularly spaced in time:** a patient may have no records for months or years while another has many interventions within days, leaving interrupted, non-uniform timelines.

Because no single approach fits all of it, the solution combines several techniques, and this section is organized accordingly: it first develops clustering over mixed-type data, from the notion of distance up to the K-Prototypes algorithm, then sequential pattern mining up to PrefixSpan. The next section then assembles both, together with a large language model, into the methodology proposed by this thesis.

4.2.1. Clustering Algorithms and Mixed-Type Data Handling

Fundamentals of Clustering

Clustering is an unsupervised learning task. We are given a set of objects, each described by its attributes, and the goal is to split them into groups, called clusters. Objects in the same cluster should resemble each other, and objects in different clusters should differ as much as possible. The word "unsupervised" means no correct answer is available in the advance. The algorithm gets no labels. It must discover the structure of the data on its own. That is exactly the situation with hospital records, where nobody has marked which patients belong together.

Here, the population to be grouped is the set of hospital patients. The general population is varied, but the goal is to craft smaller cohorts of people who are highly similar. Two forces pull in opposite directions. Patients in the same cluster should look similar, so intra-cluster variance is minimized. Patients in different clusters should look different, so inter-cluster variance is maximized. In this way the complexity of the whole population is reduced to a handful of representative prototypes, which are far easier to analyze.

The whole process rests on one central idea: the *distance*, or equivalently the *similarity*, between two objects. A distance function $d(\cdot, \cdot)$ assigns a non-negative number to every pair of objects. Identical objects sit at distance zero, the measure is symmetric ($d(x, y) = d(y, x)$), and, for proper metrics, it respects the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$. Once this function is fixed, "similar patients" stops being vague and becomes a measurable quantity. The quality of any grouping can then be scored by comparing distances inside clusters against distances between them.

Most partitional clustering algorithms follow the same general mechanism, independently of the exact distance they use. First, a number k of clusters is fixed and k representatives are chosen. Then two steps alternate: an *assignment step*, in which every object is attached to its closest representative, and an *update step*, in which every representative is recomputed from the objects currently assigned to it. The two steps repeat until the assignments stop changing, at which point the partition is considered stable.

What varies from one algorithm to another is the definition of the representative, the distance function, and the way the iteration is initialized. These differences are examined in the following subsections.

Different Clustering Distances

The distance function decides which objects the algorithm treats as similar, so its choice is the single most important decision in clustering. The measures relevant to this work are grouped below by the type of data they support.

Numerical data. The standard measure is the Euclidean distance between two points x and y in an m-dimensional space:

$$d_E(x, y) = \sqrt{\sum_{l=1}^{m} (x_l - y_l)^2} \tag{4.1}$$

Its squared form underpins the K-Means family, where the optimal center is the arithmetic mean. This minimizes the sum of squared Euclidean distances, giving the update step a closed form. Two drawbacks come with it. First, it is scale-sensitive. An attribute in large units, such as cholesterol at $[50, 500]$ mg/dL, will dominate one in small units, such as TSH at $[0.4, 4]$ mIU/L, unless the data is normalized. Second, squaring the differences makes it sensitive to outliers. A steadier alternative is the Manhattan distance $d_M(x, y) = \sum_{l=1}^{m} |x_l - y_l|$, whose optimal center is the component-wise median. Both, though, are defined only when every attribute is a number.

Categorical data. Categorical attributes such as gender, diagnosis codes, or dominant care plan have no numerical magnitude. The natural measure is then the Hamming distance (simple matching dissimilarity), which counts the attributes on which two objects disagree:

$$d_H(x, y) = \sum_{l=1}^{m} \delta(x_l, y_l), \qquad \delta(a, b) = \begin{cases} 0 & \text{if } a = b \\ 1 & \text{if } a \neq b \end{cases} \tag{4.2}$$

Under this measure the best representative of a cluster is the *mode*, the most frequent category of each attribute, since it minimizes the total number of mismatches.

Mixed data. When quantitative and qualitative variables coexist, neither family above is defined on all features at once. The Gower distance [17] solves this by scoring each feature with a measure adapted to its type and avearging the contributions:

$$d_G(i, j) = \frac{1}{p} \sum_{m=1}^{p} d_m(i, j) \tag{4.3}$$

where the per-feature contribution is defined according to the feature type:

$$d_m(i, j) = \begin{cases} \dfrac{|x_{im} - x_{jm}|}{R_m} & \text{if feature } m \text{ is numerical} \\ \mathbf{1}[x_{im} \neq x_{jm}] & \text{if feature } m \text{ is categorical} \end{cases} \tag{4.4}$$

with $R_m = \max_i(x_{im}) - \min_i(x_{im})$ the range of the variable over the data set. Each $d_m(i, j)$ then lies in $[0, 1]$, and so does $d_G(i, j)$. The range normalization removes the scale-sensitivity of the Euclidean case, and the categorical term is just the per-attribute Ham-

ming indicator. This generality has a price, and the price is computational: Gower-based methods usually need the full $n \times n$ pairwise distance matrix, which grows quadratically with the number of patients.

These measures reappear later in the thesis. K-Prototypes combines the squared Euclidean and the matching term into a single dissimilarity function, while the Gower distance is the basis of the main validation metric for mixed-type partitions, presented in Chapter 6.

Available Clustering Algorithms

The most popular and most studied clustering algorithm is **K-Means** [18]. It partitions a domain into K predefined cohorts by minimizing the total within-cluster squared Euclidean distance. Given a dataset $X = \{x_1, x_2, \ldots, x_n\}$ of n instances over a purely numerical feature space $\mathbb{R}^m$, the **K-Means** objective is:

$$J_{km} = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \tag{4.5}$$

where:

- $\boldsymbol{J_{km}}$ is the total objective function value (the within-cluster sum of squared errors).
- $\boldsymbol{k}$ is the predefined number of clusters.
- $\boldsymbol{C_j}$ is the j-th cluster.
- $\boldsymbol{x_i}$ is a single data point (patient instance) assigned to cluster C_j.
- $\boldsymbol{\mu_j}$ is the centroid of cluster C_j, computed as the arithmetic mean of all instances assigned to it.
- $\boldsymbol{m}$ is the total number of dimensions (numerical features) in the dataset.
- $\boldsymbol{n}$ is the total number of instances in the dataset X.

Equation (4.5) is well defined only when the data is entirely numeric and continuous. The arithmetic mean that fixes each centroid simply has no meaning for categorical variables. Encoding categories as integers does not rescue it. **K-Means** then reads the codes as an ordinal ranking with real distances between them, distances they do not have. The resulting centroids are averages of integer codes. They match no plausible clinical profile, and they distort the cluster boundaries geometrically.

Beyond **K-Means**, other families rest on different structural ideas. **Hierarchical (agglomerative) clustering** [19] needs no preset cluster count. It starts with every object in its own cluster and repeatedly merges the two closest groups, building a tree of nested partitions. Because it only needs a pairwise distance matrix, it works with any measure, including the Gower distance of Equation (4.3). In theory, then, it handles mixed data. At hospital scale, though, two drawbacks are decisive: the pairwise matrix costs $O(n^2)$ in time and memory, and it leaves no prototype to summarize each cluster for later interpretation.

Density-based clustering, represented by **DBSCAN** [20], groups objects in crowded regions and marks isolated ones as noise. It finds clusters of any shape without a preset k. But it assumes a single global metric under which density is meaningful, and it degrades under the "curse of dimensionality" in the highly dispersed, high-dimensional space of encoded clinical records. **Model-based clustering**, such as the **Gaussian Mixture Model**, fits a continuous parametric distribution to each cluster, something categorical attributes simply cannot satisfy.

At the opposite end, **K-Modes** was designed for purely categorical databases. To avoid undefined centers, it represents each cluster by the mode instead of the mean. It also swaps the Euclidean objective for the Hamming distance of Equation (4.2), the count of mismatched categorical attributes between two vectors. On a mixed clinical database, though, it has to bucket every continuous measurement into categories, for example high versus low blood pressure. That approximation ignores the natural variation of numeric variables and throws away much of their information.

Accordingly, none of these algorithms alone meets the theoretical requirements of Electronic Health Records:

- **K-Means** fails mathematically on nominal text information.
- **K-Modes** distorts the geometry of numeric variables.
- **hierarchical clustering** scales quadratically and yields no prototypes.
- **DBSCAN** and the **Gaussian Mixture Model** impose assumptions that mixed clinical data cannot fulfill.

What is needed is a method that:

- provides a mathematically well-defined dissimilarity over both data types at once.
- offers a closed-form update for its cluster representatives.
- is guaranteed to converge.

K-Prototypes, detailed next, satisfies all three criteria, which is why it was selected for this work.

Mixed-Type Clustering and K-Prototypes

The **K-Prototypes** algorithm, proposed by Huang [3], extends both **K-Means** and **K-Modes**. It solves the problem with a single dissimilarity function that spans the whole feature space. Assume the features of each patient split into two groups: numerical features (N) and categorical features (C). We then write the feature vector of a sample x_i as $x_i = (x_i^N, x_i^C)$. A cluster prototype q_j is defined as $q_j = (q_j^N, q_j^C)$, where q_j^N is a numerical centroid and q_j^C is a categorical mode. The mixed dissimilarity between an instance x_i and a prototype q_j is:

$$d(x_i,\, q_j) = \underbrace{\sum_{l=1}^{|N|} \left(x_{il}^N - q_{jl}^N\right)^2}_{\text{numerical term}} + \gamma \underbrace{\sum_{l=1}^{|C|} \delta\left(x_{il}^C,\, q_{jl}^C\right)}_{\text{categorical term}} \tag{4.6}$$

where:

- $\boldsymbol{d(x_i, q_j)}$ is the total mixed-type dissimilarity between a single patient instance x_i and the cluster prototype q_j.
- $\boldsymbol{|N|}$ and $\boldsymbol{|C|}$ represent the total number of continuous numerical features and discrete categorical features in the dataset, respectively.
- $\boldsymbol{x_{il}^N}$ is the value of the l-th numerical attribute for the patient instance x_i.
- $\boldsymbol{q_{jl}^N}$ is the corresponding l-th numerical centroid value for the cluster prototype q_j.
- $\boldsymbol{x_{il}^C}$ is the value of the l-th categorical attribute for the patient instance x_i.
- $\boldsymbol{q_{jl}^C}$ is the corresponding l-th categorical mode value for the cluster prototype q_j.
- $\boldsymbol{\delta(x_{il}^C, q_{jl}^C)}$ is the simple matching dissimilarity function of Equation (4.2), applied per attribute. It acts as a binary indicator, returning 0 if the categories match

$(x_{il}^C = q_{jl}^C)$ and 1 if they differ $(x_{il}^C \neq q_{jl}^C)$.

- $\boldsymbol{\gamma}$ is the scalar weight factor $(\gamma > 0)$ added to make sure the discrete feature space is not overshadowed by the continuous feature space.

The mathematical aim in K-Prototypes is to find the clustering of patients that keeps the dissimilarity among them as low as possible over the whole population. The cost function which defines this procedure is given below:

$$J = \sum_{i=1}^{n} \sum_{j=1}^{k} w_{ij}\, d(x_i,\, q_j) \tag{4.7}$$

To minimize the total error J, the algorithm updates two elements. First, it optimizes the set of cluster prototypes $Q = \{q_1, \ldots, q_k\}$, the mathematical representative of every patient group. Second, it optimizes the assignment matrix $W = [w_{ij}]$, a binary matrix used to track patients' assignments, with $w_{ij} = 1$ when x_i belongs to q_j and $w_{ij} = 0$ otherwise.

Also, there is an additional requirement, known as a hard assignment condition, formulated in the following manner: $\sum_{j=1}^{k} w_{ij} = 1$. In medical practice, this requirement ensures that every individual patient is clearly assigned to precisely one cluster without any ambiguous overlaps or unassigned patients.

The Balancing Parameter $\boldsymbol{\gamma}$

The tuning parameter γ in Equation 4.6 sets how much weight the categorical part carries against the numerical part. Make γ very small and the numerical part dominates, over the categorical one. Make it very large and the opposite happens: the spread of the continuous features is ignored, and the algorithm collapses back into K-Modes.

Huang [3] ties the choice of γ to the scale of the numerical attributes, recommending a value proportional to their average standard deviation, with half that average as a typical setting:

$$\gamma = \frac{1}{2} \cdot \frac{1}{|N|} \sum_{l=1}^{|N|} \hat{\sigma}_l \tag{4.8}$$

where $\hat{\sigma}_l$ is the standard deviation of the l-th numerical attribute across the whole dataset. Tying γ to the average standard deviation keeps the squared numerical term and the categorical mismatch term on comparable orders of magnitude. Neither attribute type then dominates the dissimilarity just because of its units. The γ from Equation (4.8) is therefore data-dependent, adjusting to the scale of the clinical attributes used in clustering. This is Huang's recommended reference value, not a hand-tuned constant: in HospitAI γ was left to the automatic selection of the `kmodes` library (reported as `auto` in the benchmark of Chapter 6), which computes essentially this quantity, and was additionally varied empirically during that benchmark.

Prototype Update Rules and Convergence

To optimize J (cost function), which is formulated using (4.7), we can use an iterative process made of two main parts: the **assignment step** and the **update step**. The algorithm iterates between the two parts such that at each cycle, the overall clustering

error decreases to a point where patient groups become stable from a mathematical perspective.

In the **assignment step**, each object x_i gets assigned to the prototype q_j^* that yields minimal mixed dissimilarity.

$$j^* = \arg \min_{j \in \{1,\dots,k\}} d(x_i,\, q_j) \tag{4.9}$$

In the **update step**, prototypes are recalculated using their instances. In terms of the numerical component, the arithmetic average is the optimal option due to its optimal reduction of the squared Euclidean component:

$$q_{jl}^N \;\leftarrow\; \frac{1}{|C_j|} \sum_{i:\, w_{ij}=1} x_{il}^N \tag{4.10}$$

In the case of the categorical part, the best solution for updating is the mode, since it maximizes the simple matching dissimilarity:

$$q_{jl}^C \;\leftarrow\; \text{mode}\big(\{x_{il}^C : w_{ij} = 1\}\big) \tag{4.11}$$

Each full iteration, assignment then update, is guaranteed not to increase J, because each step alone optimizes the objective with respect to its own variables. The series $J^{(1)} \geq J^{(2)} \geq \cdots$ is therefore monotonically non-increasing and bounded below by zero, so it converges to a local optimum of Equation (4.7).

Initialization Strategies: Huang vs. Cao

A well-known weakness of all k-partitional methods is their sensitivity to the initial centroids. Pick the prototypes poorly, and the iteration slides into a poor local minimum, far from the global one. Random initialization is especially weak on mixed-type clinical data: a prototype dropped into a sparse region can end up with no nearby points, leaving empty clusters as the iteration runs.

There are two prominent initialization techniques for the K-Prototypes algorithm proposed in literature:

- **Initialization by Huang [3]:** Introduced with the algorithm itself, this approach is frequency-based. For the categorical components (q_j^C) it counts each category and picks the most common ones. The numerical component (q_j^N) is usually set from local means. It is fast, but from a clinical angle it has a flaw: the frequency counts are done independently, attribute by attribute. So the algorithm can start from fictitious patient profiles, building centroids out of high-frequency attributes that never actually coexist in a real clinical record. It also gives no guarantee that the centers are geometrically separated.
- **Initialization by Cao [21]:** To fix that geometric problem, the authors proposed a density-based initialization. Instead of building artificial prototypes, it picks real records from the dataset. It computes the co-ocurrence density of the records and takes the densest one as the first prototype. Each later prototype is chosen for both its local density and its distance from all prototypes already selected.

The Cao approach [21], built on density and distance, is often called the theoret-

ically better choice for mixed data. Theory, though, does not always win in a specific clinical setting. EHR data is very high-dimensional, sparse in categories, and clinically skewed, so an empirical check beats blind trust in the theory.

So instead of assuming either scheme is better, this work adopts the frequency-based Huang initialization and checks it directly, through the internal validation procedures of Chapter 6. Cao initialization was tested too. Across the runs, Huang gave consistently better internal validation metrics. On the Synthea data it produced more stable, non-degenerate clusters, which is why it drives the production K-Prototypes runs.

4.2.2. Sequential Patterns Mining

Fundamentals of Sequence Mining

Static clustering divides patients into groups with similar clinical characteristics. But a cluster is only a snapshot. It captures who a patient was, not how the patient got there. To recover the time order, and the clinical processes that recur inside each subgroup, we have to look at the sequence of events. And this is the job of sequential pattern mining.

Sequential pattern mining looks for subsequences of events that appear, in the same order, in a large enough share of a collection of sequences. In healthcare, that collection holds one sequence per patient. A discovered pattern like *diagnosis* $\rightarrow$ *medication* $\rightarrow$ *procedure* then describes a care pathway many patients have followed. The rest of this subsection fixes the notation and two central notions: the support of a pattern and its anti-monotonicity.

A patient's history is modeled as an ordered sequence of clinical events. Each event carries a type prefix (`C`: condition, `M`: medication, `P`: procedure, `CP`: care plan) and a description:

$$S = \langle e_1, e_2, \ldots, e_n \rangle, \qquad t_1 \leq t_2 \leq \cdots \leq t_n \tag{4.12}$$

Events that take place on the same calendar day are linearized in a deterministic chronological way, so the mining process works on simple sequences of singletons. For a sequence $\alpha = \langle a_1, \ldots, a_r \rangle$, the definition $\alpha \sqsubseteq S$ implies the existence of indices $i_1 < i_2 < \cdots < i_r$ such that $a_j = e_{i_j}$. Gaps are allowed between matched events, reflecting the fact that clinical encounters are separated by other activity.

For a sequence database $\mathbb{S} = \{S_1, \ldots, S_{|\mathbb{S}|}\}$ and support threshold σ, the support of a pattern α corresponds to the number of sequences that include it,

$$\sup(\alpha) = \big|\{\, S_i \in \mathbb{S} \mid \alpha \sqsubseteq S_i \,\}\big|, \tag{4.13}$$

and α is called *frequent* whenever $\sup(\alpha) \geq \sigma \times |\mathbb{S}|$. All the frequent patterns are collected in the set $\mathcal{F}_\sigma$. Support is said to be *anti-monotonic*, since

$$\sup(\beta) \leq \sup(\alpha), \tag{4.14}$$

for $\alpha \sqsubseteq \beta$. Anti-monotonicity is what allows pruning of the search space and is the basis of every efficient mining approach: if a pattern is not frequent, then no extension of it can be frequent either, so the whole branch can be discarded.

Available Sequence Mining Algorithms

The mining algorithms proposed in the literature differ mainly in how they organize the search for frequent patterns, and this structural difference determines their behavior on large clinical databases.

The first family follows the *generate-and-test* strategy inherited from association-rule mining. The earliest sequential miners build candidates level by level: they find all frequent length-one patterns, combine them into length-two candidates, check each candidate against the whole database, then repeat one length at a time. Later variants add time constraints and taxonomies but keep the same skeleton. They all share two structural costs: the candidate set can grow exponentially with the event alphabet, and the whole database is re-scanned at every level. On an EHR dataset with hundreds of unique event codes, that quickly becomes impractical.

A second family takes a different route, storing the data in a *vertical* format. For every event, an id-list records the sequences and positions where it occurs, and longer patterns are built by joining id-lists rather than re-scanning the database. Counting is then fast. But the id-lists of the intermediate candidates have to stay in memory, and for long sequences over a large clinical alphabet that overhead becomes severe.

PrefixSpan [4] drops candidate generation entirely for a *pattern-growth* strategy. Instead of proposing candidates and testing them, it grows patterns straight from the data. Each recursive step is restricted to the part of the database that can actually extend the current pattern, the so-called projected database. Two properties follow. First, every pattern it counts is guaranteed to exist in the data, so no effort is wasted on candidates that will fail. Second, the projected databases shrink as the patterns grow, so the costly early stages run on progressively smaller inputs. Table 4.1 summarizes the comparison.

Approach	**Search strategy**	**Database access**	**Main weakness**
Candidate generation	Level-wise generate-and-test	Full scan at every level	Exponential candidate growth
Vertical id-lists	Id-list joins	Id-lists kept in memory	Severe memory overhead for long sequences
PrefixSpan [4]	Pattern growth via prefix projection	Shrinking projected databases	Cost of building projections (mitigated by pseudo-projection)

Table 4.1: Comparison of the main sequential pattern mining approaches.

For the clinical data here, the choice follows straight from this comparison. The size of the event alphabet rules out the generate-and-test family, and the memory profile of the vertical-format family on long patient histories rules it out too. PrefixSpan generates no candidates, and its projected database only ever shrinks as the pattern grows, so effort drops off exactly where the search space would otherwise explode. Therefore, PrefixSpan is the algorithm adopted for trajectory mining in this thesis.

The PrefixSpan Algorithm

The PrefixSpan algorithm [4] generates no candidates. It extends patterns step by step, and prunes the search space by looking only at the suffixes that can follow a given

prefix, inside a *projected database*. For any prefix α,

$$\mathbb{S}|_\alpha = \left\{ S' \mid \exists S \in \mathbb{S} : \alpha \sqsubseteq S \text{ and } S' \text{ is the suffix of } S \text{ after the first occurrence of } \alpha \right\}. \tag{4.15}$$

Mining begins with the empty prefix, mines all frequent length-1 items, performs projection, and recurses. Since the size of $\mathbb{S}|_\alpha$ is non-increasing as α is extended, less effort is needed on each recursion call as the pattern lengths grow. The correctness of the prefix decomposition ensures that the algorithm completely generates $\mathcal{F}_\sigma$, without any repetitions or missing items.

4.3. AI-Driven Patient Phenotyping and Pathway Discovery

The previous section presented the theory behind clustering and sequential pattern mining. This section assembles them, together with a large language model abstraction, into the methodology proposed by this thesis.Everything here stays at the conceptual and mathematical level. The concrete software design and implementation details are the subject of Chapter 5. A proven way to manager the complexity of healthcare data is to organize the analysis as a machine-learning pipeline. That is the shape the proposed methodology takes.

4.3.1. Formal Problem Definition

The proposed system models the hospital population as a finite set $P = \{p_1, p_2, \ldots, p_n\}$. Every individual patient will be modeled using two different mathematical structures:

1. A feature vector $p_i = \langle x_i^{(1)}, \ldots, x_i^{(m)} \rangle$, that belongs to the combined feature space, containing both numerical and categorical features.
2. A temporal event sequence $S_i = \langle e_{i,1}, e_{i,2}, \ldots, e_{i,l_i} \rangle$, indicating a strict chronological order of events, as formalized in Equation (4.12).

On top of this double representation, the problem solved by the platform decomposes into three connected tasks:

1. **Stratification:** find a partition of P into k homogeneous groups by minimizing the mixed-type objective of Equation (4.7) over the feature vectors p_i.
2. **Trajectory extraction:** for each discovered group, mine the set of closed, frequent sequential patterns of Equation (4.20) from the event sequences S_i of its members, and rank them with the interestingness measure of Equation (4.21).
3. **Interpretation and access:** translate both the statistical outputs (prototypes, patterns, supports) and the users' natural-language questions into a form that non-technical medical staff can work with, without ever allowing the generative component to modify or invent data.

Task 1 uses the K-Prototypes theory already presented, together with the feature preparation described next. Task 2 runs PrefixSpan on the event sequences and then compresses, ranks, and de-duplicates its patterns. Task 3 is developed at the end of this subsection.

4.3.2. Conceptual Pipeline Architecture

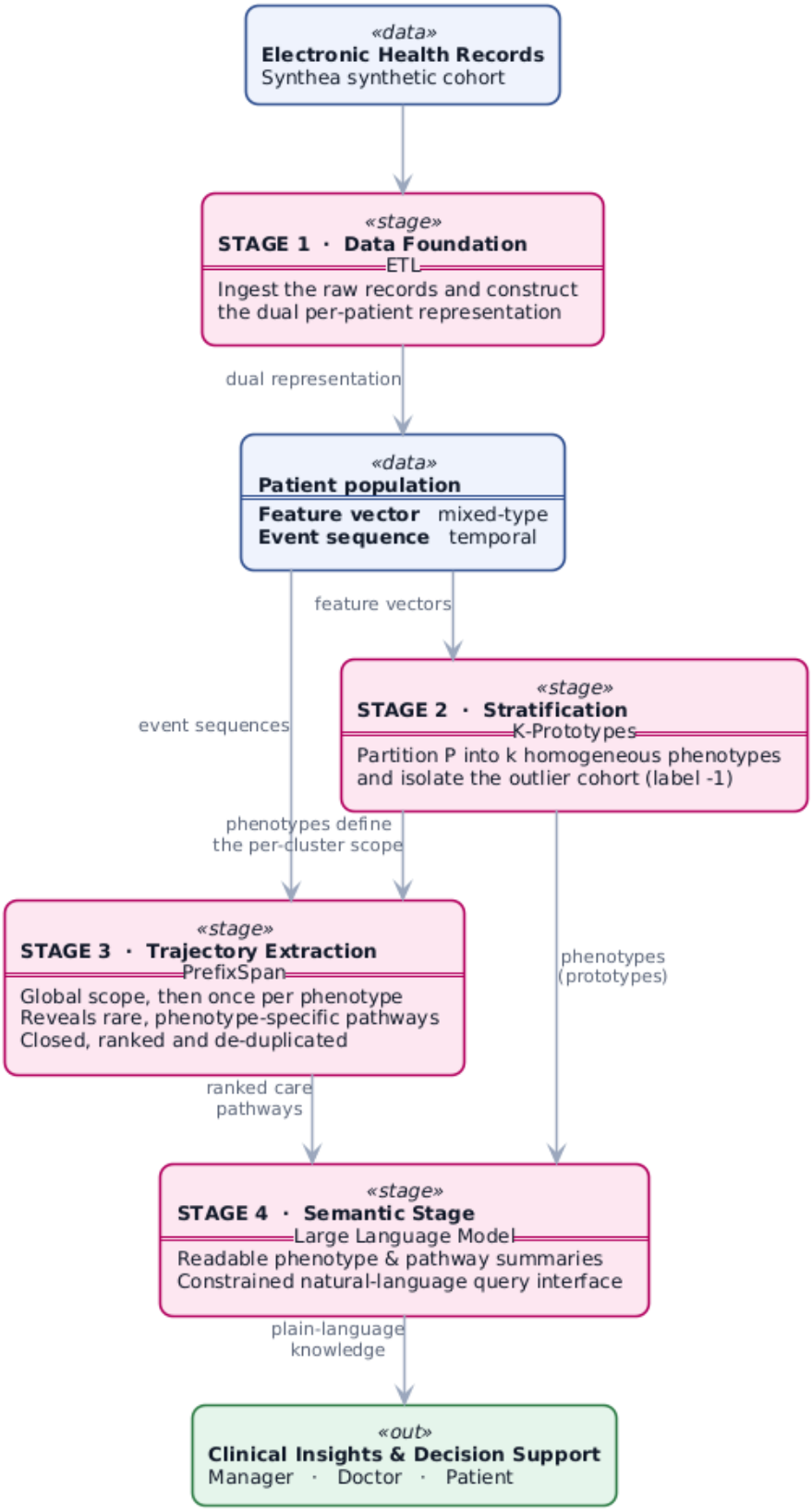


Figure 4.2: Conceptual architecture of the analytical pipeline.

The final shape of the pipeline architecture can be seen in Figure 4.2. The pipeline consist of four stages:

- **Stage 1:** Ingests the raw records and turns them into the dual patient representation, the feature vectors and event sequences that the three analytical stages consume.
- **Stage 2:** Partitions the population and, at the same time, reduces its noise.
- **Stage 3:** It firstly runs with a global scope (across all patients). Then it runs once for each cluster, so the pathways it extracts are tied to the phenotype they belong to, not averaged over the entire hospital. In this way, specific sequences are not drowned out, meaning more rare and special patterns can be found.
- **Stage 4:** The Semantic Stage sits on top of both outputs. It produces readable summaries of the clusters and patterns, and it offers a common natural-language interface for querying all the stored results.

The output of each stage is the input of the next: the data-preparation stage feeds the three analytical techniques, clustering, sequence mining, and language-model interpretation, and that chaining is precisely what turns them into a single analytical pipeline.

4.3.3. Feature Scalers and Outlier Handling

Before clustering, the numerical attributes go through two preparation steps: outlier isolation and feature scaling.

First, extreme values are identified using the **z-score**:

$$z = \frac{x - \mu}{\sigma} \tag{4.16}$$

where μ is the mean and σ is the standard deviation of the feature. This isolation happens *before* scaling and clustering. It ensures that extreme clinical cases are removed from the main dataset so they do not distort the geometry of the features or pull the cluster prototypes toward themselves. However, because these outliers carry important clinical information, they are not permanently deleted. Instead, they are set aside into a dedicated cohort to be evaluated independently during the final interpretation stage.

Second, the cleaned numerical features are normalized. Because the K-Prototypes distance is highly scale-sensitive, an attribute measured in large units would otherwise dominate one measured in smaller units. Three scalers are commonly considered:

- **Standardization** (z-score normalization) gives the feature a zero mean and unit variance. It suits roughly bell-shaped distributions but remains vulnerable to outliers:

$$x' = \frac{x - \mu}{\sigma} \tag{4.17}$$

- **Min-Max scaling** compresses the feature exactly into a bounded range, typically $[0, 1]$, preserving the original shape of the distribution:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{4.18}$$

- **Robust scaling** replaces the mean and standard deviation with the median and the

interquartile range (IQR), making it resistant to extreme values:

$$x' = \frac{x - \text{median}}{\text{IQR}} \tag{4.19}$$

This work implements **Min-Max scaling**. This approach forces every continuous feature onto the exact same $[0, 1]$ interval as the categorical mismatch penalty (Hamming distance). The method's vulnerability to extreme boundaries is neutralized by the prior outlier isolation step.

4.3.4. Pattern Reduction, Ranking, and Selection

PrefixSpan returns the full set of frequent patterns $\mathcal{F}_\sigma$, but that raw set is huge and highly redundant. Before the patterns are useful, three things have to happen: the set is compressed to its closed patterns, the survivors are ranked by how distinctive they are, and near-duplicates are filtered out. Assembling this selection stage on top of the raw mining is a contribution of this thesis, and the three steps are developed below.

Closed Patterns

Enumerating every pattern in $\mathcal{F}_\sigma$ is a poor strategy. By anti-monotonicity, all the subsequences of a frequent pattern are themselves frequent. So a single trajectory of length L alone yields $2^L - 1$ other patterns in $\mathcal{F}_\sigma$. The usual fix is to report only the patterns that are *closed*. A pattern α in $\mathcal{F}_\sigma$ is closed if there is no:

$$\beta \sqsupset \alpha \;\text{ such that }\; \sup(\beta) = \sup(\alpha). \tag{4.20}$$

The set $\mathcal{C}_\sigma \subseteq \mathcal{F}_\sigma$ of closed patterns is a *lossless compression* of $\mathcal{F}_\sigma$. For every $\alpha \in \mathcal{F}_\sigma$ there is a $\beta \in \mathcal{C}_\sigma$ with $\alpha \sqsubseteq \beta$ and $\sup(\alpha) = \sup(\beta)$, so nothing is lost. In clinical terms, a closed pattern is the most detailed trajectory shared by a patient group.

Statistical Re-ranking by Geometric Mean Lift

Closure removes structural redundancy, but not semantic redundancy. The highest-support patterns are just combinations of individually common items, so raw support keeps surfacing uninteresting interactions. To rank patterns by how distinctive they are, the closed patterns are rescored against independence, using a geometric-mean lift:

$$\text{gml}(\alpha) = \frac{\sup(\alpha)/|\mathbb{S}|}{\left(\prod_{i=1}^{m} \sup(\langle v_i \rangle)/|\mathbb{S}|\right)^{1/m}}, \tag{4.21}$$

where:

- $\boldsymbol{\alpha}$ is the evaluated sequential pattern.
- $\mathbf{sup}(\boldsymbol{\alpha})$ is the absolute support (frequency of occurrence) of the pattern in the dataset.
- $|\mathbb{S}|$ is the total number of sequences in the database $\mathbb{S}$.
- $\boldsymbol{m}$ is the length of the pattern, representing the number of individual items it contains.
- $\mathbf{sup}(\langle \boldsymbol{v_i} \rangle)$ is the support of the i-th individual item v_i treated as a singleton sequence.

In simpler terms, this metric compares how often a pattern actually occurs against how often it would be expected to appear by pure chance. If the clinical events in a sequence were entirely unrelated, their combination would be purely coincidental. The geometric mean lift measures how far the data deviates from this random baseline, effectively isolating genuine clinical relationships from random noise.

Diversity Constrained Selection

Even after re-ranking, the results can hold near-duplicates: two patterns that differ by a single item, or the same clinical events in a different order. To keep the result set $\mathcal{R}$ diverse, we run a greedy filter based on Jaccard similarity over the unordered item sets, which removes these structural permutations. For patterns α and β with item sets A and B, the Jaccard similarity is:

$$J(\alpha, \beta) = \frac{|A \cap B|}{|A \cup B|}. \tag{4.22}$$

Candidates are sorted by geometric-mean lift, in descending order. A pattern α enters $\mathcal{R}$ only if it stays below the diversity threshold against every already-accepted β, that is $J(\alpha, \beta) < \tau$. Any pattern too similar to an accepted one is dropped.

4.3.5. Bridging the Semantic Gap via Large Language Models

The clustering and sequence-mining models from earlier in this chapter produce precise mathematical output: prototype vectors, support values, ordered events. None of it can be used directly in clinical practice. This subsection describes the two components that bridge the gap with a large language model. The first is a directed-graph agent that turns natural-language queries into safe SQL. The second is a verbalized prompt that turns raw data into clinical narrative.

Problem Statement

The critical aspect of this automatic process is that the natural language input (Q_{nl}) should be mapped to the safe and read-only database query (Q_{safe}):

$$T(Q_{nl}) = Q_{safe} \tag{4.23}$$

The query can only include read-only commands, to avoid any harmful actions on the clinical database by the agent.

Using the LLM on its own is not enough. LLMs are stochastic, and they can hallucinate tables that do not exist or issue destructive writes. The model therefore has to sit inside a strict, deterministic control loop.

The Agent as a Directed Graph

The LLM agent is built as a directed graph. Its vertices are the tasks to perform, and its edges are the deterministic order in which they run. So even when the LLM generates the content, the control path is set by the code.

The state Ψ passed between the nodes must be part of the below tuple:

$$\Psi = \langle\, Q_{nl},\ \Sigma,\ \rho,\ Q,\ R,\ E_m,\ k \,\rangle, \tag{4.24}$$

where Q_{nl} denotes the user input, Σ denotes the schema of the database, $\rho \in \{\text{DB}, \text{MEDICAL}, \text{OTHER}\}$ denotes the classification label, Q denotes the generated query, R denotes the retrieved data, E_m denotes the error feedback, and k denotes the iteration number.

The vertex set $V = \{\, v_{\text{in}},\ v_{\text{med}},\ v_{\text{gen}},\ v_{\text{val}},\ v_{\text{exec}},\ v_{\text{end}} \,\}$ defines the operational states:

- v_{in} (*classification*): The LLM classifies Q_{nl} into one of three categories (DB, MEDICAL or OTHER) and rewrites a database question (if it is the case) for clarity
- v_{med} (*medical answer*): Generates a simple medical response without querying the database
- v_{gen} (*query generation*): Drafts a candidate SQL query Q based on the prompt, schema, and any previous error feedback E_m.
- v_{val} (*validation*): Evaluates Q against strict criteria: syntactic validity, schema compliance, and read-only safety.
- v_{exec} (*execution*): Runs the validated query against the database to capture the result set R. Any runtime error is treated as a validation failure.
- v_{end} (*termination*): Summarizes R into natural language for the user and ends the process.

From the entry node v_{in}, the control flow branches deterministically based on the assigned classification label:

$$v_{\text{in}} \longrightarrow \begin{cases} v_{\text{gen}} & \text{if } \rho = \text{DB}, \\ v_{\text{med}} & \text{if } \rho = \text{MEDICAL}, \\ v_{\text{end}} & \text{if } \rho = \text{OTHER}. \end{cases} \tag{4.25}$$

In database query processing, the bounded retry cycle includes generation, validation, and execution ($v_{\text{gen}} \to v_{\text{val}} \to v_{\text{exec}}$). When validation fails or an error occurs during execution, the particular error message gets placed in E_m, the value of k is increased, and the flow proceeds to v_{gen}.

For the bounded retry cycle to terminate, there must be a successful query result or k reaches its maximum value $k_{\max}$:

$$\text{exit cycle} \iff f_{\text{val}}(Q, \Sigma) = \text{True} \quad \vee \quad k \geq k_{\max} \tag{4.26}$$

LLM Interpretation of Analytical Output

The second LLM function takes the numeric output of the clustering and pattern mining processes and generates a summary in a form readable by clinicians. To prevent hallucination, the LLM can be used only as a *verbalizer* for deterministic inputs. No invention or computation of any statistics is allowed.

Each cluster is then provided as input to the LLM through a pre-computed feature pack:

$$F_c = \langle\, \mu_c,\ \pi_c,\ \delta_c,\ \tau_c,\ \omega_c \,\rangle, \tag{4.27}$$

where:

- $\boldsymbol{\mu_c}$ aggregates continuous clinical magnitudes (such as age and lab values).

- $\boldsymbol{\pi_c}$ denotes the prevalences of categorical attributes.
- $\boldsymbol{\delta_c}$ captures the demographic distributions.
- $\boldsymbol{\tau_c}$ signifies the distributional mixes over ordinal risk strata.
- $\boldsymbol{\omega_c}$ contains the top frequent clinical events.

This tuple encodes all necessary statistics about the cluster. The LLM's task is to identify the most discriminative signals within F_c and phrase them into a natural and accurate clinical analysis.

When this exact same principle is applied to the sequences, the output of the LLM process becomes strictly constrained. Use of deterministic inputs ensures high fidelity of the results produced.

Taken together, the two constrained roles of the language model close the loop opened by the formal problem definition. The pipeline stratifies the population, extracts the care pathways of each stratum, and hands both results to the user in plain language.

Chapter 5. System Design and Implementation

Following the theoretical framework from Chapter 4, this chapter present in detail the system's design and implementation. The first part, System Design 5.2, outlines the conceptual architecture, the database schema, and the internal structure of the backend and frontend. The second part, System Implementation 5.2, presents the technical execution. It uses concrete code listings to explain the libraries, algorithmic logic, and the complete workflow, from the initial processing of patient data to the delivery of clinical insights.

5.1. System Design

5.1.1. System Architecture

Figure 5.1 presents the high-level architecture of the platform. Four kinds of components cooperate: a **Frontend** that renders the interface and cosnumes data, a **Backend** that exposes the API and hosts the analytical logic, a **Database** that persists both the raw clinical records and every analytical result, and a small set of **external services** that feed the pipeline or extend it with generative capabilities.

The **Frontend** is a single **Next.js** application. It never talks to the database or to any external service directly. Every piece of data it renders, patient records, cluster dashboards, sequence dashboards, statistics, and the pipeline management page, is fetched over REST from the Backend.

The **Backend** is a single **FastAPI** process with three cooperating parts. The **REST API** (Section 5.1.4) is the only entry point the Frontend is allowed to use. The **analytics pipeline** is an in-process chain of four stages: **ETL ingestion** (Section 5.2.2.1), K-Prototypes **clustering** (Section 5.2.2.2), PrefixSpan **sequence mining** (Section 5.2.2.3), and **AI-driven interpretation** (Section 5.2.2.5). This chain is formalized in Section 4 and orchestrated as a single job in Section 5.2.2.4. The **LLM SQL Agent** (Section 5.2.2.6) translates natural-language questions into safe, read-only SQL. All three parts share the same process, connection pool, and domain models.

The **Database** is a single **PostgreSQL** instance, detailed in Section 5.1.2: eleven relations are populated once by the ETL step from the Synthea dataset, and the rest record the junctions and analytical outputs of the pipeline.

Finally, the platform relies on a few **external services**. The Synthea generator produces the source dataset offline, and the LLM provider, OpenAI, sits behind the interpretation stage and the SQL Agent. The monthly re-triggering that keeps the clusters and mined pathways current as the data evolves is handled internally, by the in-process scheduler shown inside the Backend (Section 5.2.2.4), not by any external service.

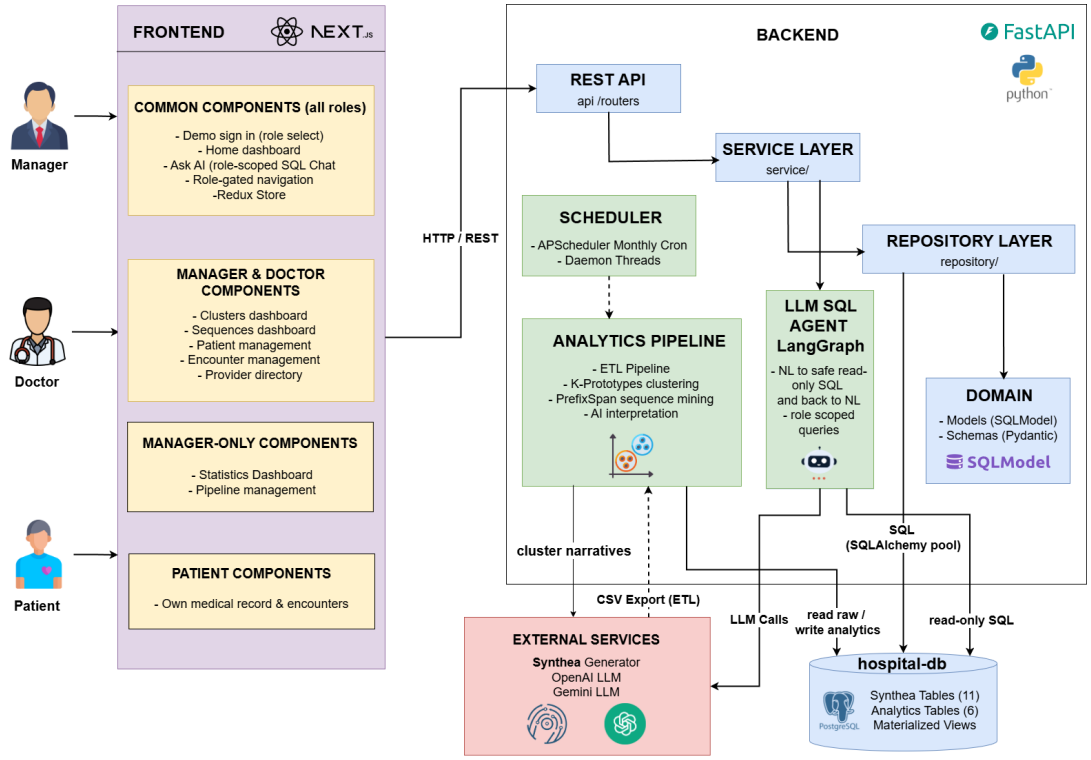


Figure 5.1: System Architecture Diagram

Why a Monolithic, Client-Server Architecture

The platform is built as a single **monolithic** application rather than a set of distributed microservices. This is deliberate: the object of this thesis is not a production-grade, independently-scalable web platform, but the analytics pipeline itself (Section 4). A microservice split woudl add network hops, service discovery, a message broker, and duplicated deployment infrastructure, none of which improves the clustering, the mined patterns, or the LLM narratives. A single backend sharing one database engine and one set of domain models removes that overhead without giving up any capability the platform needs.

FastAPI was chosen because it lets Python serve the REST API directly. Python is the language of the whole machine-learning stack used here (`scikit-learn`, `kmodes`, `prefixspan`, `pandas`), so the clustering and mining modules are called in-process rather than over the network. **PostgreSQL** and **Next.js** are justified in Sections 5.1.2 and 5.1.3. Long-running and scheduled work, such as the monthly analytics job, runs inside the same backend process through an in-process scheduler, so no external worker or message queue is needed.

5.1.2. Database Design

A relational database was chosen over a NoSQL store because the domain is inherently relational. Every clinical event belongs to exactly one patient and, usually, one encounter. The analytical modules constantly join across these events, for instance to build one chronologically ordered sequence per patient out of four event tables (Section 5.2.2.3). A document store would either duplicate that context in every event document or push

the joins into application code, giving up the transactional consistency and query optimizer PostgreSQL provides. Since the platform reads far more than it writes, this was judged more valuable than the write-scalability a NoSQL store would offer instead.

The schemas below show eighteen tables split into two groups. The *normalized raw-data relations* mirror the ingested Synthea dataset (Figure 5.2). The *analytics relations*, produced and consumed by the pipeline, are shown in Figure 5.3.

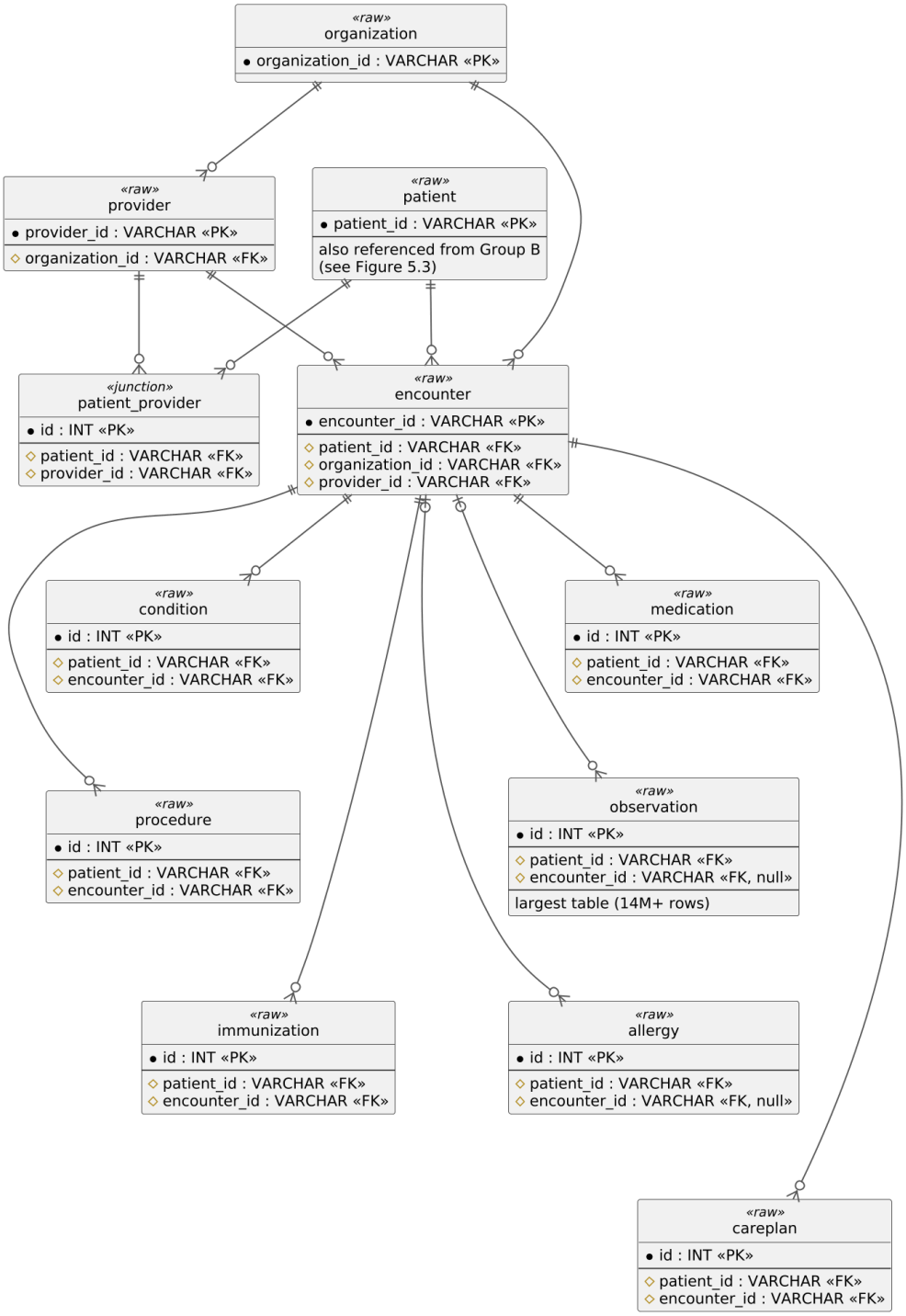


Figure 5.2: Entity–Relationship diagram of the database schema (1)

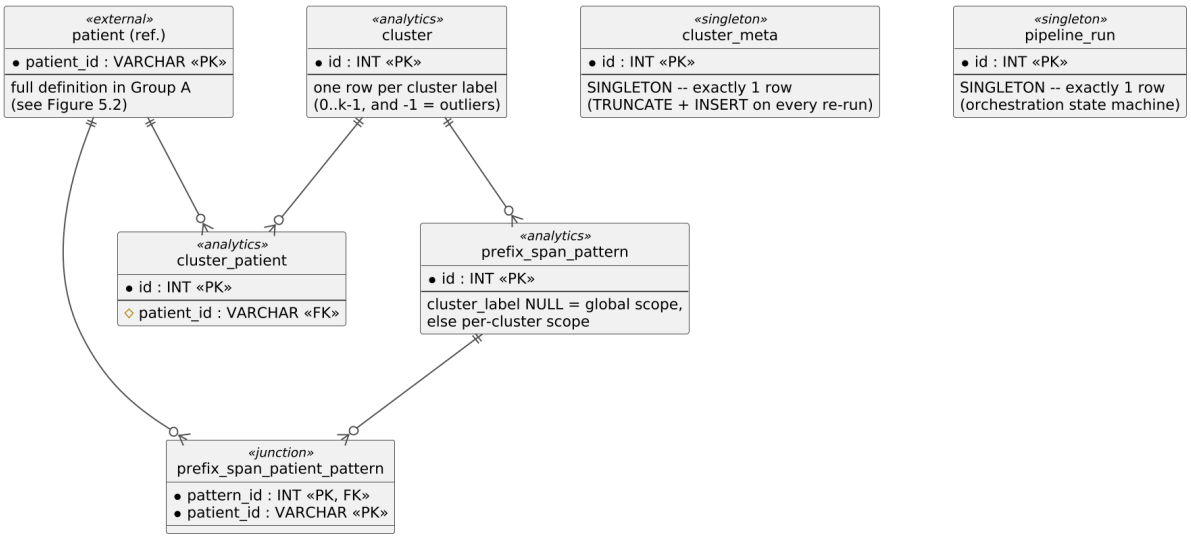


Figure 5.3: Entity–Relationship diagram of the database schema (2)

Normalized Raw-Data Relations

Eleven tables correspond one-to-one to the ingested Synthea CSV files. Four are core entities: `patient`, `organization`, `provider`, and `encounter`. `encounter` is the hub, referencing both `patient` and `provider`. `provider` maintains a reference to `organization`. The remaining seven, `condition`, `medication`, `procedure`, `observation`, `immunization`, `allergy`, and `careplan`, are clinical-event tables. Each one references the patient it belongs to and, where relevant, the encounter. The single many-to-many relationship, between patients and providers, is resolved through the `patient_provider` junction table, materialized during ETL from the distinct pairs found in `encounter`.

Analytics Relations

Six further tables, not part of the raw dataset, store the pipeline's outputs and its own state. `cluster_meta` is a singleton holding the metadata of the current clustering run (algorithm, feature lists, parameters, metrics, plots). A re-run replaces its single row. `cluster` holds one row per cluster label (the k clusters plus the -1 outlier group) with the LLM-generated card content. `cluster_patient` assigns each patient to a cluster and caches its features and 2-D projection coordinates in a JSONB column. `prefix_span_pattern` stores each mined pattern with a nullable `cluster_label` (`NULL` for the global run, a label for a per-cluster run), and `prefix_span_patient_pattern` is its patient junction. `pipeline_run` is the singleton state machine of the orchestration job (Section 5.2.2.4).

Two conventions apply here. Free-form results (feature packs, LLM narratives, run metrics) live in `JSONB` columns rather than rigid columns, so the analytics tables stay compact and their shape can evolve between runs without a migration. And the only foreign key linking the analytics group back to the raw group is `cluster_patient.patient_id`. The rest are logical joins left unlinked, since the analytics tables are truncated and repopulated on every pipeline run.

Integrity Constraints and Indexing

Every table has its own primary key, and foreign keys such as `encounter.patient_id` and `encounter.provider_id` enforce referential integrity across the raw-data group. Indexes target concrete access patterns rather than being added randomly. B-tree indexes on the `description/start_date` columns speed up the statistics filters and the sequence-construction queries of Section 5.2.2.3. Trigram `GIN` indexes on `observation.description` and `condition.description` back the SQL Agent's free-text search. And `cluster_label/is_outlier` are indexed on `cluster` and `cluster_patient`, because every cluster-scoped dashboard read filters on them.

`observation`, the largest table with over fourteen million rows, is also pre-aggregated into five materialized **views** (`mv_obs_totals`, `mv_obs_per_patient`, `mv_obs_top_types`, `mv_obs_numeric_stats`, `mv_obs_categorical_stats`). This way, the statistics dashboard reads pre-computed totals instead of aggregating millions of rows per request. These are rebuilt at backend startup and refreshed after every ETL reload.

Normal Form and Its Limits

The raw-data group satisfies the third normal form (3NF): every non-key attribute depends on whole key and, with the two exceptions below, on nothing else. First, a few columns are computed and stored rather than derived on demand (patient age, medication/careplan duration, provider/organization counters). A stored `age` is strictly transitively dependent on the non-key `birthdate`, which 3NF forbids. But recomputing it on every query would be far costlier than the redundancy. And it cannot drift, since it is written once at ETL time.

Second, the analytics tables keep several attributes inside JSONB columns instead of one column each, a departure from first normal form's atomicity requirement. This is accepted because those tables are regenerated on every run. Their shape must evolve freely across the 63 clustering configurations of Section 5.2.2.2. A rigid, one-column-per-feature schema would not allow that without a migration per experiment.

5.1.3. Frontend Design

Next.js (App Router) was chosen for the Frontend, written in React and TypeScript. Its file-system routing and mature UI ecosystem let the interface be built directly around the platform's REST API, without any extra client-side routing framework. Routing splits the app into two groups: `(auth)` for the demonstration sign-in and `(main)` for the authenticated area, nesting the layouts of `/clusters`, `/sequences`, `/statistics`, `/patients`, `/pipeline`, and the other pages. After signing in, each user reaches only the pages their role exposes: a Patient sees their own records, a Doctor works with patient and encounter management as well as the cluster and sequence analytics, and a Manager additionally reaches the manager-only views, namely the hospital statistics and the pipeline management page. As established in Chapter 4, this client-side role gating is a demonstration of per-role UX, not a security boundary.

Pages are composed of small, reusable components that read from and dispatch to a central store rather than fetching data themselves. This way, the same table, chart, and card components are reused across the patient, cluster, and sequence pages. Server data lives in **Redux Toolkit** slices, one per domain, each pairing asynchronous thunks (the REST calls) with a reducer that shapes the fetched data and its loading/error state.

No component calls `fetch` directly, so every chart is guaranteed to render exactly what the Backend returned.

Responsiveness follows the same discipline. Layout, spacing, and breakpoints come from a shared Emotion-based system, `theme.ts` plus the `useClasses` hook, rather than being hard-coded per page. A single theme change therefore propagates to every table, scatter plot, and dashboard that uses it.

5.1.4. Backend Design

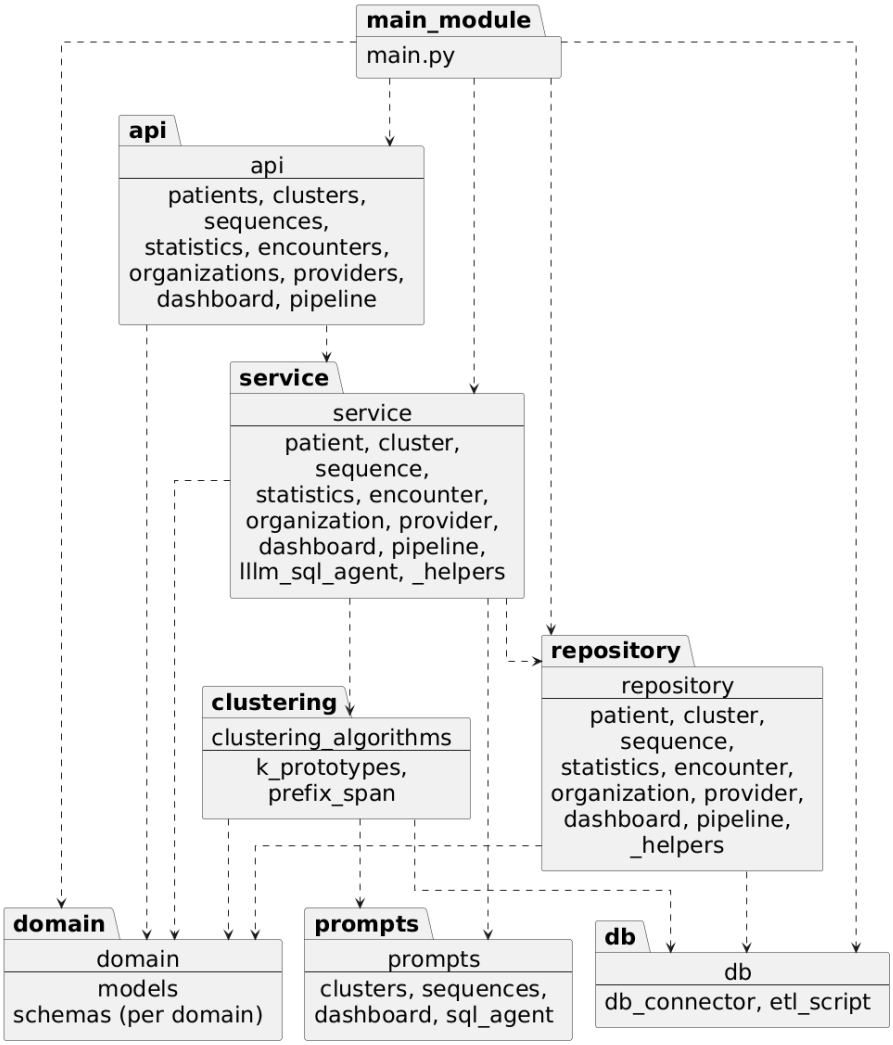


Figure 5.4: Backend package diagram

As introduced in Section 5.1.1, the Backend is a single FastAPI process with three cooperating parts: the REST API, the analytics pipeline, and the LLM SQL Agent. All three are built on the same strict, top-down layered architecture, shown in Figure 5.4. No cross-layer imports are allowed, which localizes the impact of any change. The **presentation layer** (`api/` routers, aggregated in `api/__init__.py` and registered in `main.py`) sits on top. Below it, the **business layer** (`service/`) validates requests and delegates to the clustering, mining, and SQL-Agent modules. The **persistence layer** (`repository/`)

owns all database access. And the **domain layer** (`domain/models` - SQLModel tables and `domain/schemas` -Pydantic models) sits at the bottom.

REST API

The presentation layer is a REST API, not GraphQL. Each page needs one shaped resource, such as a cluster overview or a patient record, not an arbitrary client-chosen graph of fields. A query language's added complexity would not pay off here. Endpoints are grouped by domain (`/patients`, `/clusters`, `/sequences`, `/statistics`, `/pipeline`, the SQL-Agent chat endpoint), each exposing listings, a detail view, and, where relevant, an action endpoint such as `POST /pipeline/run`. Every request/response body is a Pydantic schema in `domain/schemas`, so FastAPI validates and rejects malformed input before the business layer is reached, with no hand-written checks in the endpoint itself.

Request Processing and the Role of the Pipeline and the SQL Agent

Two processing modes coexist behind the same API. Ordinary CRUD and dashboard endpoints are **synchronous**: router to service to repository to database, and the response returns once that chain completes. The analytics pipeline is **decoupled from the request that starts it**. Clustering and sequence mining take minutes, so `POST /pipeline/run` only checks that no run is in progress and hands the work to a background daemon thread. It returns immediately, while the Frontend polls a lightweight status endpoint (Section 5.2.2.4). The LLM SQL Agent is synchronous from the client's view, but internally drives a multi-step LangGraph state machine before returning (Section 5.2.2.6).

Middleware: Validation and the Demonstration Role Layer

The only cross-cutting concerns are input validation and the demonstration role layer. There is no session, token, or authentication middleware, since the three roles exist purely to demonstrate a per-role interface over synthetic data (Chapter 4). Validation is enforced declaratively through the Pydantic schemas above, not custom middleware. The one place role awareness reaches into query logic is the SQL Agent, which injects a `DOCTOR_SCOPE_CLAUSE` or a `PATIENT_SCOPE_CLAUSE` into the prompt depending on the caller's role, restricting a doctor's questions to their own `provider_id` and a patient's to their own `patient_id`. A Manager gets neither clause and so queries unscoped. This is a demonstration of role-aware behavior, not an access-control boundary.

5.2. System Implementation

This section presents the concrete realization of the design above. The backend is implemented in Python on top of **FastAPI**, the frontend in TypeScript on top of **Next.js**/React, and all storage is a standard **PostgreSQL** instance. All backend configuration is supplied through environment variables loaded with `python-dotenv` from a `.env` file (the PostgreSQL connection parameters and the LLM API keys), so the identical codebase can be pointed at a local, testing, or production database without any source change, which satisfies the portability requirement of Chapter 4. The frontend implementation is presented first, followed by the backend, whose six modules (ETL,

clustering, pattern mining, orchestration, AI results generation, and the SQL agent) are each detailed in their own subsection.

5.2.1. Frontend Implementation

Routing and External Libraries

The application is a Next.js App Router project in TypeScript. Every folder under `app/` is a route. The `(auth)`/`(main)` groups introduced in Section 5.1.3 are implemented as plain folders whose parentheses exclude them from the URL, while still letting each group nest its own layout. Beyond Next.js and React, the implementaion relies on a small, fixed set of libraries: **Redux Toolkit** and **react-redux** for state, **MUI** and **Emotion** for components and styling. **Recharts** and **ECharts** render the analytical charts, **react-toastify** handles error notifications, and **Yup** handles form validation. No HTTP client library is used. Every call goes through the native `fetch` API, wrapped by the helper described next.

State Management and API Access

All server data lives in Redux Toolkit slices under `store/slices`, one per domain. Each slice pairs asynchronous *thunks*, which issue the REST calls, with a reducer that holds the fetched data together with its loading and error flags. Listing 5.1 shows the pattern used in the `patients` domain. A small typed `get<T>(url, label)` helper centralizes the request headers, scopes the URL to the signed-in user through `appendScopeToUrl`, and turns a failed response into a de-duplicated toast. A thunk such as `fetchPatients` builds its query string and simply awaits that helper. Other domains apply the same headers and toast handling inline in their own thunks, rather than through a shared function.

```
async function get<T>(url: string, label: string): Promise<T> {
  const scoped = appendScopeToUrl(url);
  const res = await fetch(scoped, { headers: defaultHeaders });
  if (!res.ok) {
    toast.error(`Failed to load ${label}`, { toastId: `fetch-${label}` });
    throw new Error(`${res.status}`);
  }
  return res.json() as Promise<T>;
}

export const fetchPatients = createAsyncThunk<
  PatientListResponse,
  FetchPatientsArgs,
  { rejectValue: ErrorResponse }
>("patients/fetchPatients", async (args, thunkAPI) => {
  try {
    const q = new URLSearchParams();
    if (args.search) q.set("search", args.search);
    // ...additional filters (gender, ageGroup, status, race, marital, city) set
    the same way
    q.set("page", args.page.toString());
    q.set("rows", args.rows.toString());
    q.set("sortBy", args.sortBy);
    q.set("sortType", args.sortType);
    return await get<PatientListResponse>(`${API_BASE}/patients?${q}`, "patients");
```

```
  } catch {
    return thunkAPI.rejectWithValue({ error: true, message: "Failed to fetch
    patients" });
  }
});
```

Listing 5.1: Generic fetch helper and a domain thunk built on top of it

A component never calls `fetch` itself. It dispatches a thunk, renders a loading state while the thunk is pending, and reads the result back from the slice once the reducer has stored it. So every chart and table shows exactly what the Backend returned.

Frontend Flow for Patient CRUD

The patient pages exercise the full read/write cycle over this same pattern. The list page dispatches `fetchPatients` with the current filters, page, and sort order, and renders the response in a `StyledTable`. Opening a record dispatches `fetchPatientDetail` for the detail view. Creating, editing, and deleting all go through three further thunks, `createPatient`, `updatePatient`, and `deletePatient`, which `POST`/`PATCH`/`DELETE` to `/patients` through small wrappers around `fetch` analogous to `get`.

The `patients` slice's `extraReducers`, shown in Listing 5.2, is where these thunks actually change state. `fetchPatients.fulfilled` replaces the page of `items` wholesale. `createPatient.fulfilled` and `updatePatient.fulfilled` patch that same array in place instead, either prepending or finding and replacing by `patientId`. This way, the table reflects a create or an edit without a second round tripp. `deletePatient` is the exception. Its reducer has no dedicated case. Instead, the page dispatches `fetchPatients` again once the delete resolves successfully, since removing a row can shift page boundaries and pagination totals in a way that is simpler to re-fetch than to patch locally.

```
const patientsSlice = createSlice({
  name: "patients",
  initialState,
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchPatients.fulfilled, (state, action) => {
        state.loading = false;
        state.items = action.payload.items;
        state.total = action.payload.total;
      })
      .addCase(createPatient.fulfilled, (state, action) => {
        state.items.unshift(action.payload);
        state.total += 1;
      })
      .addCase(updatePatient.fulfilled, (state, action) => {
        const i = state.items.findIndex((p) => p.patientId ===
    action.payload.patientId);
        if (i !== -1) state.items[i] = action.payload;
      });
  },
});
```

Listing 5.2: Patients slice: reducer state changes driven by the CRUD thunks

Conversation State for the SQL Agent

The Ask AI page maintains the dialogue with the SQL agent as an ordered list of user and assistant messages held in its slice. Each new question is sent to the backend together with the recent history, so the agent can resolve follow-up questions that refer to earlier turns (for example "and how many of them are women?"). The answer returned by the agent is appended to the same list, which the chat component renders in order. Because the history lives in the Redux store rather than in the component, it survives navigation within the session (content is not lost until refresh).

Reusable Components and Charts

The Fronted uses Emotion CSS through a custom `useClasses` hook, with a single `theme.ts` file as the source of truth for colors, spacing, and typography. Reusable wrappers (styled tables, inputs, dropdowns, date pickers, and chart components) live under `components/generic_components/` and are shared across every page rather than rebuilt per feature.

`PieChartComponent` is a representative example. Listing 5.3 shows it stripped to its essentials. It takes a data array, a title, and a color palette, and renders them through Recharts. All colors, fonts, and container styling come from the theme and `useClasses`, rather than being hard-coded in the component.

```
interface ChartEntry { name: string; value: number }

export default function PieChartComponent({
  data, title, colors,
}: { data: ChartEntry[]; title: string; colors: string[] }) {
  const classes = useClasses(PieChartStyles, { name: "PieChart" });

  return (
    <div className={classes.container}>
      <h3 className={classes.title}>{title}</h3>
      <ResponsiveContainer width="100%" height={380}>
        <PieChart>
          <Pie data={data} dataKey="value" nameKey="name" outerRadius={110}>
            {data.map((_, i) => (
              <Cell key={i} fill={colors[i % colors.length]} />
            ))}
          </Pie>
          <Tooltip />
          <Legend />
        </PieChart>
      </ResponsiveContainer>
    </div>
  );
}
```

Listing 5.3: Simplified reusable pie-chart component

The same component is instantiated, with different data and colors, on the statistics, home, and cluster-detail pages. Figure 5.5 shows it on the home page, where it renders the "Visits by Type" breakdown for the selected time range.

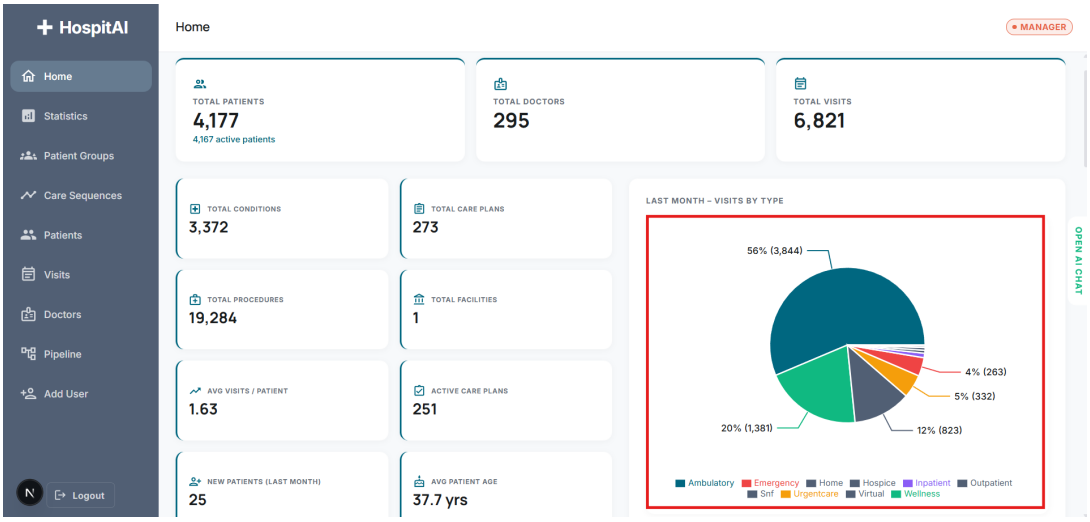


Figure 5.5: Home page "Visits by Type" card, built from a `PieChartComponent`

Role-Gated Navigation

Navigation is filtered by a single route-to-role map, `rolePermissions.ts`, that lists, for each route prefix, the roles allowed to see it. Listing 5.4 shows the essential shape: an array of prefix-to-roles entries and a lookup function that the layout calls on every navigation to decide whether to render the page or redirect.

```
export const ROUTE_ROLES: RouteRoles[] = [
  { prefix: "/home", allowed: ["MANAGER", "DOCTOR", "PATIENT"] },
  { prefix: "/encounters", allowed: ["MANAGER", "DOCTOR", "PATIENT"] },
  { prefix: "/clusters", allowed: ["MANAGER", "DOCTOR"] },
  { prefix: "/sequences", allowed: ["MANAGER", "DOCTOR"] },
  { prefix: "/patients", allowed: ["MANAGER", "DOCTOR"] },
  { prefix: "/providers", allowed: ["MANAGER", "DOCTOR"] },
  { prefix: "/statistics", allowed: ["MANAGER"] },
  { prefix: "/pipeline", allowed: ["MANAGER"] },
];

export function isRouteAllowedForRole(pathname: string, role: UserRole): boolean {
  const match = ROUTE_ROLES.find((r) => pathname.startsWith(r.prefix));
  return !match || match.allowed.includes(role);
}
```

Listing 5.4: Route-to-role map and its lookup function

As already stated, this is a demonstration of role-specific interfaces rather than an access-control mechanism.

5.2.2. Backend Implementation

Building on the layered design of Section 5.1.4, this subsection first explains the request-serving machinery shared by all endpoints, and then dedicates one part to each of the six backend modules.

Routing and Registration

Each endpoint lives in its own file under `api/<domain>/` and attaches itself to a shared `APIRouter`. The routers are collected in `api/__init__.py` through wildcard re-exports and registered in `main.py`. Registration order matters: static routes (for example `/patients/resources`) are registered before dynamic ones (`/patients/{id}`), so a literal path is never shadowed by a parameterized one.

Listing 5.5 shows a representative endpoint, `POST /patients`. The router only checks the caller's scope and delegates everything else, request validation, the actual insert, and error translation, to the schema and the service layer described next.

```python
@patient_router.post("", response_model=PatientDetailResponse, status_code=201)
async def create_patient(
    payload: PatientCreateRequest,
    db: Session = Depends(get_db),
    scope: Scope = Depends(scope_dep),
):
    if scope.is_patient:
        raise HTTPException(status_code=403, detail="Patients cannot create patient records")
    provider_id_scope = scope.provider_id if scope.is_doctor else None
    try:
        return PatientService(PatientRepository(db)).create_patient(payload, provider_id_scope)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Listing 5.5: A representative endpoint: creating a patient

Listing 5.6 shows the two layers behind that call: the service only forwards the request, while the repository builds the row, commits it, and links it to the creating doctor's panel when relevant.

```python
# service/patient_service.py
def create_patient(self, payload, provider_id_scope=None):
    return self.repo.create_patient(payload, provider_id_scope)

# repository/patient_repository.py
def create_patient(self, payload, provider_id_scope=None):
    patient = PatientModel(patient_id=uuid.uuid4().hex,
    first_name=payload.first_name, ...)
    self.db.add(patient)
    self.db.commit()
    if provider_id_scope:
        link_patient_provider(self.db, patient.patient_id, provider_id_scope)
    return self.get_patient(patient.patient_id)
```

Listing 5.6: Service and repository layers backing `create_patient`

Validation and Serialization

Request and response bodies are declared as Pydantic models in `domain/schemas`, kept separate from the SQLModel tables in `domain/models` that the repository layer reads and writes. Listing 5.7 shows the latter for `patient`: a plain SQLModel table with its column constraints. Listing 5.8 shows the former for the same resource: the request schema FastAPI validates `POST /patients` against before `create_patient` is ever called.

```python
class PatientModel(SQLModel, table=True):
    __tablename__ = "patient"

    patient_id: str = Field(primary_key=True)
    birthdate: Optional[date]
    deathdate: Optional[date]
    age: int = Field(gt=0)
    first_name: str = Field(index=True, nullable=False, max_length=50)
    last_name: str = Field(index=True, nullable=False, max_length=50)
    gender: Optional[str]
```

Listing 5.7: Domain model: the `patient` SQLModel table

```python
class CamelModel(BaseModel):
    model_config = ConfigDict(alias_generator=to_camel, populate_by_name=True)

class PatientCreateRequest(CamelModel):
    first_name: str
    last_name: str
    age: int
    birthdate: Optional[date] = None
    gender: Optional[str] = None
```

Listing 5.8: Domain schema: the patient-creation request

A shared `CamelModel` base, reused by every request/response schema, transparently converts between the `snake_case` used in Python and the `camelCase` expected by the TypeScript frontend, so both sides keep their idiomatic naming. FastAPI validates every incoming body against these schemas before the service layer is reached, rejecting malformed input with a descriptive error and without any hand-written checks.

Database Access and Configuration

The persistence layer follows a singleton pattern: the SQLAlchemy `engine` and its `SessionLocal` factory are created once at module import and reused for the process's lifetime, so the connection pool is shared across requests instead of being rebuilt each time. The `get_db()` FastAPI dependency, shown in Listing 5.9, draws a session from this shared factory per request and guarantees it is closed once the response is sent.

All connection parameters (host, credentials, database name) are read from environment variables through `python-dotenv`, keeping credentials out of the source and letting the same code target a local, testing, or production database with no change.

```python
engine = create_engine(url, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Listing 5.9: Singleton engine/session factory and the request-scoped session dependency

5.2.2.1 ETL Pipeline

This thesis uses the open-source Synthea generator to produce a synthetic but realistic electronic-health-record dataset, generating patient demographics together with the clinical events (conditions, medications, procedures, immunizations, allergies, care plans, and observations) of 20,000 living patients (plus 2,284 further patients who die during the simulation, for 22,284 patient records in total), each with a decades-long history (Section 6.1 gives the row counts of every source file).

The Ingestion Script (`db/etl_script.py`) loads this CSV output into the normalized schema of Section 5.1.2 via SQLModel, following Figure 5.6. It reuses the `engine/get_db()` connectivity of Section 5.2.2, runs once per database refresh, and can be run standalone or through the orchestration job.

Only eleven of Synthea's output tables are kept. Financial, imaging, and device files (`claims`, `payers`, `devices`, `imaging_studies`, `supplies`, and similar) are dropped as out of scope.

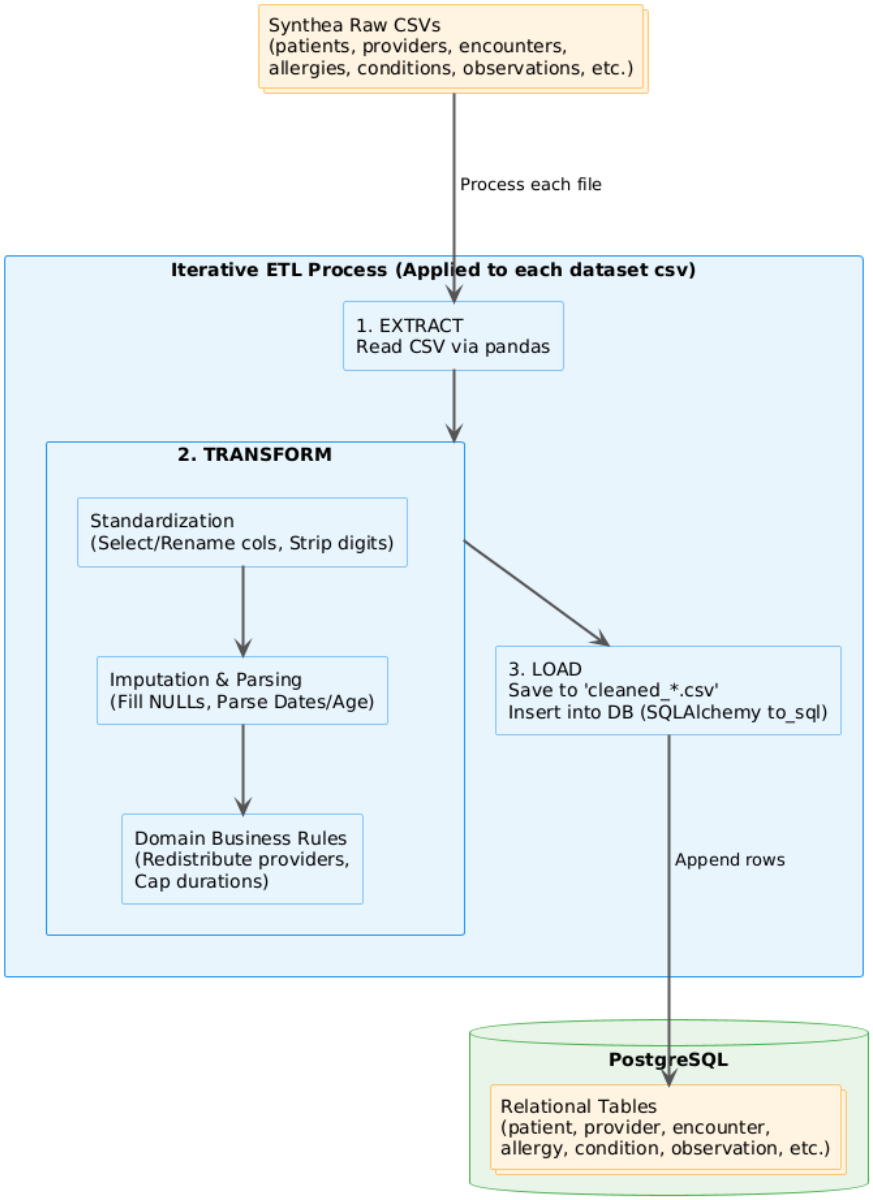


Figure 5.6: ETL Pipeline Diagram

Reducing Synthea to a Single Hospital

Synthea's `organizations.csv` covers an entire region (5,833 organizations), while this thesis targets a *single* hospital. `_pick_single_org` keeps the organization with the highest `UTILIZATION`, and `_pick_top_provider_ids` keeps its 300 busiest providers out of the original 5,833. Every encounter left with a dropped provider is reassigned rather than discarded. It goes to one of the kept providers, chosen uniformly at random through a seeded `numpy` draw for reproducibility, and is rewritten to the one retained organization. Original Synthea UUIDs are kept as primary keys throughout, so no look-up table is needed.

```
1 def _pick_single_org() -> dict:
2     df = pd.read_csv(dataset_path / "organizations.csv")
3     row = df.loc[df["UTILIZATION"].idxmax()]
4     return {"organization_id": row["Id"], "name": row["NAME"], ...}
```

```
5
6  def _pick_top_provider_ids(max_providers: int = 300) -> list[str]:
7      enc_df = pd.read_csv(dataset_path / "encounters.csv", usecols=["PROVIDER"])
8      return enc_df["PROVIDER"].value_counts().head(max_providers).index.tolist()
```

CSV Ingestion and Computed Columns

Each CSV is read with `pandas.read_csv`, mapped to the columns the schema needs, and bulk-inserted with `DataFrame.to_sql`. The derived columns of Section 5.1.2 (age, durations) are computed in-frame at this same step, as the listing below shows for patient age. The one exception is `observation` (over fourteen million rows), streamed in 10 000-row chunks to keep memory bounded. The single many-to-many relationship, `patient_provider`, is filled separately with one `INSERT ... SELECT DISTINCT` over `encounter`.

```
patients_df = df[list(keep_cols.keys())].rename(columns=keep_cols)
patients_df["first_name"] = patients_df["first_name"].apply(_strip_digits)
patients_df["age"] = (
    patients_df["deathdate"].fillna(pd.Timestamp.now()) - patients_df["birthdate"]
).dt.days // 365

patients_df.to_sql(name="patient", con=engine, if_exists="append", index=False)
```

Execution Order

`run_etl` wires the per-table loaders together so parents load before children: patients, one organization, and 300 providers before any encounter, and encounters before allergies, procedures, medications, immunizations, careplans, conditions, and observations. `_truncate_all_tables` clears the same tables beforehand, in child-first order, so no `TRUNCATE CASCADE` violates a foreign key.

```
def run_etl(engine):
    _truncate_all_tables(engine)
    load_patients(engine); load_organizations(engine); load_providers(engine)
    load_encounters(engine); load_patient_providers(engine)
    load_allergies(engine); load_procedures(engine); load_medications(engine)
    load_immunizations(engine); load_careplans(engine)
    load_conditions(engine); load_observations(engine)
```

Libraries

The ETL layer relies on `pandas` for CSV processing, `SQLAlchemy`/`SQLModel` for schema and connection pooling, `psycopg2-binary` as the PostgreSQL driver, and `python-dotenv` for configuration. Bulk loads go through `DataFrame.to_sql` or raw `text()`, never the ORM.

5.2.2.2 K-Prototypes Clustering

Patient profiles mix numeric (age) and categorical (gender, dominant care plan) attributes, so we chose K-Prototypes, which extends K-Means with a categorical distance term (Section 4.2.1), over an artificial encoding. The clustering module, `k_prototypes.py`,

wraps `kmodes.kprototypes.KPrototypes` with everything the algorithm itself does not do: feature construction, scaling, outlier handling, evaluation, plotting, persistence, and handing off to the AI interpretation of Section 5.2.2.5. Being time-consuming, it runs offline, either as a standalone script or as a pipeline stage, following the three phases below (input preparation, algorithm execution, output persistence).

Listing 5.10 traces `run_clustering_stage`, the production entry point that runs these phases end to end for the single champion configuration (`v55`, Chapter 6), stripped of file/plot bookkeeping to keep the call sequence readable.

```
def run_clustering_stage(engine):
    cfg = CONFIGS[0]

    df = build_patient_features(engine)
    df_cleaned, df_outliers = remove_outliers_zscore(df, threshold=6)

    df_scaled = log1p(df_cleaned[skewed_cols])
    df_scaled["clinical_severity_score"] = pca_1d(df_scaled[severity_cols])
    df_scaled[numerical_cols] =
    MinMaxScaler().fit_transform(df_scaled[numerical_cols])
    matrix = concat(df_scaled[numerical_cols],
    df_scaled[categorical_cols].astype(str))

    kproto = KPrototypes(n_clusters=cfg.k, init="Huang", random_state=42)
    clusters = kproto.fit_predict(matrix, categorical=categorical_positions)
    df_result = df_cleaned.assign(cluster=clusters)
    plot_famd_scatter(df_result); plot_cluster_heatmap(df_result);
    plot_cluster_bars(df_result)

    report, eval_metrics = full_evaluation_report(df_result, clusters, ...)
    summary_text = get_clusters_summary(df_result, df_outliers, engine)

    save_cluster_results(
        df_clustered=df_result, df_outliers=df_outliers,
        numerical_features=cfg.numerical, categorical_features=cfg.categorical,
        metrics=eval_metrics, cluster_summaries=summary_text,
    )
    return cfg.k
```

Listing 5.10: Pseudocode of the production clustering run (`run_clustering_stage`)

Feature Engineering

Many candidate features were tried across 628 K-Prototypes runs before settling on the list below, split into **direct attributes** (age, gender) and **clinical signals** derived from the raw event tables. Figure 5.7 traces this derivation: raw data in, features extracted, high-cardinality/numeric variables bucketed, and a feature vector out for K-Prototypes.

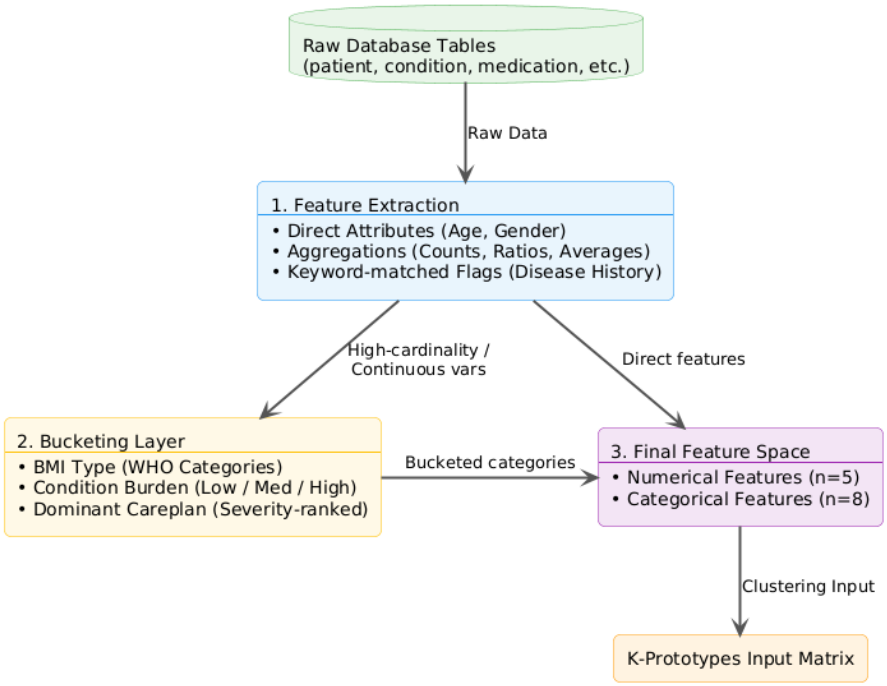


Figure 5.7: Feature engineering pipeline for the K-Prototypes input matrix

The final configuration uses **5 numerical** and **8 categorical** features, which are described in detail below.

Numeric Features

- **age** – from the patient record.
- **active_conditions_count** – unresolved conditions (no end date). Reflects current chronic burden.
- **total_medications** – lifetime medication record count.
- **total_procedures** – lifetime procedure count.
- **preventive_to_reactive_ratio** – wellness visits over (acute + severe visits + 1). A higher value means more preventive-oriented care.

Categorical Features

- **gender** – from the patient record.
- **has_diabetes**, **has_hypertension**, **has_heart_disease** – keyword flags for this high-prevalence, high-impact triad.
- **pregnancy_history** – Active/Past/None, since an ongoing pregnancy needs different care than a past one.
- **condition_burden** – binned `active_conditions_count`.
- **bmi_type** – binned average BMI (below).
- **dominant_careplan** – most severe care plan, binned from descriptions (below).

Feature Construction

`build_patient_features(engine)` runs one standalone SQL query per feature category (demographics, encounters, condition flags, BMI/BP/glucose, care plan, and so

on) via `pandas.read_sql`, then chains `df.merge(right_df, on="patient_id", how="left")` across them and applies type-aware imputation.

The listing below shows the pattern for *dominant care plan*: 40 raw care plan descriptions are bucketed into 6 clinical categories by severity rank, and `ROW_NUMBER() OVER (PARTITION BY patient_id ORDER BY severity_rank ASC)` keeps only the most severe one per patient.

```
WITH CategorizedPlans AS (
    SELECT patient_id,
           CASE
               WHEN description IN (
                   'Cancer care plan (record artifact)',
                   'Dialysis care plan (record artifact)',
                   'Terminal care (regime/therapy)'
               ) THEN 'Complex, Oncology & Palliative'
               WHEN description = 'Routine antenatal care (regime/therapy)'
                                THEN 'Maternal & Antenatal'
               WHEN LOWER(description) ~ '(fracture|wound|surgery|injury|trauma)'
                                THEN 'Acute Trauma, Wound & Surgery'
               WHEN LOWER(description) ~ '(respiratory|asthma|pulmonary|covid)'
                                THEN 'Respiratory & Infectious'
               WHEN LOWER(description) ~ '(diabetes|heart|hypertension|cardiac)'
                                THEN 'Chronic Cardio-Metabolic'
               ELSE 'General Rehab & Routine'
           END AS care_plan_category,
           CASE
               WHEN description IN (
                   'Cancer care plan (record artifact)',
                   'Dialysis care plan (record artifact)',
                   'Terminal care (regime/therapy)'
               ) THEN 1
               WHEN description = 'Routine antenatal care (regime/therapy)' THEN 2
               WHEN LOWER(description) ~ '(fracture|wound|surgery|injury)'   THEN 3
               WHEN LOWER(description) ~ '(respiratory|asthma|pulmonary)'    THEN 4
               WHEN LOWER(description) ~ '(diabetes|heart|hypertension)'     THEN 5
               ELSE 6
           END AS severity_rank
    FROM careplan
),
RankedPatientPlans AS (
    SELECT patient_id, care_plan_category,
           ROW_NUMBER() OVER (
               PARTITION BY patient_id ORDER BY severity_rank ASC
           ) AS rank_id
    FROM CategorizedPlans
)
SELECT patient_id, care_plan_category AS dominant_careplan
FROM RankedPatientPlans
WHERE rank_id = 1;
```

The same technique produces the other derived categories: `substance_use_profile`, `pain_management_tier`, `readmission_risk_tier`, `cholesterol_risk_category`, `respiratory_disease_profile`, and others.

Feature Bucketing

Once merged, sparse clinical features are deliberately bucketed rather than mean-imputed, to avoid distorting the numerical centroids K-Prototypes assumes are real measurements. Missing values fall into an explict `"Unknown"` bucket instead. Count-type features are simply filled with zero, since that correctly represents an absence of events.

```
for col in ["wellness_count", "inpatient_count", ...]:
    df[col] = df[col].fillna(0)        # zero is semantically correct
df["dominant_careplan"] = df["dominant_careplan"].fillna("None")
df["bmi_type"] = pd.cut(df["avg_bmi"],
                        bins=[-inf, 18.5, 25, 30, 35, 40, inf],
                        labels=["Underweight", "Normal", "Overweight",
                                "Obese I", "Obese II", "Obese III"]) \
                   .astype(object).where(df["avg_bmi"].notna(), "Unknown")
```

The Role of `log1p`

`total_medications`, `lifetime_medication_days`, and `total_encounter_duration_hours` have strong right tails, so `np.log1p` is applied to them at two points in the pipeline:

```
for i, col in enumerate(cols):
    if col in _SKEWED_OUTLIER_COLS:
        data[:, i] = np.log1p(np.clip(data[:, i], 0, None))

for col in ["total_medications", "lifetime_medication_days",
            "total_encounter_duration_hours"]:
    if col in numerical_cols:
        df_scaled[col] = np.log1p(df_scaled[col].clip(lower=0).astype(float))
```

The first block runs inside `remove_outliers_zscore`, so these heavy-tailed counts do not flag a quarter of the cohort as outliers on distribution shape alone. The second runs inside `run_kprototypes`, so the scaler does not compress the "normal" mass of patients into a tiny corner of $[0, 1]$. `log1p` is preferred over plain log since it is defined at $x = 0$, the mode of these columns. Both transforms apply only to `df_scaled`, never to `df_work`, which stays raw for the LLM prompt and the database.

PCA for Severity Compression, FAMD for Visualization

PCA and FAMD both reduce dimensionality here, but for different jobs. **PCA** compresses the three correlated severity features (`total_procedures`, `total_medications`, `active_conditions_count`) into one `clinical_severity_score` axis. Configurations that want an overall severity signal do not have to carry all three, and configurations combining the composite with its raw inputs are disallowed as redundant. The score itself is computed by log1p-ing the three columns, standardizing, and projecting onto the first principal component, which is already a z-score and so skipped by the outer scaler:

```
sev_input        = np.log1p(df[sev_cols].clip(lower=0).values.astype(float))
sev_standardized = StandardScaler().fit_transform(sev_input)
css              = PCA(n_components=1).fit_transform(sev_standardized)[:, 0]
```

FAMD (Factor Analysis of Mixed Data, from `prince`) instead produces the dashboard's 2-D scatter plot. It is PCA's natural extension to mixed data: numerics are standardized, and categoricals are one-hot-encoded and reweighted by inverse category

frequency, so both types contribute on equal footing. An ordinal encoding, by contrast, would distort the categorical distance geometry K-Prototypes relies on.

```
famd   = prince.FAMD(n_components=2, random_state=42).fit(df_feat)
coords = famd.transform(df_feat).values
```

Z-score Outlier Detection

`remove_outliers_zscore()` applies the z-score of Equation (4.16) (Section 4.3.3) to a fixed list of numerical attributes, log1p-ing the skewed ones first, before clustering ever begins:

```
data = df[cols].to_numpy(dtype=float)
for i, col in enumerate(cols):
    if col in _SKEWED_OUTLIER_COLS:
        data[:, i] = np.log1p(np.clip(data[:, i], 0, None))

mask = np.ones(len(df), dtype=bool)
for i, col in enumerate(cols):
    z    = np.abs((data[:, i] - np.nanmean(data[:, i])) / np.nanstd(data[:, i]))
    mask &= z < threshold      # threshold = 6 in production
```

A patient is retained only if every column's z-score stays under the threshold. Outliers are not discarded, though. The function returns two frames, and `df_outliers` is carried through the rest of the chain and persisted with cluster label -1. It forms its own cluster on the dashboard, gets an LLM explanation like any other cluster, and is mined separately by PrefixSpan. No patient is dropped, while the four real clusters' centroids still represent ordinary patients.

Scaling the Numerical Block

Section 4.3.3 already covers what MinMax, standard, and robust scaling compute. Here they are simply exposed through a name-to-instance map and selected per run:

```
_SCALER_MAP = {
    "minmax": MinMaxScaler(), "robust": RobustScaler(), "standard":
    StandardScaler(),
}
```

MinMax is the production choice. The K-Prototypes objective adds a squared-Euclidean numerical term to a γ-scaled Hamming term bounded in $[0, n_{\text{cat}}]$. Scaling the numerics to the same $[0, 1]$ range as that bound keeps the two terms comparable regardless of the numerics' original spread, so `kmodes` can auto-tune γ without either term dominating. StandardScaler centers at zero but leaves the long tails untouched. RobustScaler is outlier-resistant, but it still leaves some values outside $[0, 1]$. Both produced less stable clusters across the benchmark.

Running the Algorithm

The matrix passed to `kmodes` places numeric columns first and string-cast categorical columns after, with their indices passed via `categorical=`, so the library's Cython loop can split the two blocks efficiently:

```
num_matrix    = df_scaled[numerical_cols].to_numpy(dtype=np.float64)
cat_matrix    = df_scaled[categorical_cols].astype(str).to_numpy()
matrix        = np.concatenate([num_matrix, cat_matrix], axis=1)
cat_positions = list(range(len(numerical_cols),
                           len(numerical_cols) + len(categorical_cols)))
kproto = KPrototypes(n_clusters=k, init="Huang",
                     random_state=rs, n_jobs=-1, gamma=gamma)
clusters = kproto.fit_predict(matrix, categorical=cat_positions)
```

`init="Huang"` seeds prototypes from attribute frequency distributions (Huang, 1998) and, combined with a fixed `random_state`, keeps runs reproducible. With `gamma=None`, `kmodes` derives γ from the standard deviation of the (already min-max-normalized) numeric block, balancing the two terms of the objective automatically.

Configuration Loop and Persistence to the Database

`configs.py` holds **63 configurations** (`v1`–`v63`), each testing a different feature hypothesis (severity-only, utilization-only, cardio-metabolic, and so on) and archived with comments as an audit trail. Their benchmarking and the selection of `v55` as champion are covered in Chapter 6. Only `v55` stays uncommented in the code:

```
{
    "version": 55,
    "preferred_k": 4,
    "numerical": [
        "age",
        "active_conditions_count",
        "total_medications",
        "total_procedures",
        "preventive_to_reactive_ratio",
    ],
    "categorical": [
        "gender",
        "pregnancy_history",
        "has_heart_disease",
        "has_diabetes",
        "has_hypertension",
        "condition_burden",
        "bmi_type",
        "dominant_careplan",
    ],
},
```

Outside production, the same code doubles as a benchmark harness: replacing any of `THRESHOLDS`, `CONFIGS`, `SCALERS`, `GAMMAS`, `RANDOM_STATES`, or k with a non-singleton list turns the single run above into a loop over their Cartesian product. Each iteration writes one `conclusions.xlsx`/plot set, plus a combined `general_conclusion.xlsx` for comparison across the whole sweep. Chapter 6 presents the results of that sweep and the selection of `v55`.

From DataFrame to `numpy` and Back to Words

K-Prototypes and the sklearn metrics need `numpy` arrays, but the LLM prompt, the dashboard cards, and `cluster_patient.features` need the original strings back

(`gender = "F"`, `bmi_type = "Obese II"`). `run_kprototypes` avoids any decoding step by keeping two frames throughout: `df_work` stays raw and is what gets persisted, while `df_scaled` (log1p, severity PCA, scaler) is the one converted to `numpy`. The resulting cluster labels are appended back onto the untouched raw frame:

```
df_work   = df.copy()
df_scaled = df_work.copy()
clusters  = kproto.fit_predict(matrix, categorical=cat_positions)
df_work["cluster"] = clusters
```

Reuse of Derived Features Across Configurations

`build_patient_features` computes its SQL features *once* per run. The configuration loop then only *selects* columns from that same frame, so sweeping 63 configurations costs one round of SQL/merging plus 63 in-memory `numpy` fits, not 63 full feature builds.

The same reuse extends to persistence: `save_cluster_results` takes an `extra_feature_cols` list, currently `avg_bmi`, `avg_systolic_bp`, `avg_glucose`, `qaly`, `daly`, `days_since_last_encounter`, the four care-setting counts, `distinct_providers_seen`, `has_cancer`, `has_mental_health`, `is_polypharmacy`, `has_respiratory_disease`, `marital`, `dominant_encounter_class`, and `smoking_status`. None of these enter the K-Prototypes distance computation, but all are stored in the `cluster_patient.features` JSONB column. They play no role in forming the clusters, yet they still reach the AI interpretation step (Section 5.2.2.5) and the frontend's per-patient and per-cluster detail views.

Libraries

The principal libraries used are `kmodes` (algorithm), `scikit-learn` (`StandardScaler`, `MinMaxScaler`, `RobustScaler`, `PCA`, evaluation metrics), `prince` (`FAMD`), `matplotlib` and `seaborn` (plots), `pandas`/`numpy` (feature pipeline), `SQLAlchemy` (read/write paths), and the OpenAI Python SDK behind `ask_ai` for the cluster-narrative step of Section 5.2.2.5.

5.2.2.3 PrefixSpan Customizations

Clustering tells us which group a patient belongs to, but not *how* patients move through the care system over time. To capture that, we add a pattern-mining layer on top of the clustered patients. We use PrefixSpan (Section 4.2.2) because it grows patterns by projection instead of by repeated candidate generation, which stays fast even with the hundreds of distinct clinical event codes in this dataset, unlike algorithms such as GSP.

Figure 5.8 shows the four stages of the mining process. Stage 1 turns raw clinical events into one sequence per patient. Stage 2 mines frequent candidate patterns with PrefixSpan. Stage 3 scores and ranks them so the clinically interesting ones rise to the top. Stage 4 filters out near-duplicates to keep only the strongest, most distinct results.

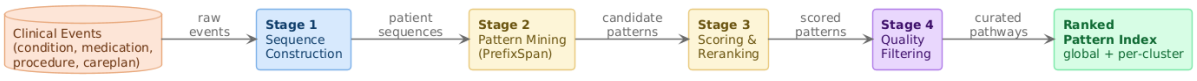


Figure 5.8: Pattern mining pipeline stages

Plain PrefixSpan only ranks patterns by support, and on clinical data that raw ranking is dominated by repetitive, near-duplicate, and purely administrative sequences. To fix this, the pipeline adds **seven customizations** on top of the base algorithm:

1. **Over-mining with a closed top-k operator**: a candidate pool twenty times larger than the final result set is mined, restricted to closed patterns, so later filters can prune aggressively without running out of candidates.
2. **Geometric-mean-lift re-ranking**: candidates are scored against how independent their items are, so a distinctive combination outranks a pattern built only from common events.
3. **Type-diversity bonus and same-type-run filter**: patterns spanning several event types (condition, medication, procedure, care plan) are boosted, while patterns that repeat the same type three times in a row are dropped.
4. **Full-spectrum quota**: a fixed number of slots is reserved up front for patterns that touch all four event types, so this small but clinically rich group is never crowded out by the far more common two-type patterns.
5. **Jaccard deduplication**: a greedy pass rejects any candidate that shares most of its items with an already accepted pattern.
6. **Prefix, suffix, and dental caps**: only a fixed number of accepted patterns may start or eend with the same item, and majority-dental patterns are capped so routine dental care cannot fill the whole result set.
7. **Dual-scope mining with cross-scope deduplication**: the miner runs once globally and once per cluster (including the outlier group), and each per-cluster run drops any pattern already found globally, so cluster results only show what is distinctive to that cluster.

Listing 5.11 traces the top-level orchestration end to end: `run` loops over scopes, and `_run_single` mines one of them. The rest of this subsection zooms into each customization in turn, starting with how the input sequences are built.

```
def run(engine, db_session, n_clusters=None):
    if n_clusters is None:
        _run_single(engine, db_session, cluster_label=None)
    else:
        for label in list(range(n_clusters)) + [-1]:
            _run_single(engine, db_session, cluster_label=label)

def _run_single(engine, db_session, cluster_label):
    sequences, patient_ids = build_sequences(engine, cluster_label)
    if not sequences:
        return

    mined = mine_patterns(sequences)
    if cluster_label is not None:
        mined = drop_already_global(mined, db_session)

    if not mined:
        return

    memberships = attribute_patients(mined, sequences, patient_ids)
    save_prefix_span_results(db_session, mined, memberships, cluster_label)
```

Listing 5.11: Pseudocode of the mining orchestration (`run` and `_run_single`)

Sequence Construction

Clinical events come from four tables (`condition`, `medication`, `procedure`, `careplan`), each tagged with its source so that the same description in two tables still counts as two different events. Figure 5.9 shows this for one patient: five raw rows from four tables become one ordered clinical sequence.

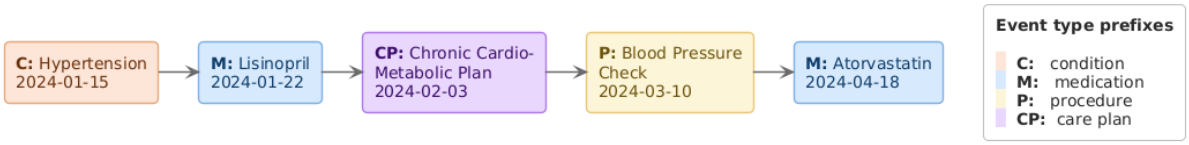


Figure 5.9: Example of a length 5 sequence

A single `UNION ALL` over the four tables builds one combined event stream, with each `description` prefixed by a source tag (`C:`, `M:`, `P:`, `CP:`):

```
SELECT patient_id, start_date AS event_date, 'C:' || TRIM(description) AS item
FROM condition
WHERE description IS NOT NULL AND start_date IS NOT NULL
UNION ALL
SELECT patient_id, start_date, 'M:' || TRIM(description) FROM medication
UNION ALL
SELECT patient_id, start_date, 'P:' || TRIM(description) FROM "procedure"
UNION ALL
SELECT patient_id, start_date, 'CP:' || TRIM(description) FROM careplan;
```

`build_sequences` sorts events by `(patient_id, event_date, _pref_rank, item)`, so same-day ties always break in the order `C` < `M` < `P` < `CP`. Four filters then clean the stream, each fixing a specific problem seen in early prototypes.

(1) Noise blacklists. Two substring blacklists remove events that are not real clinical patterns. `ITEM_BLACKLIST_SUBSTRINGS` drops routine workflow notes (for example "medication reconciliation"), which showed up as the leading item of nearly every top-ranked pattern and drowned out the real ones. `SOCIAL_STATUS_SUBSTRINGS` drops psychosocial/SDOH screening findings (employment status, social isolation, and similar), which sit on almost every wellness visit and only add a vague, repetitive antecedent. Both match by substring (`str.contains(sub, regex=False)`), since Synthea often appends extra codes to these descriptions.

```
ITEM_BLACKLIST_SUBSTRINGS: frozenset[str] = frozenset({
    "assessment of health and social care needs",
    "medication review due",
    "medication reconciliation",
})
```

(2) Recency window. Events older than `WINDOW_DAYS=730` days before a patient's own most recent event are dropped, using `groupby(patient_id).transform("max")` so the two-year window is anchored per patient, not to a single dataset-wide date:

```
maxd  = events.groupby("patient_id", sort=False)["event_date"].transform("max")
cutoff = maxd - pd.Timedelta(days=WINDOW_DAYS)
events = events[events["event_date"] >= cutoff]
```

Two years keeps sequences clinically relevant without growing them past what is useful, and anchoring the cutoff to each patient's own last event means the most recent two years of history are kept even for patients who died long ago.

(3) Consecutive duplicates and timeline cap. A repeated `item` right after itself for the same patient is dropped (compared via `shift()`), and each timeline is then capped at `MAX_EVENTS_PER_PATIENT=25` with `groupby().tail(25)`:

```
same_pid  = events["patient_id"].eq(events["patient_id"].shift())
same_item = events["item"].eq(events["item"].shift())
events    = events[~(same_pid & same_item)]
events    = events.groupby("patient_id", sort=False,
    group_keys=False).tail(MAX_EVENTS_PER_PATIENT)
```

Without deduplication, a patient on a long-term medication would repeat `M:Lisinopril` many times in a row, and PrefixSpan would happily report the trivial pattern `M:Lisinopril` $\rightarrow$ `M:Lisinopril` $\rightarrow$ `M:Lisinopril`. Without the length cap, mining time grows as $O(\text{tail}^2)$, and a heavy-utilizer's timeline can run to hundreds of events.

Temporal-shuffle hook (validation). `build_sequences` also takes an optional `shuffle_seed` that drives the order-sensitivity null test used to validate the mined sequences in Section 6.3. When it is set, each patient's timeline is permuted in place *after* all four filters above, so the exact same events enter mining in the real and the shuffled runs:

```
if shuffle_seed is not None:
    rng = np.random.default_rng(shuffle_seed)
    for seq in sequences:       # permute WITHIN each patient
        rng.shuffle(seq)        # item set unchanged, order destroyed
```

Because the permutation stays inside a single patient, every patient keeps the same *set* of items, so single-item supports, and therefore the geometric-mean-lift denominator introduced below, are left unchanged. Only chronological order is destroyed. Exposed on the CLI as `-shuffle`, this run is deliberately read-only: it writes its top-k to a separate `patterns_*_shuffled_*.txt` file and never touches the database or the real mined results.

Execution: the Mining Core

The pattern mining pipeline lives in `app/clustering_algorithms/prefix_span/prefix_span.p` runnable as a standalone script (cluster-wide, or per cluster via `-n-clusters N`) or as a pipeline stage, and, like the clustering module, offline rather than inside a request handler. Mining itself is done by the `prefixspan` package. Everything else here (preparation, scoring, deduplication, patient assignment, persistence) wraps around it. Every numerical knob lives at the top of the file:

```
MIN_SUPPORT_PCT       = 0.01      # floor: max(2, 1% of patients)
MIN_PATTERN_LENGTH    = 3
```

```
MAX_PATTERN_LENGTH     = 6
MAX_EVENTS_PER_PATIENT = 25
TOP_K                  = 500
WINDOW_DAYS            = 730        # 2 years of recent events per patient
OVERMINE_MULTIPLIER    = 20         # mine 10000, keep 500 after rerank
JACCARD_THRESHOLD      = 0.80
MAX_PER_PREFIX         = 6
MAX_PER_SUFFIX         = 20
TYPE_DIVERSITY_BONUS   = 0.3
MAX_DENTAL_PATTERNS    = 15
MULTITYPE_MIN_TYPES      = 4         # types needed to count as "full spectrum"
MIN_MULTITYPE_PATTERNS   = 25        # reserved slots for full-spectrum patterns
```

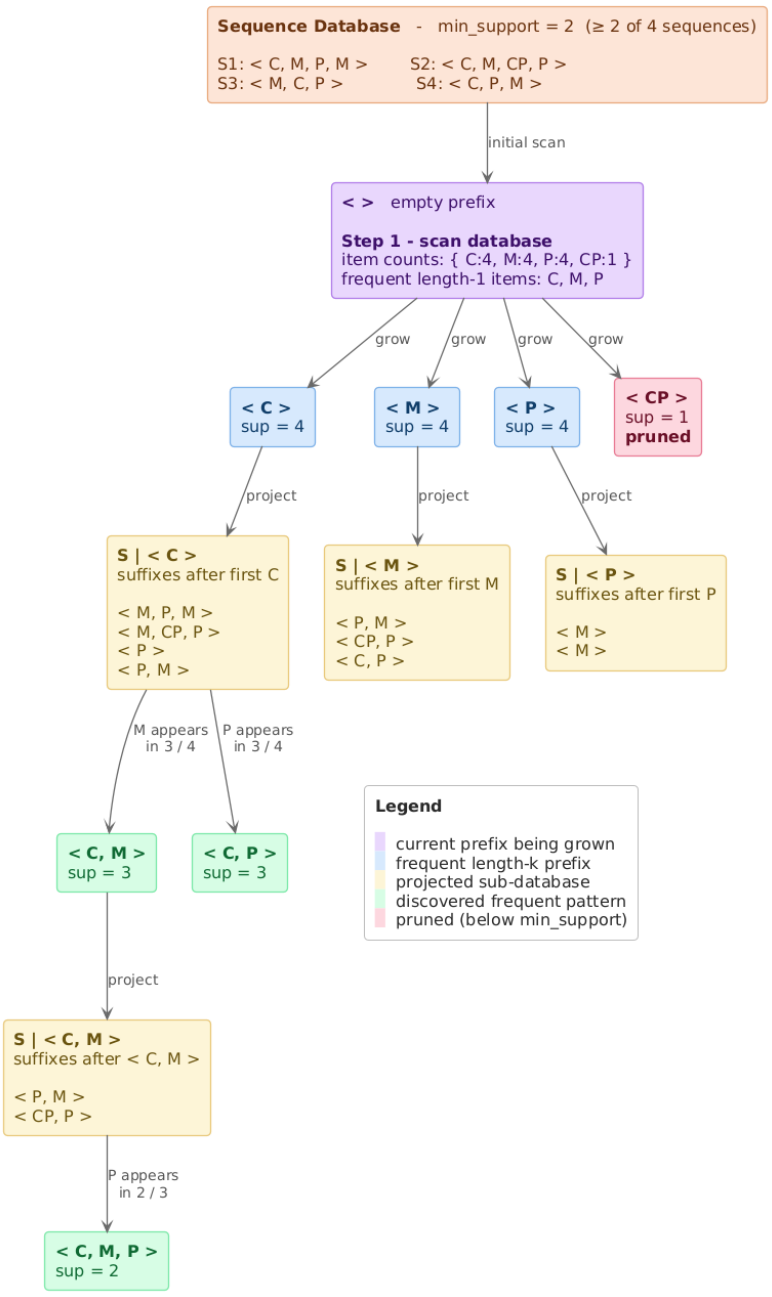


Figure 5.10: PrefixSpan projection tree

Figure 5.10 shows the core algorithm on four toy sequences with a minimum support of 2. Starting from the empty prefix, it scans the dataset and counts each item, keeping the frequent length-1 prefixes (<C>, <M>, <P>, each with support 4) and pruning infrequent ones like <CP> right away.

For each frequent prefix, PrefixSpan then builds a projected sub-database of only the sequences (and only the parts after that prefix) that contain it, and grows the prefix one item at a time by mining that smaller database. From <C>, this is how <C,M> and <C,P> (support 3) are found, and from $S|$<C,M>, the length-3 pattern <C,M,P>. The recursion stops once no frequent item is left to append. Each projection is smaller than the one before it, so the search keeps shrinking on its own.

A deliberate design choice here is to keep all the clinical-quality logic (scoring, filtering, capping) as a post-processing step *after* PrefixSpan runs, rather than wiring it into the mining loop itself. This keeps the mining step a plain, well-tested library call, and lets every scoring rule below be tuned or replaced without re-running PrefixSpan.

Before mining, sequences are encoded to integers by a one-pass vocabulary builder (`_encode`), since `prefixspan.PrefixSpan` works on integers, and this is the single biggest speedup in the pipeline.The call itself uses the closed top-k operator (customization 1 in the list above), asking for a pool twenty times larger than the final result:

```
ps         = PrefixSpan(int_seqs)
ps.minlen  = MIN_PATTERN_LENGTH        # 3
ps.maxlen  = MAX_PATTERN_LENGTH        # 6
pool_size  = TOP_K * OVERMINE_MULTIPLIER  # 10000
raw        = ps.topk(pool_size, closed=True)
abs_minsup = max(2, int(n_patients * MIN_SUPPORT_PCT))   # 1%
raw        = [(sup, pat) for sup, pat in raw if sup >= abs_minsup]
```

Why over-mine. The later filters (rescoring, dedup, capping) are meant to be aggressive, and aggressive filters can easily cut a pool of 500 down to around 250, well short of what the dashboard needs. Mining 20 times more candidates up front gives those filters enough room to prune hard without running out of good patterns. `closed=True` additionally keeps only closed patterns (Section 4.2.2). The 1% minimum-support floor (at least 2 patients) is loose enough to surface small but real pathways, such as a maternal-care pattern in a mostly non-pregnant cohort, while still ignoring a single noisy patient.

Geometric-Mean-Lift Scoring

Customizations 2 and 3 rescore every surviving candidate with geometric mean lift (GML): the ratio between a pattern's actual support and the geometric mean of its items' individual frequencies, i.e., how much more often these items appear together than chance would predict.

```
log_item_freqs = np.log(np.maximum(single_support[unique_items], 1)) - log_n
raw_gml        = np.exp(np.log(sup) - log_n - np.mean(log_item_freqs))
gml            = raw_gml * (1.0 + TYPE_DIVERSITY_BONUS * (n_types - 1))
```

`TYPE_DIVERSITY_BONUS` then multiplies this score up for patterns that mix several event types (C/M/P/CP), so a pattern like `C:` → `M:` → `P:` outranks a same-support pattern built from a single domain. Conversely, any pattern with three consecutive events of the same type is dropped before scoring even starts.

Full-Spectrum Quota

Customization 4 addresses a side effect of the scoring just described. Even with the diversity bonus, patterns touching all four event types are rare compared to the much larger pool of two-type patterns, and a plain top-k cut can still leave them out. Before the normal ranking is applied, up to `MIN_MULTITYPE_PATTERNS=25` patterns with at least `MULTITYPE_MIN_TYPES=4` distinct types are reserved first. These reserved slots are still subject to Jaccard deduplication and the dental cap below, but not to the prefix/suffix caps. This guarantees that multi-domain trajectories, such as a condition leading to a care plan, a medication, and a procedure, reach the dashboard instead of being crowded out. The remaining slots are then filled normally by GML rank. In the saved pattern listing, each reserved full-spectrum pattern is annotated with a `[4-type]` tag.

Closedness and Jaccard Deduplication

A pattern p is dropped as redundant if a longer pattern p' with the same support contains every item of p and p is one of its subsequences. Comparisons are bucketed by support value, so this only costs $O(\text{bucket}^2)$ instead of scanning the whole candidate list. Jaccard deduplication (customization 5) then removes near-duplicates during selection: a candidate is accepted only if it shares under 80% of its items with every pattern already chosen.

```
def _jaccard(a, a_len, b, b_len):
    inter = len(a & b)
    denom = a_len + b_len - inter
    return inter / denom if denom else 0.0

if all(_jaccard(pat_set, pat_len, s, s_len) < JACCARD_THRESHOLD
       for s, s_len in chosen_sets):
    chosen.append(entry)
```

Two more caps (customization 6) stop any single item from dominating the result set: at most `MAX_PER_PREFIX=6` accepted patterns may start with the same item, and at most `MAX_PER_SUFFIX=20` may end with it. A separate cap limits *majority-dental* patterns to `MAX_DENTAL_PATTERNS=15`, since Synthea's routine dental visits (fluoride, cleaning, examination) otherwise produce so many near-identical patterns that they alone would fill the top 50, pushing out cardiometabolic and respiratory patterns entirely. A pattern counts as dental if its items match these keywords:

```
DENTAL_KEYWORDS: frozenset[str] = frozenset({
    "dental", "fluoride", "gingiv", "supragingival", "subgingival",
    "oral health", "tooth", "teeth",
})
```

Every majority-dental pattern is annotated with a `[dental]` tag in the saved listing, and this is the same tag that marks routine dental trajectories in the result tables of Section 6.3: they stay visible, but clearly flagged as dental noise rather than clinically distinctive pathways.

Dual Scope Mining and Persistence

Customization 7 mines at two scopes so the platform captures both the big picture and the per-cluster detail (Figure 5.11): one *global* run over every patient, and one *per-cluster* run for each discovered cluster plus the outlier group, all indexed together by scope. To keep the two complementary rather than repetitive, every per-cluster run checks the pattern strings already stored for the global scope and drops any match, so a cluster's results show only what is distinctive to it.

Once the result set is final, an inverted index over the sequences resolves which patients follow each pattern, written to the `prefix_span_patient_pattern` junction. The patterns themselves (rank, support, scope label) go to `prefix_span_pattern`. Their `ai_analysis` column is left `NULL` on purpose: the LLM interpretation is generated lazily, as described in Section 5.2.2.5.

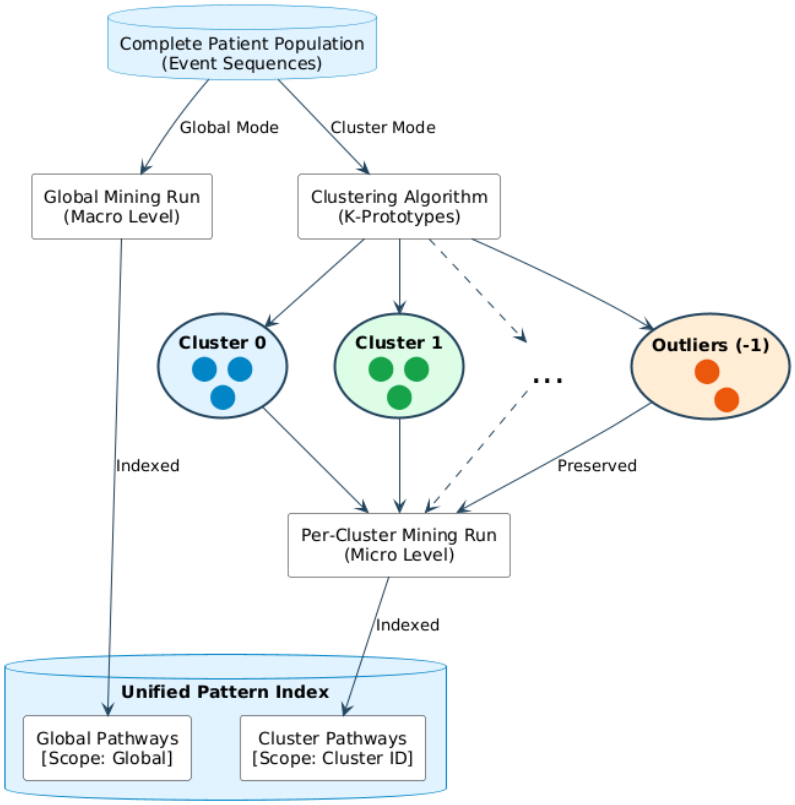


Figure 5.11: Sequence Mining Operating Modes

Libraries

`prefixspan` provides the closed top-k mining itself. `pandas` handles sequence filtering and preparation, and `numpy` computes the GML and diversity scores. `SQLAlchemy` (configured via `python-dotenv`) handles database access, while `collections.defaultdict`, `argparse`, and `pathlib` cover indexing, CLI flags, and paths.

5.2.2.4 Pipeline Orchestration

ETL ingestion, K-Prototypes clustering, and PrefixSpan sequence mining were originally three independent scripts. Each was launched by hand, and each assumed the

previous one had already run. An *orchestration job* now chains them into one monitored flow. It is tracked as a finite state machine in a singleton database row (`pipeline_run`), moving between `IDLE`, `RUNNING`, `SUCCESS`, `FAILED`, and `STOPPED` (if a manager cancels a run in progress via `POST /pipeline/stop`). While running, it advances through `ETL` (skipped once the database is populated), `CLUSTERING`, `SEQUENCES_GLOBAL`, and `SEQUENCES_CLUSTERS`, one run per cluster label $0..k-1$ plus the outlier group -1, as shown in Figure 5.12.

AI interpretation, the final stage of Chapter 4's conceptual pipeline, is folded into this same flow rather than run as its own tracked stage. Cluster cards are generated automatically, in-line, at the end of `CLUSTERING` (Section 5.2.2.5). Sequence patterns, though, are left uninterpreted until a user opens one. Only then is an explanation generated, lazily, and cached. Any exception along the way is caught and recorded, marking the run `FAILED` instead of crashing.

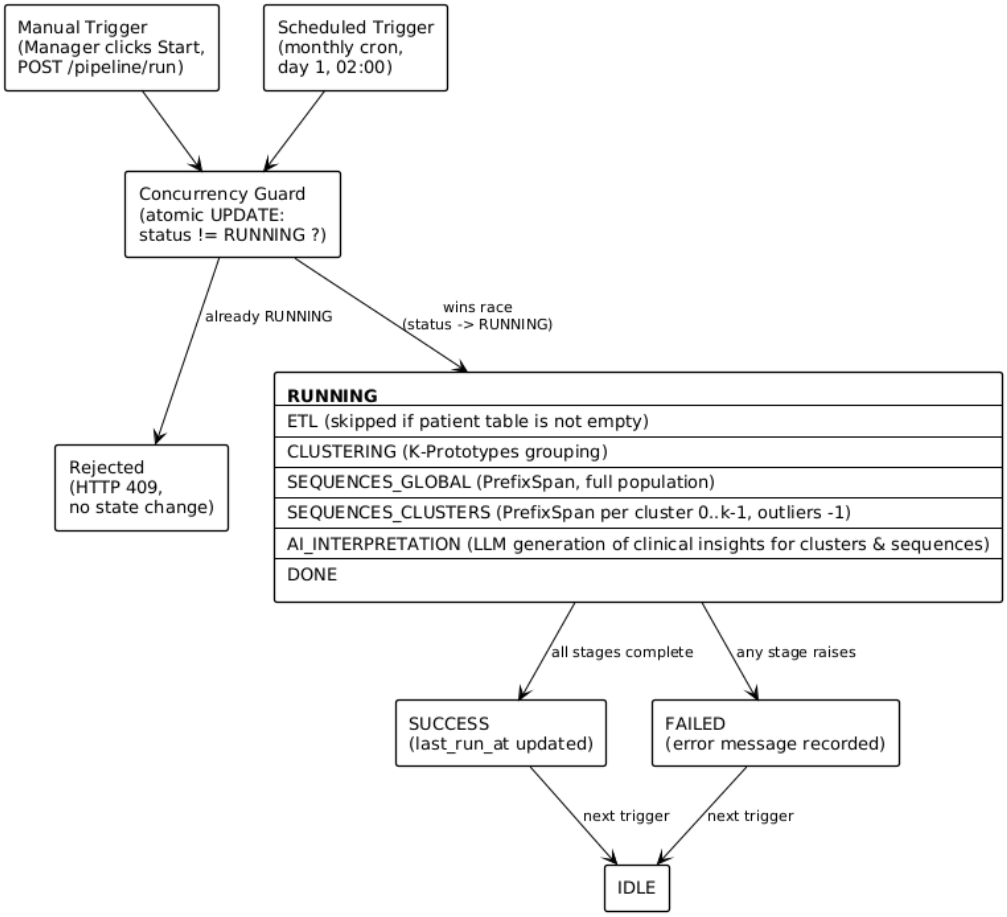


Figure 5.12: Analytics pipeline orchestration: stages and state transitions

Each module can also be run standalone, straight from the backend. That is how the benchmarking in Chapter 6 was produced:

```
python app/clustering_algorithms/k_prototypes/k_prototypes.py

python app/clustering_algorithms/prefix_span/prefix_span.py
python app/clustering_algorithms/prefix_span/prefix_span.py --n-clusters 4
```

The first command runs K-Prototypes clustering. The other two run PrefixSpan, globally and per cluster.

In normal operation, though, a manager triggers the whole pipeline from the pipeline management page (Section 5.2.1, `POST /pipeline/run`). The same job also runs automatically once a month, on the 1st at 02:00, via an in-process APScheduler job. Both triggers call the identical orchestrator function. An atomic database guard rejects a second trigger with `409 Conflict` if a run is already in progress, so the manual and scheduled paths can never overlap.

5.2.2.5 AI Results Generation

The LLM turns raw statistics into readable text at two different moments. The first is automatic: right after clustering finishes, one prompt writes the short card, title, color, insight, and recommendation, shown for every cluster on the dashboard. The second is manual. The first time a manager opens the detail view of a cluster or of a mined sequence, a separate prompt writes the full clinical interpretation on demand. The result is cached in the object's `ai_analysis` JSONB column, so it is written only once.

All three prompts follow the same shape. A fixed set of statistics is injected as *expected input*, and the *prompt* states explicit rules grounded in the clinical prompt-engineering guidance of Liu et al. [15]. The framing is guideline-anchored and evidence-based, bias mitigation is explicit, and the two manual prompts add an internal chain-of-thought step. The model must then answer with a strict *expected output*, either a pipe-delimited line or a JSON object. This keeps every response a paraphrase of the numbers it was given, framed as the "preliminary draft a clinician must verify" that the paper itself recommends, and never a number the model invented.

Cluster Cards at Creation

`get_clusters_summary`, using the template in `app/prompts/clusters`, builds one prompt covering every cluster at once and sends it via `ask_ai`.

Expected input. Per cluster: its size and population share, key numeric averages (age, QALY, DALY, procedures, medications, active conditions), the demographic and disease-rate mix, the dominant care plan share, and the top conditions, medications, and procedures.

```
Structure your response EXACTLY as: [Cluster ID] | [#HexColor] | [Short Title]
| [1-2 sentence Description] | [1 short Insight] | [1 short Recommendation]

Title rules: 2-4 words, plain everyday language, no medical jargon
(e.g. "Older Adults with Chronic Conditions", "Young and Healthy").
Description rules: 1-2 short sentences a non-medical reader can follow.
Insight: pick the MOST distinguishing signal, using concrete numbers from the
data only; do NOT base it on demographics (age, gender) alone.
Recommendation: tie the action to that insight and to standard care for the
disease focus (cardiac dominance -> cardiology follow-up per ACC/AHA;
polypharmacy -> medication review; long gap since last visit -> outreach).

Choose a hex color that reflects the cluster's clinical profile: red for
high-risk, green for healthy, blue for elderly chronic, orange for moderate,
purple for psychiatric, teal for maternal. Use strong, saturated colors, not
```

```
pastel tones. The outlier cluster (-1) is always gray (#9CA3AF).
Do NOT use em dashes anywhere in the response.
```

Expected output. One pipe-delimited line per cluster:

```
[Cluster ID] | [#HexColor] | [Short Title] | [1-2 sentence Description]
             | [1 short Insight] | [1 short Recommendation]
```

`_parse_cluster_summaries` splits each line on `|` and writes it straight into the `cluster` table, so this is exactly what the dashboard cards show.

On-Demand Analysis: Clusters and Sequences

Opening a cluster or a sequence for the first time triggers a second, heavier prompt, `PROMPT_AI_ANALYSIS` for clusters and `PROMPT_SEQUENCE_ANALYSIS` for sequences, both built the same way and both answering in strict JSON.

Expected input (cluster). The same per-cluster statistics as above, but each metric paired with its population average, plus the clustering quality scores (silhouette, Davies–Bouldin), so the model can describe every deviation as "higher" or "lower" than average.

```
Base every claim strictly on the numbers provided. This is clinical decision
support, not a diagnosis: a preliminary draft a clinician must verify.
Gender: "balanced" if 40-60%, "more female/male" if 60-70%, "predominantly
female/male" only if >70%.
Translate vitals into plain risk language (systolic BP >=140 = hypertensive,
glucose >=126 = diabetic range, BMI >=30 = obese).
Let the dominant careplan category guide the clinical framing (Cardiac,
Respiratory, Metabolic, Maternal, Musculoskeletal, Psychiatric, Other).
Evidence-based framing: anchor risks and interventions to recognized
guidelines (ADA, ACC/AHA, KDIGO, GOLD) only when the data supports that focus;
never quote a threshold you are unsure of.
Bias mitigation: do not attribute risk to age, gender, or marital status
beyond what the prevalence and vitals data explicitly support.
If a value is "N/A" or 0 with no clinical meaning, do not mention that metric.

REASONING (work through internally, do NOT include in the output):
1. Identify the dominant careplan and the prevalences with the largest deltas.
2. Translate the mean vitals and quality-of-life metrics into risk language.
3. Decide the overall risk level (high / moderate / low).
4. Derive interventions from the largest deltas and dominant disease focus.

TASK: analyse this group for TWO audiences: clinicians (2-3 sentences,
anchored on dominant careplan and top prevalence deltas) and the general
public (1-2 sentences, plain language, no medical terms).
```

Expected output (cluster). Strict JSON, parsed by `parse_ai_json` and cached into `cluster.ai_analysis`:

```
{
  "risk_level": "high or moderate or low",
  "medical_interpretation": "...",
```

```
  "layperson_summary": "...",
  "key_characteristics": ["...", "...", "...", "..."],
  "priority_interventions": ["...", "...", "..."],
  "care_recommendations": ["...", "...", "..."]
}
```

Expected input (sequence). The ordered pattern items with their `C:`/`M:`/`P:`/`CP:` type tags, its support and population share, and the same demographic, vitals, and disease-rate summary, computed only over the patients who follow that pattern.

```
A "care trajectory" is an ordered sequence of clinical events. The arrow (->)
denotes temporal ordering; items in parentheses occurred on the same day.
Interpret item prefixes: C: = Condition, M: = Medication, P: = Procedure,
CP: = CarePlan.
Focus on the clinical narrative: what is the likely disease progression or
care pathway this sequence represents? Frame it as a plausible
interpretation of the ordering, not a proven causal chain.
Gender, vitals, evidence-based framing, and bias mitigation follow the same
rules as the cluster prompt above.

REASONING (work through internally, do NOT include in the output):
1. Read the sequence in order and map each item to its clinical meaning.
2. Infer the most plausible disease progression or care pathway.
3. Cross-check that inference against the patient statistics for this group.
4. Identify the key risks or care gaps and the recommendations that follow.

TASK: explain this trajectory for TWO audiences: clinicians (2-3 sentences,
referencing specific items in the sequence and the patient statistics) and
the general public (1-2 sentences, plain language, no medical terms).
```

Expected output (sequence). Strict JSON, parsed by `parse_ai_json` and cached into `prefix_span_pattern.ai_analysis`. The schema is shorter than the cluster one: a pathway calls for a narrative rather than a static profile.

```
{
  "clinical_interpretation": "...",
  "layperson_summary": "...",
  "risk_signals": ["...", "...", "..."],
  "recommendations": ["...", "...", "..."]
}
```

Neither object is produced twice: once cached, every later visit to that cluster or sequence is served straight from the database.

5.2.2.6 LLM SQL Agent (LangGraph)

The SQL agent lives in `app/service/lllm_sql_agent_service/` and lets a user query the database in plain natural language. It is built as a **LangGraph** state machine: a directed graph of node functions over one shared, typed state, compiled once and invoked per request via `graph.invoke(initial_state, {"recursion_limit": 20})`, shown in Figure 5.13. Each node makes at most one call to `ask_ai` and returns a partial state update. Every prompt is a constant in `app/prompts/sql_agent`, grounded against one

shared `DB_SCHEMA` string, and follows the clinical prompt-engineering guidance of Liu et al. [15] reviewed in Section 3.2.

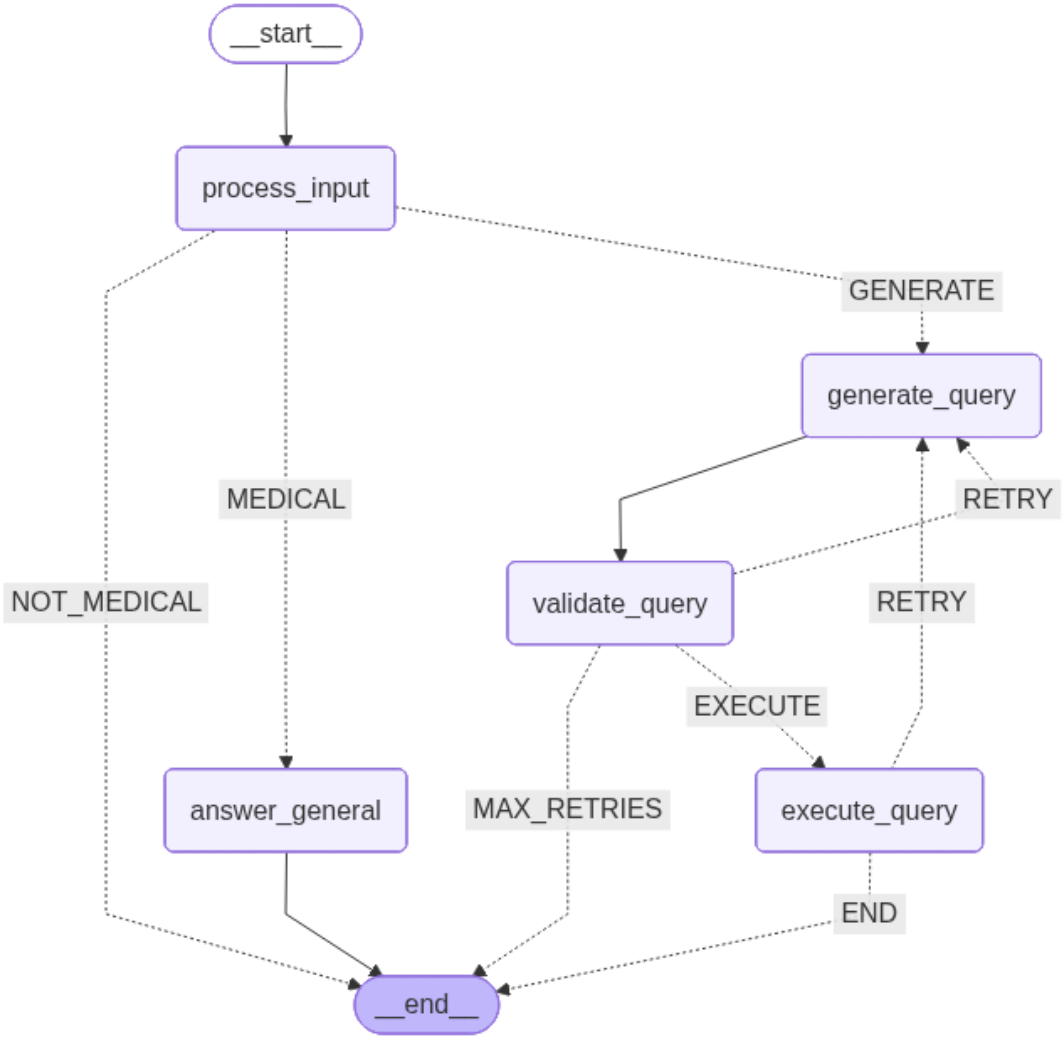


Figure 5.13: LangGraph state machine for the SQL agent

`process_input` is the entry point. Using the last 5 conversation turns as context, it classifies the message into one of three routes. `NOT_MEDICAL` is out of scope, and the graph ends immediately with a fixed refusal. `MEDICAL` is a health question that needs no database access, answered directly by `answer_general`. `GENERATE` is a data question, rewritten as one standalone English sentence and passed on.

`generate_query` drafts a SQL statement from that sentence. `validate_query` then checks it in two independent passes before anything touches the database. A static filter, `is_select_only`, first rejects any statement that is not a bare `SELECT`/`WITH` query, or that contains a blacklisted keyword (`INSERT`, `UPDATE`, `DELETE`, `DROP`, `TRUNCATE`, `ALTER`, and similar). No write statement can reach the database, whatever the model produces. Only a query that survives this filter is handed to a second, independent LLM call, which audits it against the schema and the original question.

A query that passes both checks moves to `execute_query`, which runs it through the request-scoped session and truncates the result to 3,000 characters. `PROMPT_RESULT` then turns it into a natural-language answer. A failure at either validation or execution

feeds its exact cause back into the next `generate_query` attempt, up to `MAX_RETRIES=3`. Because a single request-scoped SQLAlchemy session is reused across every retry, `execute_query` rolls the session back whenever the database raises an error, so a failed attempt (a bad cast, an unknown column) cannot leave the session in a failed-transaction state and silently doom every subsequent retry in the same request. When the caller has a scoped role, Doctor or Patient, a matching clause restricts both generation and validation to that person's own records. A post-execution check also refuses the answer if the final SQL does not reference the required identifier. As established in Chapter 4, this illustrates provider/patient-level separation rather than acting as a hardened access-control boundary, since the platform runs only on synthetic data.

Validating the Generated Query

`validate_query` is the one prompt worth detailing on its own, since it is what keeps the agent read-only.

Expected input. The original question, the candidate SQL, the `DB_SCHEMA` string, and, if the caller is scoped, the doctor or patient clause.

```
Decide if the SQL is safe AND correctly answers the request.
Validation checklist (reject if ANY fails):
1. Must be a single SELECT or WITH ... SELECT.
2. Every referenced table and column must exist in the schema.
3. JOIN conditions must use documented foreign-key relationships.
4. Query logic must answer THIS specific question.
5. Cluster questions: reject if `NOT is_outlier` is missing.
6. Patient counts: reject COUNT(*) where COUNT(DISTINCT patient_id) is needed.
7. If a MANDATORY ROLE SCOPE is present: reject if its filter is missing.
Do NOT flag: ILIKE vs =, aliasing, ordering, whitespace.
```

Expected output. Line 1: `YES` or `NO`. Line 2, only if `NO`: a one sentence reason, which becomes the feedback for the next `generate_query` attempt.

The Remaining Prompts

The other four prompts are simpler variations on the same idea. `PROMPT_TRANSLATE` does the classification described above, and rewrites data questions into one clean English sentence, using the chat history and the schema only to resolve ambiguous terms. `PROMPT_MEDICAL_ANSWER` answers general health questions directly: grounded in established clinical knowledge, never inventing a statistic or a personal diagnosis, and always closing with a disclaimer to consult a licensed professional. `PROMPT_GENERATE_SQL` writes the actual query, carrying every project-specific rule: `LIMIT`, exact-cased enums for categorical codes (matching how Synthea actually stores them, e.g. lowercase `race`/`ethnicity` values), `COUNT(DISTINCT patient_id)`, the JSONB accessors for `cluster_patient.features`, and the mandatory `NOT is_outlier` filter. For free-text columns such as `description` it uses `ILIKE`, but rather than wrapping the user's whole phrase in one pattern, it is instructed to split it into its distinct keywords (drug or condition name, strength, form) and `AND` together one `ILIKE '%keyword%'` per keyword, since clinical names rarely share the exact word order of a question (e.g. "Metformin extended release tablet" versus

the stored "24 HR Metformin hydrochloride 500 MG Extended Release Oral Tablet"). For sequence-mining questions it distinguishes a pattern's raw `support_count` ("most frequent") from its curated `rank` ("most representative"), and, for "how many patterns were mined for cluster X", filters `prefix_span_pattern.cluster_label` directly instead of joining through `prefix_span_patient_pattern`, which instead answers the different question of which patterns have a member in cluster X. On a retry, it also receives the previous query and exactly why it failed. `PROMPT_RESULT` phrases the final row set the way a data analyst would, never mentioning SQL, tables, or columns.

All three query-facing prompts share one static resource, `DB_SCHEMA` (`db/db_data.py`): a description of the 16 tables the agent may query (the `patient_provider` junction and the `pipeline_run` state table are deliberately left out), with every column, type, foreign key, and JSONB access convention, so the model never has to guess the schema.

```
DB_SCHEMA = """
OVERVIEW:
- 16 tables: 4 infrastructure (patient, organization, provider, encounter)
  + 7 clinical (observation, condition, medication, procedure, immunization,
    allergy, careplan) + 2 sequence-mining + 3 clustering.
- Encounters link Patients to Providers and Organizations (many-to-one).
- Only use clustering or sequence tables when the user explicitly asks
  about clusters, patient segments, care trajectories, or sequences.

TABLE: patient
  - patient_id (str, PK) - UUID. Never treat as integer.
  - gender (str) - Values: 'M' (Male), 'F' (Female). Do NOT use 'Male'/'Female'.
  - marital (str) - Values: 'M', 'S', 'D', 'W'.
  ...
"""
```

Every other table follows the same pattern: its columns, their types, and any value it would be unsafe for the model to guess (foreign keys, enum codes, nullable dates). This last part matters in practice, not just in principle: `prefix_span_pattern.cluster_label` was initially missing from `DB_SCHEMA` even though the underlying model has it, and the model would then either omit it and fall back to a join-based approximation, or reference it correctly only for the validator (reading the same incomplete schema) to reject the query as touching a non-existent column. Keeping `DB_SCHEMA` in lockstep with the SQLModel definitions is therefore as load-bearing as the prompts themselves.

When the caller is a Doctor or a Patient, an extra block of text, `DOCTOR_SCOPE_CLAUSE` or `PATIENT_SCOPE_CLAUSE`, is inserted into `PROMPT_GENERATE_SQL` and `PROMPT_VALIDATE_SQL` right after the schema, carrying that person's own `provider_id` or `patient_id` and instructing the model to restrict every query to it. As established in Chapter 4, this is a demonstration of role-aware scoping rather than an enforced access-control boundary, since the platform runs only on synthetic data.

Chapter 6. Testing and Validation

6.1. Used Dataset

This thesis uses the open-source Synthea generator to produce a synthetic but realistic electronic-health-record dataset, generating patient demographics together with the clinical events (conditions, medications, procedures, immunizations, allergies, care plans, and observations) of 20,000 living patients (with a further 2,284 patients who die during the simulation, giving 22,284 patient records in total), each with a decades-long history. Table 6.1 summarizes the size of each source file.

Synthea CSV	Rows
`patients.csv`	22,284
`organizations.csv`	5,833
`providers.csv`	5,833
`encounters.csv`	1,211,526
`allergies.csv`	21,761
`procedures.csv`	3,225,114
`medications.csv`	913,100
`immunizations.csv`	335,814
`careplans.csv`	70,197
`conditions.csv`	727,501
`observations.csv`	14,250,746

Table 6.1: Row counts of the ingested Synthea CSV files

As Walonoski et al. [2] show, Synthea's demographic and disease statistics converge toward the official ones as the simulated population grows, so a cohort of this size is expected to carry a realistic epidemiological signal.

Statistic	Value
Patients	22,284
Mean age	37.2 years
Female	49.6%
Alive / Deceased	89.8% / 10.2%
Diabetes prevalence	11.9%
Hypertension prevalence	19.5%
Heart disease prevalence	15.3%

Table 6.2: Population-level descriptive statistics for the full analytical cohort

The full analytical population is the 22,284 patients produced by the single-hospital ETL reduction of Section 5.2.2.1. Table 6.2 summarizes it at the population level. These

are the same statistics the platform's own `/statistics` endpoints (Chapter 5) surface on the dashboard, the same figures every cluster in Section 6.2 is compared against, and every "population average" injected into the AI interpretation prompts of Section 5.2.2.5.

The recorded encounters span from 1916 to 2026, a roughly 110-year synthetic timeline. Mining full patient lifetimes over such a span would mix events with no clinical relationship to each other, which is exactly why PrefixSpan (Section 5.2.2.3) restricts each sequence to the 730 most recent days of a patient's own history rather than their entire record.

Race	**Share**
White	79.5%
Black	12.8%
Asian	4.8%
Native American	1.0%
Native Hawaiian	1.0%
Other	0.9%
Ethnicity	**Share**
Non-Hispanic	59.6%
Hispanic	40.4%

Table 6.3: Race and ethnicity distribution across the full population

Table 6.3 shows the population is not demographically balanced: patients are 79.5% White and 40.4% Hispanic, with every other race category in the low single digits. This is precisely the kind of skew the bias-mitigation rules in the AI interpretation prompts of Section 5.2.2.5 guard against, so that a cluster's higher-than-average risk is attributed to its clinical profile and never to its demographic composition.

The patients accumulate 1,211,523 encounters, concentrated in routine care: Table 6.4 shows that ambulatory and wellness visits alone account for more than three-quarters of them, with emergency and inpatient care making up a small minority. This skew is exactly why `preventive_to_reactive_ratio` (Section 5.2.2.2) is a useful clustering feature: most patients sit on the preventive end of the spectrum, and the numerator/denominator split is what separates them from the smaller acute-care-heavy groups.

Encounter class	**Share**
Ambulatory	55.1%
Wellness	21.6%
Outpatient	12.8%
Emergency	4.0%
Urgent care	3.8%
Inpatient	1.6%

Table 6.4: Encounter class distribution across the full population

The 727,499 raw condition records span only 311 distinct descriptions, and the five most frequent are "Medication review due" (147,256), "Stress (finding)" (53,885), "Gingivitis (disorder)" (52,681), "Full-time employment (finding)" (49,706), and "Part-time employment (finding)" (30,620). None of these is a disease. Four of them, the

medication-review entry and the employment/stress screening findings, are administrative or socioeconomic annotations that Synthea attaches to nearly every visit. This is precisely the noise that `ITEM_BLACKLIST_SUBSTRINGS` and `SOCIAL_STATUS_SUBSTRINGS` (Section 5.2.2.3) were built to filter out of the mined sequences before mining even starts. The fifth, gingivitis, is routine dental noise instead, handled separately by the dental cap of the same section. Their sheer frequency here is the empirical justification for both filters, rather than an arbitrary design choice.

Table 6.1 gives the total row count for every clinical table, but says nothing about how those events are spread across patients. Table 6.5 fills that gap: on average a patient accumulates 32.65 conditions, 42.23 medications, 145.09 procedures, 15.08 immunizations, 5.56 allergies, and 3.49 care plans over their lifetime. The maximum column, though, shows the distribution is heavily right-tailed, exactly the skew that motivated the `log1p` transform of Section 5.2.2.2.

Event type	Avg. / patient	Max / patient
Condition	32.65	538
Medication	42.23	2,618
Procedure	145.09	1,955
Immunization	15.08	40
Allergy	5.56	13
Careplan	3.49	14
Encounter	54.37	827

Table 6.5: Per-patient event volume across all clinical event tables

This volume is also sharply age-dependent (Table 6.6). Mean conditions per patient rise from 13.93 in the 0–18 group to 67.54 in the 65+ group, nearly a five-fold increase. Mean medications rise from 8.26 to 139.86 over the same range, more than a sixteen-fold increase. This holds even though the population is not skewed toward older patients: the 0–18 group is in fact the largest, at 25.9%. This gradient is the empirical basis for the high effect sizes `age` (η^2=0.66) and `active_conditions_count` (η^2=0.74) attain as clustering features in Section 6.2. Age is not just a demographic label here. It is a strong proxy for clinical burden.

Age group	% of patients	Avg. conditions	Avg. medications
0–18	25.9%	13.93	8.26
19–35	23.7%	25.34	13.87
36–50	19.8%	31.07	26.56
51–65	17.9%	46.32	71.67
65+	12.7%	67.54	139.86

Table 6.6: Age-group distribution and mean conditions/medications per patient

6.2. K-Prototypes Results

Chapter 4 showed that the K-Prototypes method can successfully partition mixed-type Electronic Health Records data. Still, since the algoritm does not need labels, it will automatically generate clusters whether or not a natural grouping exists. Since no ground truth data exists, internal validation measures are needed. The following paragraphs

introduce six such measures and their ranges, which are the ones used to select the final configuration later in this section.

These metrics fall into three groups, chosen so they give different, complementary information. The Silhouette and Gower Silhouette Scores assess geometric cohesion and separation at the level of individual data points. The Calinski-Harabasz and Davies-Bouldin Indices evaluate, at the macro level, the ratio of inter-cluster to intra-cluster dispersion, with Davies-Bouldin specifically exposing the worst overlaps that averaging may hide. And, because geometric distances cannot isolate feature-level importance, Cramér's V and Eta Squared (η^2) quantify the statistical effect size of individual categorical and numerical features on cluster membership.

Silhouette Score

The Silhouette Score, introduced by Rousseeuw [22], measures how well an item fits into the cluster it belongs to. A high score means that the patient is close to other patients in the same cluster and far away from patients in neighboring clusters. For a data point i assigned to cluster C_a, two quantities are defined.

The *intra-cluster distance* $a(i)$ is the mean distance from point i to all other points in its own cluster:

$$a(i) = \frac{1}{|C_a| - 1} \sum_{j \in C_a,\, j \neq i} d(i, j) \tag{6.1}$$

The *inter-cluster distance* $b(i)$ evaluates the proximity of point i to its closest neighboring group. It is defined as the minimum average distance between i and all instances belonging to any other cluster $C_b \neq C_a$:

$$b(i) = \min_{C_b \neq C_a} \frac{1}{|C_b|} \sum_{j \in C_b} d(i, j) \tag{6.2}$$

The silhouette coefficient of point i is then:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\ b(i)\big)} \tag{6.3}$$

The global Silhouette Score $\bar{s}$ is the arithmetic mean over all n points:

$$\bar{s} = \frac{1}{n} \sum_{i=1}^{n} s(i) \tag{6.4}$$

$s(i)$ is bounded in $[-1, 1]$. When the result for $s(i)$ is near $+1$, this means that data point i has been matched to its proper cluster and is far away from other clusters. If the result is near 0, it shows that data point i belongs to the border of clusters. Lastly, when the result is near -1, it means that the data point has been misclassified. Similarly, $\bar{s}$ is interpreted in the same way. Values greater than 0.5 for $\bar{s}$ suggest a reasonable clustering process, while those above 0.7 imply very good clustering results, as pointed out by Kaufman et al. [19].

In its standard form, the Silhouette Score uses Euclidean distance as $d(i, j)$, which restricts its applicability to purely numerical data.

Gower Silhouette Score

In the case where we have mixed types in the feature space, the Euclidean distance used in the conventional Silhouette Score is no longer defined on all features. The Gower distance introduced in Equation (4.3) is used instead. It is defined for any mixture of quantitative and qualitative variables, and it treats the two attribute types in the same spirit as K-Prototypes itself: numerical values are range-scaled and categorical values are compared by binary agreement.

The $n \times n$ pairwise distance matrix is built using equation (4.3), and the silhouette formulae of Equations (6.1) - (6.4) are directly used on it. While the calculated value of the Gower Silhouette Score $\bar{s}_G$ still lies within the theoretical interval of [-1, 1], the standard textbook conditions (where scores $\bar{s} > 0.7$ are needed for successful clustering) do not apply to clinical data. As said before, clinical datasets naturally contain noise and sparsity, and their clusters overlap in biological features. Given the extreme multidimensional nature of these dataset, in practice, scores of $\bar{s}_G > 0.5$ are rarely observed.

Calinski-Harabasz Index

The Calinski-Harabasz (CH) index [23] measures a clustering based on the ratio of the between-cluster variance and the within-cluster variance. Assuming the dataset contains a total of n data points partitioned into k clusters, we define $\boldsymbol{\mu}$ as the centroid of the entire dataset, and $\boldsymbol{\mu}_j$ as the centroid of cluster C_j. The between-cluster scatter matrix trace is:

$$\operatorname{tr}(B_k) = \sum_{j=1}^{k} |C_j| \cdot \|\boldsymbol{\mu}_j - \boldsymbol{\mu}\|^2 \tag{6.5}$$

The within-cluster scatter matrix trace is:

$$\operatorname{tr}(W_k) = \sum_{j=1}^{k} \sum_{i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \tag{6.6}$$

The CH Index is the normalized raito of these two quantities:

$$\mathrm{CH} = \frac{\operatorname{tr}(B_k)}{\operatorname{tr}(W_k)} \cdot \frac{n-k}{k-1} \tag{6.7}$$

Ranging between $[0, +\infty)$, the higher the value of CH, the more clearly defined the clusters are with greater separation and less variation within themselves. Without any clear defined cutoff value, its only use is for comparison purposes.

Davies-Bouldin Index

The Davies-Bouldin (DB) Index [24] measures clustering quality by quantifying the worst-case overlap between every cluster and its most similar neighbor. For each cluster C_i, the internal scatter σ_i is the mean distance from points in C_i to the centroid $\boldsymbol{\mu}_i$:

$$\sigma_i = \frac{1}{|C_i|} \sum_{j \in C_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\| \tag{6.8}$$

The similarity score between two clusters C_i and C_j can be formulated as:

$$R_{ij} = \frac{\sigma_i + \sigma_j}{\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|} \tag{6.9}$$

For each C_i, the most similar neighbor is the cluster C_j that maximizes R_{ij}:

$$D_i = \max_{j \neq i} R_{ij} \tag{6.10}$$

The DB Index is the mean of these worst-case similarity values over all k clusters:

$$\text{DB} = \frac{1}{k} \sum_{i=1}^{k} D_i \tag{6.11}$$

The DB index ranges between $[0, +\infty)$. In contrast to the CH index, a smaller value is desirable because it suggests small clusters and high separation between different clusters. However, there is no predetermined upper bound to determine poor clustering results, but as a general rule of thumb, a value greater than 1 suggests overlapping clusters.

Cramér's V

In contrast with the previous geometric measures, the following two metrics evaluate the strength of association between cluster membership and individual attributes in terms of effect size.

Cramér's V measure [25] evaluates the strength of association for categorical attributes. The χ^2 statistic is defined for the contingency table of observed frequencies O_{ij} for k clusters and q categories as follows:

$$\chi^2 = \sum_{i=1}^{k} \sum_{j=1}^{q} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \qquad E_{ij} = \frac{R_i \cdot C_j}{n} \tag{6.12}$$

where E_{ij} is the expected frequency, R_i and C_j are the corresponding row and column totals, and n is the total number of observations. The χ^2 statistic is converted into a range $[0, 1]$ using Cramér's V measure, regardless of the table dimensions:

$$V = \sqrt{\frac{\chi^2}{n \cdot \min(k - 1,\ q - 1)}} \tag{6.13}$$

The boundaries are easy to read: $V = 0$ implies no association, while $V = 1$ implies a perfect association. In line with Cohen's guidelines [26], the effect size is considered weak ($V < 0.10$), moderate ($0.10 \leq V < 0.30$), or strong ($V \geq 0.30$).

Eta Squared

Eta Squared (η^2) represents the relationship between clustering and the numeric attribute. This statistic captures the degree to which the variation in the feature can be attributed to the clusters and is based on a one-way analysis of variance (ANOVA).

Let $\bar{x}$ denote the overall mean of the numeric attribute, $\bar{x}_j$ the intra-cluster mean of cluster C_j, and x_{ij} the numeric attribute value of individual i in cluster j. The inter-cluster sum of squares and the total sum of squares are defined as follows:

$$\mathrm{SS}_{\text{between}} = \sum_{j=1}^{k} |C_j| \cdot (\bar{x}_j - \bar{x})^2, \qquad \mathrm{SS}_{\text{total}} = \sum_{j=1}^{k} \sum_{i \in C_j} (x_{ij} - \bar{x})^2 \tag{6.14}$$

Eta Squared is the ratio:

$$\eta^2 = \frac{\mathrm{SS}_{\text{between}}}{\mathrm{SS}_{\text{total}}} \tag{6.15}$$

The value of η^2 lies in the interval [0, 1]. An η^2 value of 0 indicates that there is no relationship between the partitioning and the variance of the feature, while an η^2 value of 1 implies that the numeric attribute value depends entirely on the cluster assignment. Based on Cohen's guidelines [26], an effect size of $\eta^2 < 0.01$ is considered trivial, $0.01 \leq \eta^2 < 0.06$ small, $0.06 \leq \eta^2 < 0.14$ medium, and $\eta^2 \geq 0.14$ large.

The objective of clustering is to partition the patients into distinct groups based on there clinical profiles so that each group can be used as a scope for pattern mining. After removing outliers using the z-score criterion described in Section 5.2.2.2, exactly 21,995 (out of a total of 22,284) patients remained for the K-Prototypes clustering. Patients marked as outliers (`is_outlier = TRUE`) are included in the analysis but not included in the model fitting phase.

Selecting the Number of Clusters

The optimal k was chosen with the "Elbow" method over the within-cluster cost for $k \in \{2, \ldots, 9\}$. Figure 6.1 shows a clear diminishing-returns point at $k = 4$: the cost drop from k=3 to 4 is almost 900 points, versus about 350 from 4 to 5. The choice is corroborated by the 63-configuration benchmark below.

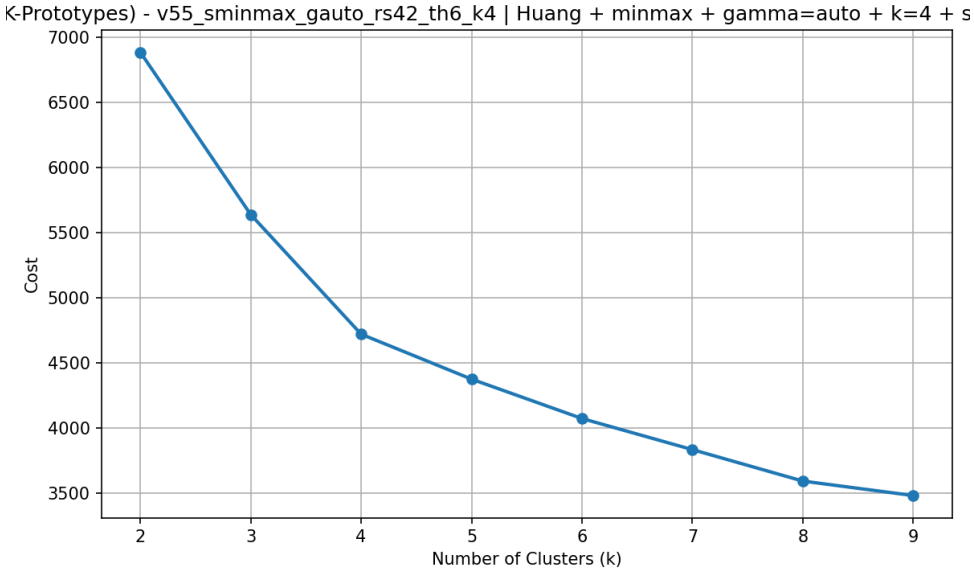


Figure 6.1: Elbow plot used to select the number of clusters

Benchmark Protocol

We benchmarked **63 feature-engineering versions** (`v1` . . . `v63`), each a different subset of the 11 numerical and 10 categorical features from Section 5.2.2.2. These 63 were not chosen upfront. We first ran a few dozen candidate configurations, then repeatedly recombined whichever numerical and categorical features scored highest. We arrived at the final 63 through iterative refinement, not an exhaustive search of every possible subset.

For every version, we swept `scaler` ∈ {`minmax`, `robust`, `standard`} across all four `k` ∈ {4, 5, 6, 7}. The `gamma`, `random_state`, and `threshold` axes were held fixed at `auto`, 42, and 6 throughout. The first 47 versions received the full scaler×k sweep (12 runs each). Once `minmax` proved best, the remaining 16 versions, including the eventual champion `v55`, were tested with `minmax` only across the same four `k` values (4 runs each). This gave 628 runs in total, each writing a `conclusions.txt` with six diagnostic metrics (Silhouette, Gower Silhouette, Calinski-Harabasz, its FAMD variant, Davies-Bouldin, its FAMD variant), per-cluster Gower silhouette, and feature importance (η^2, Cramer's V), all collected into one `general_conclusion.xlsx` sheet and ranked.

Composite Score and Why the Top Result Was Discarded

Only three of the six diagnostic metrics feed the ranking itself. Let g, c, and d denote the Gower Silhouette, the FAMD Calinski-Harabasz, and the FAMD Davies-Bouldin scores after each has been min-max normalized to the $[0, 1]$ range across all 628 runs, using $(x - \min x)/(\max x - \min x)$. The composite score is then then their weighted sum:

$$\text{score} = 0.5\,g \;+\; 0.3\,c \;+\; 0.2\,(1 - d) \tag{6.16}$$

The $(1 - d)$ term flips the Davies-Bouldin direction, since a lower raw score is better there. This weighting is empirical, not derived from theory. Its bias toward the Gower Silhouette, though, follows directly from Section 6.2: it is the only one of the three that respects the mixed numerical/categorical geometry natively. The FAMD variants, by contrast, only see that structure through a lossy 2-D projection. Plain Silhouette, plain Calinski-Harabasz, plain Davies-Bouldin, η^2, and Cramer's V are still computed and reported per run, but play no part in the ranking. They exist only to sanity-check the winner after the fact.

Sorting all 628 runs by Equation (6.16) does not hand back a usable answer directly. The single highest-scoring row was `v42`, a configuration with only `age` as its numerical feature and `gender`, `pregnancy_history`, `race`, and `ethnicity` as categoricals. With so few, mostly binary categorical values to split on, K-Prototypes finds a geometrically tight partition (inflating both Gower silhouette and FAMD-CH) that is clinically empty. It is a coarse demographic split, not a group of patients defined by a distinct clinical profile, so it cannot serve as a scope for the pattern mining in Section 6.3. We therefore discarded every top-ranked run of this kind and instead took, for each k, the highest-scoring configuration whose features included a real clinical signal (disease flags, utilization counts, or care plan), not just demographics.

Feature Tuning per k

Applying that filter and scanning the upper end of the ranking gives the leaderboard in Table 6.7. The top block lists the highest-scoring rows in the neighborhood of

the three winners. The intervening ranks that were skipped are collapsed to $\cdots$, and the recurring `v42` rows are shown in red. The three columns reported are exactly the ones that feed Equation (6.16): the Gower silhouette g, the FAMD Calinski-Harabasz c, and the FAMD Davies-Bouldin d. Scaler, seed, and threshold are omitted since all three winners share the same `minmax` scaler, seed 42, and `threshold=6`.

The red `v42` rows make the discard rule concrete. This demographic-only configuration (a single numerical feature, `age`, plus four low-cardinality categoricals) recurs at ranks 1, 3, 6, and 11 across different k. It even takes the overall top spot, with a score of 0.873. Its metrics are not the problem. They look competitive precisely because a coarse split on `age` and demographics is geometrically easy to separate, which inflates the Gower silhouette. What it lacks is any clinical signal, so it cannot scope the pattern mining of Section 6.3 and is skipped. The first row carrying real clinical features is `v55` at rank 2, our final pick for k=4. `v40` (rank 12) and `v60` (rank 16) are the corresponding best clinical configurations for k=5 and k=6.

#	Config	k	Feat. $(n+c)$	Gower g	CH_F c	DB_F d	Score
1	v42	5	1 + 4	0.547	16475	0.99	0.873
2	**v55** (final pick)	**4**	5 + 8	**0.350**	26813	0.72	**0.818**
3	v42	4	1 + 4	0.494	15898	1.14	0.816
				$\cdots$			
6	v42	7	1 + 4	0.506	12826	1.47	0.786
				$\cdots$			
11	v42	6	1 + 4	0.466	12959	1.84	0.747
12	v40 (best k=5)	5	4 + 8	0.324	22547	0.87	0.741
				$\cdots$			
16	v60 (best k=6)	6	4 + 8	0.287	22273	0.95	0.703

Per-cluster Gower silhouette

v55 (k=4)	c_0=0.423,	c_1=0.199,	c_2=0.320,	c_3=0.396	
v40 (k=5)	c_0=0.277,	c_1=0.225,	c_2=0.311,	c_3=0.186,	c_4=0.484
v60 (k=6)	c_0=0.166,	c_1=0.132,	c_2=0.469,	c_3=0.210,	c_4=0.384, c_5=0.271

v55	v40	v60
Numerical Features		
`age`	`age`	`age`
`active_conditions_count`	`active_conditions_count`	`active_conditions_count`
`total_medications`	`total_medications`	`total_medications`
`total_procedures`	`lifetime_medication_days`	`preventive_to_reactive_ratio`
`preventive_to_reactive_ratio`		
Categorical Features		
`gender`	`gender`	`gender`
`pregnancy_history`	`pregnancy_history`	`pregnancy_history`
`has_heart_disease`	`has_heart_disease`	`has_heart_disease`
`has_diabetes`	`has_diabetes`	`has_diabetes`
`has_hypertension`	`has_hypertension`	`has_hypertension`
`condition_burden`	`condition_burden`	`condition_burden`
`bmi_type`	`bmi_type`	`bmi_type`
`dominant_careplan`	`cholesterol_risk_category`	`cholesterol_risk_category`

Table 6.7: Composite-score leaderboard and the three winning configurations.

Below the leaderboard, the table also reports each winner's per-cluster Gower silhouettes and its full feature set. Base feature sets are nearly identical across the three winners, with minor substitutions in secondary numerical metrics and categorical variables (e.g., `v55` uses `dominant_careplan` instead of `cholesterol_risk_category`).

Final Configuration

`v55` uses `age`, `active_conditions_count`, `total_medications`, `total_procedures`, and `preventive_to_reactive_ratio` numerically, and `gender`, `pregnancy_history`, `has_heart_disease`, `has_diabetes`, `has_hypertension`, `condition_burden`, `bmi_type`, and `dominant_careplan` categorically (Table 6.7). With that feature set fixed, `v55` was run with `minmax` scaling, already established as the best of the three scalers on the first 47 versions, across the four `k` values. Gamma, random state, and threshold stayed at their fixed values throughout the benchmark (auto ≈0.084, seed 42, threshold 6).

`v55` (k=4) posts the highest overall Gower silhouette of the three clinical winners (0.350, versus 0.324 for `v40` at k=5 and 0.287 for `v60` at k=6). All four of its clusters sit at the 0.20 stability heuristic or within rounding of it (min = 0.199, max = 0.423). Cohesion degrades as k grows: at k=5 one of `v40`'s clusters drops below the heuristic (0.186), and at k=6 two of `v60`'s do (0.132 and 0.166). Since these clusters become the mining scopes for PrefixSpan, we picked the smallest k for which every cluster stays cohesive rather than the highest raw score. `active_conditions_count` (η^2=0.74) and `age` (η^2=0.66) dominate the numerical side. `condition_burden` (Cramer's V = 0.80), `gender` (0.70), `has_diabetes` (0.69), and `has_heart_disease` (0.66) dominate the categorical side.

The cluster archetypes themselves confirm this choice: all three winners describe the same population at different resolutions rather than genuinely different partitions. Each recovers the same stable groups, which the interpretation layer titles, in the final `v55` run, *Younger Injury Care* (Cluster 0, the large young / low-burden adult group), *Pregnancy Focused Adults* (Cluster 1), *Older High Risk* (Cluster 2, the chronically ill high-utilizers), *Kids and Well Visits* (Cluster 3), and the *Complex High Need* outliers. `v40` and `v60` recover the same archetypes but split the large general-adult group further, `v40` into two sub-clusters and `v60` into three. These added sub-clusters are exactly the ones that fall below the 0.20 stability heuristic (0.186 for `v40`, and 0.132 and 0.166 for `v60`), so they carve one coherent group into arbitrary slices instead of surfacing new clinical archetypes. We therefore selected `v55`, the coarsest partition whose every cluster remains both cohesive and clinically distinct. These titles, together with the per-cluster description, insight, and recommendation shown on the platform, are generated by the LLM interpretation layer (Section 5.2.2.5) at each run and are therefore not deterministic: a re-run can word them differently while describing the same groups.

Figure 6.2 plots every patient at the two FAMD coordinates of Section 5.2.2.2, colored by cluster. Each cluster occupies its own region, but the boundaries visibly bleed into one another, especially between clusters 0 and 1. This is expected for two reasons, not a sign of a bad partition. First, FAMD compresses 13 mixed-type features into 2 dimensions for the sole purpose of plotting. K-Prototypes itself never sees this projection and clusters on the full feature space, so some structure that separates patients in 13 dimensions is necessarily lost when flattened to 2. Second, real patients are not 100% distinct. A patient can share some characteristics with one cluster's typical profile and other characteristics with another's, so a degree of visual overlap near the boundaries is exactly what clinically fuzzy category edges should look like.

Visual Inspection of the Final Partition

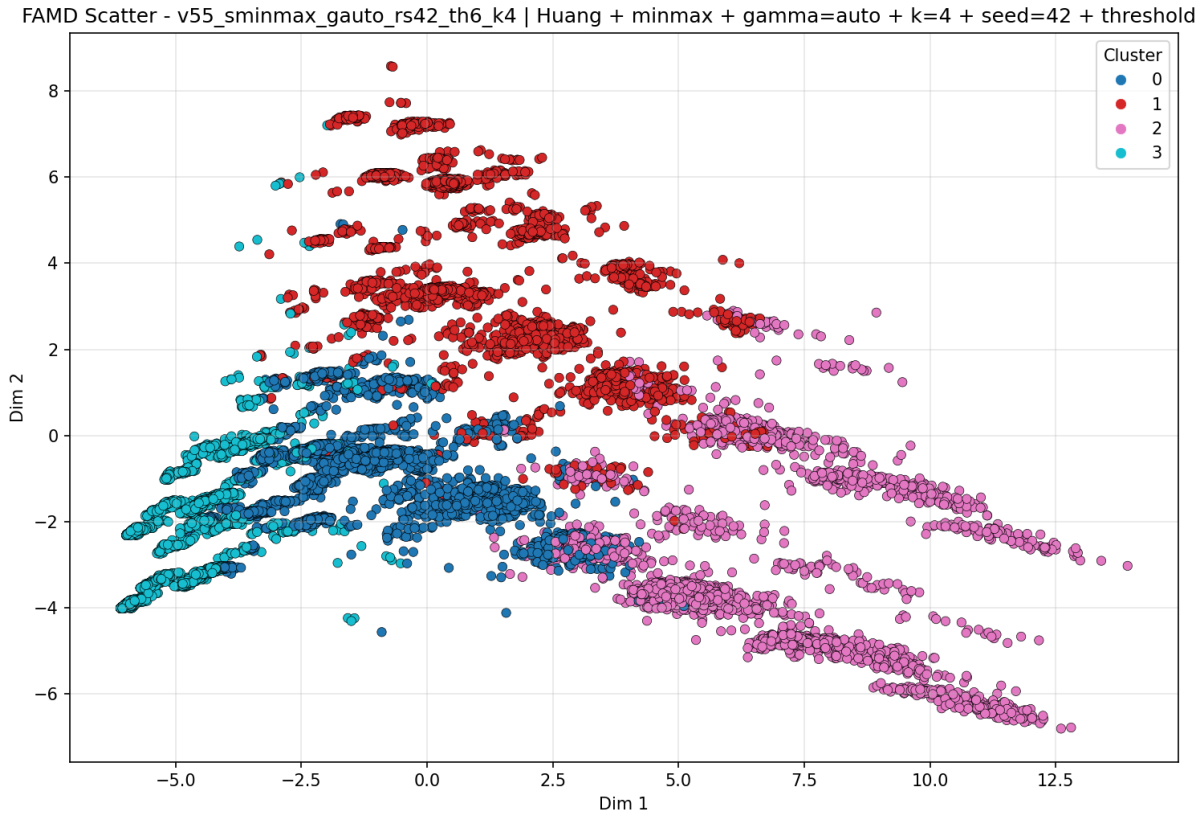


Figure 6.2: FAMD 2-D projection of the four clusters (`v55`, k=4).

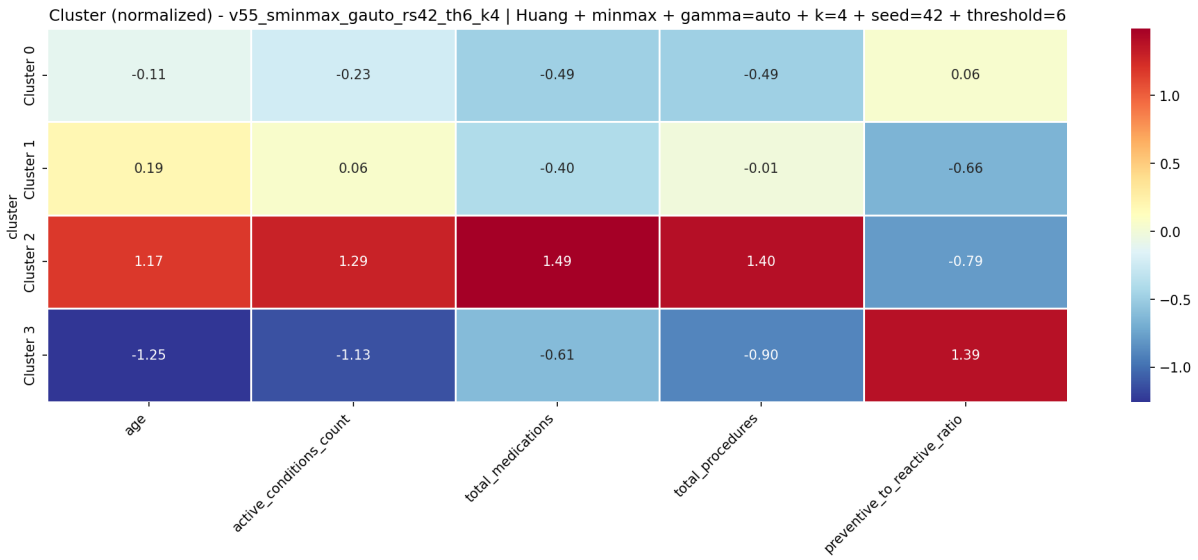


Figure 6.3: Heatmap of z-scored numerical feature means per cluster.

Figure 6.3 reads as a small table: one row per cluster, one column per numerical feature. Each cell is that cluster's mean re-expressed as a z-score against the whole

population, that is, how many standard deviations above (red) or below (blue) the population average that cluster sits on that feature. Read this way, the heatmap shows a clear morbidity gradient. Cluster 2 is the extreme on one end: strongly positive on age, active conditions, medications, and procedures, and the only cluster below average on `preventive_to_reactive_ratio`, i.e., older, high-burden patients. Cluster 3 is the opposite extreme, strongly negative on every burden measure and strongly positive on the preventive ratio, consistent with the pediatric, mostly-wellness-visit profile discussed in Section 6.3. Clusters 0 and 1 sit in between, with cluster 1 skewing slightly older and sicker than cluster 0.

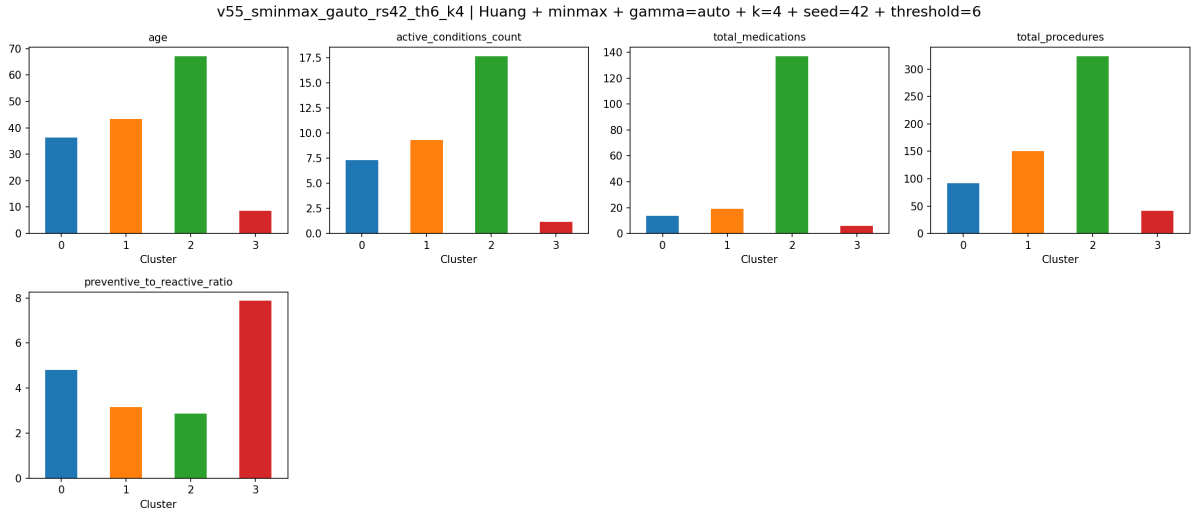


Figure 6.4: Per-cluster means of the five numerical features.

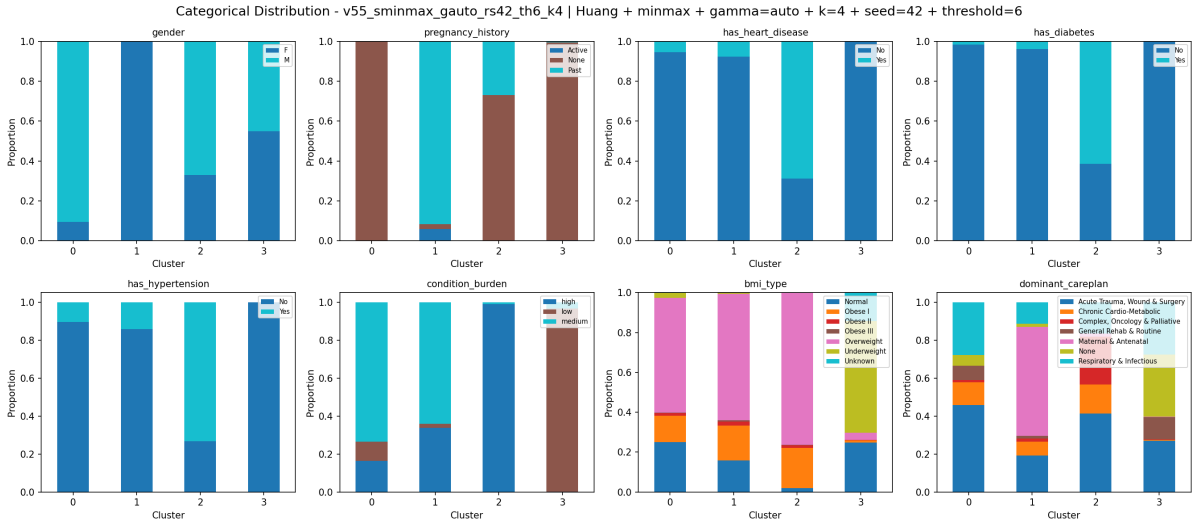


Figure 6.5: Per-cluster proportions of the eight categorical features.

Figure 6.4 shows the same five features, but as raw, unstandardized means rather than z-scores, one panel per feature. It is the natural-units complement to the heatmap: where the heatmap ranks clusters relative to each other, this figure gives the actual scale of the gap. Cluster 2's average age and active-condition count are both roughly nine

times cluster 3's, and its average medication count is over an order of magnitude higher. This confirms that the heatmap's gradient corresponds to a large real difference in clinical burden, not a small one stretched by normalization.

Figure 6.5 gives one panel per categorical feature. Within a panel, each cluster is a single bar split into stacked, colored segments whose heights are the proportion of that cluster in each category, so every bar sums to 1.0. Three patterns stand out. `condition_burden` cleanly separates cluster 2 (almost entirely "high") from cluster 3 (almost entirely the lowest tier), mirroring the numerical panels above. `has_diabetes`, `has_hypertension`, and `has_heart_disease` all concentrate their "Yes" segment inside cluster 2. And `gender` together with `pregnancy_history` isolates cluster 1, which is almost entirely female and carries nearly all of the "Active"/"Past" pregnancy history in the dataset, identifying it as the reproductive-age-women cluster used in Section 6.3.

6.3. Sequence Mining Results

Tables 6.8–6.12 report the ten highest-ranked sequences mined for each of the four clusters and for the outlier scope, out of the several hundred patterns retained per scope. Each row gives the number of patients carrying the sequence (**pts**), its share of the scope (**%**), the geometric-mean-lift score (**gml**, Section 5.2.2.3), and the ordered pattern itself, with every event prefixed by its source table (`C` condition, `M` medication, `P` procedure, `CP` care plan). Rows tagged `[dental]` are majority-dental trajectories, kept but flagged as routine dental care rather than distinctive pathways.

No clinician labeled this data,and Synthea provides no ground-truth set of "correct" pathways to score against. We therefore validate the mined sequences along three empirical axes rather than a single clinical gold standard. The first is a statistical bar: a retained sequence must occur far more often than the frequencies of its individual events would predict. The second is clinical face-validity: the top-ranked sequences of each scope must recover recognizable standard-of-care pathways, and do so in the clinically appropriate cluster. The third is an order-sensitivity null test: a genuine sequence must lose its score once the temporal order that defines it is destroyed. The per-scope tables that follow are read against these three axes in turn.

#	pts	%	gml	Pattern
1	252	3.6	0.97	C:Laceration → CP:Wound care → P:Suture open wound
2	113	1.6	0.78	C:Fracture of bone → CP:Fracture care → P:Bone immobilization
3	106	1.5	0.96	M:Fluticasone/salmeterol DPI → M:albuterol → P:Spirometry
4	432	6.1	0.70	C:Acute bronchitis → CP:Respiratory therapy → M:Acetaminophen
5	2766	39.0	0.82	[dental] M:sodium fluoride gel → P:Dental fluoride treatment → P:Oral health education
6	295	4.2	0.62	C:Acute bronchitis → CP:Respiratory therapy → P:Measurement of respiratory function
7	2705	38.1	0.73	[dental] P:Examination of gingivae → M:sodium fluoride → P:Dental fluoride → P:Oral health education
8	127	1.8	0.73	C:Sprain → C:Sprain of ankle → CP:Physiotherapy care plan
9	229	3.2	0.44	C:Acute bronchitis → CP:Respiratory therapy → P:Measurement resp. function → M:Acetaminophen

10	104	1.5	0.47	C:Streptococcal sore throat → M:Penicillin V → P:Throat culture

Table 6.8: Cluster 0, "Younger Injury Care" (young / low-burden) - top 10 patterns out of 238.

#	pts	%	gml	Pattern
1	154	2.4	0.97	C:Laceration → CP:Wound care → P:Suture open wound
2	121	1.9	0.92	C:Sterilization requested → P:Sterilization education → C:Hist. tubal ligation → CP:Minor surgery → P:Postoperative care
3	123	1.9	0.90	C:History of tubal ligation → CP:Minor surgery care → P:Postoperative care
4	107	1.7	0.81	P:Sterilization ed. → C:Hist. tubal ligation → CP:Minor surgery → P:Ligation of fallopian tubes → P:Postoperative care
5	147	2.3	0.96	M:Fluticasone/salmeterol DPI → M:albuterol → P:Spirometry
6	99	1.5	0.96	P:Prone position → M:Furosemide → P:Oxygen by mask → P:Prone position → M:Furosemide
7	308	4.8	0.71	C:Acute bronchitis → CP:Respiratory therapy → M:Acetaminophen
8	2058	32.1	0.82	[dental] M:sodium fluoride → P:Dental fluoride → P:Oral health education
9	109	1.7	0.79	CP:Minor surgery → P:Ligation of fallopian tubes → P:Postoperative care
10	432	6.7	0.72	C:Normal pregnancy → P:Standard pregnancy test → P:Ultrasound fetal viability

Table 6.9: Cluster 1, "Pregnancy Focused Adults" (women of reproductive age) - top 10 patterns out of 252.

#	pts	%	gml	Pattern
1	154	4.5	0.95	M:Fluticasone/salmeterol DPI → M:albuterol → P:Spirometry
2	164	4.8	0.89	M:Furosemide → P:Oxygen by mask → P:Prone position → M:Furosemide
3	138	4.0	0.81	M:Epoetin Alfa → P:Renal dialysis → M:Epoetin Alfa
4	1222	35.6	0.80	[dental] M:sodium fluoride → P:Dental fluoride → P:Oral health education
5	1163	33.9	0.77	[dental] P:Examination of gingivae → M:sodium fluoride → P:Dental fluoride
6	105	3.1	0.74	M:albuterol → P:Pulmonary rehabilitation → P:Spirometry
7	101	2.9	0.70	M:Fluticasone/salmeterol DPI → M:albuterol → P:Pulmonary rehab → P:Spirometry
8	1162	33.8	0.69	[dental] P:Examination gingivae → M:sodium fluoride → P:Dental fluoride → P:Oral health education
9	1111	32.3	0.68	[dental] P:Removal subgingival plaque → P:Examination gingivae → M:sodium fluoride → P:Dental fluoride
10	97	2.8	0.67	P:Prone position → M:Furosemide → P:Heart failure education

Table 6.10: Cluster 2, "Older High Risk" (hypertensive adults, polypharmacy) - top 10 patterns out of 202.

#	pts	%	gml	Pattern
1	73	1.6	1.00	C:Acute allergic reaction → P:Injection of epinephrine → M:prednisone
2	137	2.9	0.98	C:Laceration → CP:Wound care → P:Suture open wound

continued on next page

#	pts	%	gml	Pattern
3	134	2.9	0.84	CP:Self-care interventions → P:Allergy screening test → M:Epinephrine auto-injector
4	169	3.6	0.81	C:Fracture of bone → CP:Fracture care → P:Bone immobilization
5	459	9.8	0.78	C:Acute bronchitis → CP:Respiratory therapy → M:Acetaminophen
6	48	1.0	0.91	C:Fracture of clavicle → P:Admission to orthopedic dept. → P:Plain X-ray of clavicle
7	93	2.0	0.91	C:History of seizure → C:Seizure disorder → P:Electroencephalogram
8	1873	40.1	0.83	[dental] M:sodium fluoride → P:Dental fluoride → P:Oral health education
9	1856	39.8	0.83	[dental] P:Examination of gingivae → M:sodium fluoride → P:Dental fluoride
10	87	1.9	0.80	C:Concussion injury → C:Concussion no LOC → CP:Head injury rehabilitation

Table 6.11: Cluster 3, “Kids and Well Visits” (pediatric) - top 10 patterns out of 175.

#	pts	%	gml	Pattern
1	7	2.5	1.00	P:Oxygen by mask → P:Prone position → M:Furosemide → P:Oxygen by mask → P:Prone → M:Furosemide
2	6	2.1	1.00	M:Fluticasone/salmeterol DPI → M:albuterol → P:Pulmonary rehab → P:Spirometry
3	142	49.8	0.97	P:Combined chemotherapy + radiation → M:Cisplatin → P:Combined chemotherapy + radiation
4	9	3.2	0.90	P:Renal disorder medication review → M:tacrolimus [Prograf] → P:Transplantation of kidney
5	116	40.7	0.85	M:Cisplatin → M:Paclitaxel → P:Combined chemotherapy + radiation
6	132	46.3	0.85	M:Cisplatin → P:Combined chemotherapy + radiation → P:Hospice care
7	31	10.9	0.78	P:Renal dialysis → M:Epoetin Alfa → P:Renal dialysis → M:Epoetin Alfa
8	35	12.3	0.76	[dental] M:sodium fluoride → P:Dental fluoride → P:Oral health education
9	105	36.8	0.72	P:Combined chemo+radiation → M:Cisplatin → M:Paclitaxel → P:Combined chemo+radiation → P:Hospice
10	81	28.4	0.67	P:Combined chemo+radiation → P:Hearing examination → M:Cisplatin → P:Combined chemo+radiation

Table 6.12: Cluster -1, “Complex High Need” (oncology / renal / heart failure) - top 10 patterns out of 224.

6.3.1. Statistical Bar: Lift over Chance

The first axis is built directly into the ranking. GML (Section 5.2.2.3) is bounded above by 1 and measures how much more often a pattern’s events co-occur, in order, than the geometric mean of their individual frequencies would predict. A value near 1 marks a near-deterministic association, a value near 0 an incidental one. Together with the 1% support floor, this is the retention rule, since a sequence is reported only if it clears both a minimum prevalence and a minimum lift. The leading rows of every table sit in the 0.7–1.0 band, well clear of chance, which is the first, and weakest, evidence that they are not artifacts of a few very common codes.

6.3.2. Clinical Face-Validity and Per-Cluster Relevance

The second axis asks whether these high-lift sequences are clinically meaningful, and specifically whether each cluster's top pathways match the clinical identity the clustering assigned it in Section 6.2. They do, and this alignment is itself empirical evidence: the sequence layer, mined independently of the K-Prototypes partition, agrees with it on the same patient structure. The leading pathways of each scope are valuable *for that scope in particular*, and they surface most strongly exaclty where they belong.

Cluster 0 (Table 6.8), *Younger Injury Care*, the young, low-burden group, is dominated by self-limiting acute care. The laceration-to-wound-care-to-suture chain (`gml` 0.97), fracture-to-immobilization (0.78), and streptococcal-sore-throat-to-penicillin (0.47) are textbook single-episode responses, precisely what a low-comorbidity population accumulates. Cluster 1 (Table 6.9), *Pregnancy Focused Adults*, women of reproductive age, surfaces tubal-ligation and normal-pregnancy pathways at the top (`gml` up to 0.92). These pathways are meaningful only for this cohort, and they would be diluted to nothing in the global pool. Cluster 2 (Table 6.10), *Older High Risk*, hypertensive adults on polypharmacy, recovers the furosemide/prone-position/oxygen sequence of acute decompensated heart failure (0.89) and the dialysis/epoetin-alfa loop of renal-anemia management (0.81), the two regimens that define a high-burden cardiometabolic group. Cluster 3 (Table 6.11), *Kids and Well Visits*, the pediatric group, is led by the acute-allergic-reaction-to-epinephrine-to-prednisone chain (`gml` 1.00) and the allergy-screening/auto-injector pathway (0.84), reflecting the allergy-and-injury profile typical of children. Finally, the outlier scope (Table 6.12), *Complex High Need*, isolates the most intensive protocols: cisplatin/paclitaxel chemoradiation (0.85), the kidney-transplant/tacrolimus sequence (0.90), and repeated dialysis and heart-failure loops. These are the oncology, transplant, and end-stage regimens that justified separating these patients as outliers in the first place.

Each scope's leading pathways are therefore not merely valid in isolation. They are the pathways one would expect from that particular cohort, and they rank highest precisely in the cluster to which they belong. That the mining recovers the clinical logic Synthea used to synthesize the data, in the correct scope, closes the loop between generator and discovery.

6.3.3. Order-Sensitivity Null Test

The third axis is the sharpest. Face-validity confirms that a pattern is clinically sensible, but not that its *order* carries information: three drugs that always co-occur would score highly whether or not they follow a fixed sequence. To isolate the contribution of order, we re-mine each scope after randomly permuting every patient's event timeline (the shuffle hook of Section 5.2.2.3), averaged over three seeds. Because the permutation is within-patient, it preserves every patient's *set* of events, and therefore every single-item frequency and the entire GML denominator. Only chronological order is destroyed. A genuine sequence must lose its score under this shuffle. An incidental co-occurrence must not.

Scope	real	shuffled	drop	real≥0.5	shuf.≥0.5
Cluster 0 (Younger Injury Care)	0.73	0.18	4.0×	8	0
Cluster 1 (Pregnancy Focused Adults)	0.87	0.17	5.2×	10	0
Cluster 2 (Older High Risk)	0.77	0.49	1.6×	10	4
Cluster 3 (Kids and Well Visits)	0.87	0.21	4.2×	10	0
Cluster -1 (Complex High Need)	0.86	0.72	1.2×	10	10

Table 6.13: Temporal-shuffle null test: mean top-ten `gml`, real vs. shuffled, and the count clearing `gml` ≥ 0.5.

The test discriminates cleanly (Table 6.13). For the acute-care clusters 0, 1, and 3, the mean top-ten `gml` collapses by 4.0–5.2×, from 0.73–0.87 down to 0.17–0.21, and *no* pattern clears the 0.5 mark. Under shuffling, the top spots are taken over by generic dental co-occurrences at `gml` ≈ 0.18, while the ordered pathways vanish. For these cohorts, order *is* the pattern, since a laceration must precede its suture.

The chronic-regimen clusters 2 and -1 behave differently, and revealingly. Their scores barely move (a 1.2–1.6× drop), and most patterns survive. These small cohorts are dominated by a few agents repeated in tight loops, such as chemotherapy cycles, dialysis/epoetin loops, and heart-failure furosemide/prone/oxygen loops, which co-occur in nearly every member regardless of order. Their signature is the repeated item: two to four of the surviving top-ten sequences literally repeat an event (`A`→`A`→`B`), a structure shuffling cannot break. This is not a defect of the miner but the correct clinical distinction: an acute pathway *is* its order, whereas a chronic regimen is a recurring bundle in which order is secondary. The null test recovers exactly that difference. In doing so, it confirms that GML behaves as intended: destroyed by shuffling wherever order genuinely matters, and persistent only where the "pattern" is really a co-occurrence set.

6.4. LLM Validation

The platform uses large language models in two roles: the cluster and sequence interpretation of Section 5.2.2.5, and the SQL agent of Section 5.2.2.6. Each one needs a different way to validate it.

6.4.1. Cluster and Sequence Interpretation

No qualified clinician reviewed the AI-generated cluster and sequence narratives. Checking them against real medical judgment is future work (Chapter 8). Until then, we validated the text against the data, not against clinical correctness. For a sample of clusters and mined sequences, we checked that every claim matched the number it was meant to summarize: the right prevalence, the right centroid feature, the right support count. We also checked that no number was invented or assigned to the wrong group. On this narrower test the interpretations held up. They restated the underlying numbers correctly, and we saw no invented figure. This shows that the text agrees with its own numbers. It does not show that the text is medically correct, and the reader should keep that difference in mind in Sections 6.2 and 6.3.

6.4.2. SQL Agent

Unlike the narrative interpreter, the SQL agent (Section 5.2.2.6) can be checked objectively. Once a question is translated into SQL, its answer is either right or wrong. We evaluated it from the perspective of a hospital manager, the role the agent is intended for, on a benchmark of **50 natural-language questions** of mixed difficulty. The questions span three levels. The easy ones are single-table lookups, such as patient counts or the most common condition. The medium ones are filtered aggregates, such as obesity prevalence or the average inpatient stay. The hard ones are true analytical queries: multi-table joins across the provider, encounter, and clinical tables, per-cluster analytics, a thirty-day-readmission self-join, and cross-table set intersections. For each question, we wrote a gold SQL query by hand and ran it directly on the database. We then scored the agent's final answer against that result.

We scored the answers two ways. Under a strict exact match, the agent got 42 of the 50 right. Under a looser rule, which accepts an answer within a one-percent counting tolerance or under a reasonable reading of an unclear question, it got **48 of the 50 correct (96%)**. The six-question gap between the two is small and easy to explain. Four are count differences below one percent, and two are questions that allow more than one reasonable answer. Only two answers are wrong under any reading. Table 6.14 lists the ten hardest questions with the agent's answer and the hand-computed truth. Both real errors and both interpretation differences appear there. In the Expected column, *(miss)* marks a real error, and *(interpretation)* marks a difference that comes only from a reasonable reading of the question.

Question	**Agent Answer**	**Expected**
Which patient has the highest number of encounters, and how many?	Doug McGlynn, 827 encounters.	Doug McGlynn, 827
Which medical speciality performed the most procedures, and how many?	General Practice, 0 procedures.	General Practice, 3,225,114 (miss)
Which provider has seen the most distinct patients, and how many?	Jeanett Goodwin, 21,832.	Jeanett Goodwin, 21,832
Which speciality prescribed the most medications, and how many?	General Practice, 913,099.	General Practice, 913,099
What is the most common procedure performed during inpatient encounters?	Combined chemotherapy and radiation therapy, 34,890.	34,890
How many patients were readmitted to inpatient care within 30 days of discharge?	345 patients.	591 (interpretation)
How many distinct patients have both a Viral sinusitis condition and take Acetaminophen 325 MG Oral Tablet?	6,863 patients.	626 (miss)
What is the average age of patients in the "Older High Risk" cluster?	67.3 years.	67.3
Which cluster has the highest average number of inpatient encounters per patient?	Older High Risk, 1.71.	Complex High Need, 21.0 (interpretation)

What is the most common medication in the "Complex High Need" cluster?	Cisplatin 50 MG Injection, 32,894.	Cisplatin, 32,894

Table 6.14: The ten most complex questions from the 50-question manager benchmark.

The six differences from the strict-match gold fall into three groups: two real errors, two reasonable readings, and four differences below one percent.

Two real errors. Only two answers are wrong under any reading, and both fall on the hardest queries. The first is the speciality that performed the most procedures. Here the agent summed the pre-computed `provider.procedures` column instead of counting the `procedure` table through the encounter join. That column is empty in this extract, so the count dropped to zero, even though the speciality label was right. The base-table join used by the gold query returns 3.2 million. This is a gap in the schema description. The model was never told the column is unreliable. The second error is worse, and it is about free-text name matching. We asked for patients with a *Viral sinusitis* condition who also take *Acetaminophen 325 MG Oral Tablet*. The agent split the drug name into separate keyword filters and matched any product that shared those tokens, including an unrelated opioid-acetaminophen combination. That pushed the count from 626 up to 6,863. Checking the filter against the actual list of stored medication names, instead of keyword matching, would fix it. We leave that as future work.

Two reasonable readings. Two more answers differ from the gold, but not because the agent mistranslated the question. The question itself allows more than one reading, so we count these as correct. The first is thirty-day readmissions. The agent counted only consecutive admissions, that is, the next inpatient stay within thirty days of discharge. This is the standard clinical definition, and it returned 345. Our gold counted any later admission within thirty days of any earlier discharge, and returned 591. The second is the cluster with the most inpatient encounters per patient. The agent read "cluster" as the main clusters only and excluded the outlier group with `WHERE NOT is_outlier`, so it reported *Older High Risk* (1.71). Our gold kept the outlier group, whose members are the heaviest inpatient users, and returned *Complex High Need* (21.0). In both cases the agent's SQL is correct for one valid reading of the request.

Four differences below one percent. The last four are the mild form of the name-matching problem above, and all stay within a one-percent tolerance: a lisinopril patient count of 2,852 against 2,844, a two-drug intersection of 1,504 against 1,497, an over-65 lisinopril count of 974 against 968, and a Houston male-patient count of 1,177 against 1,171. Each is close enough to be the right answer for a manager, which is why the looser rule accepts them. They share the same literal-matching root as the severe case, and the same fix would remove them.

On a benchmark built around multi-table joins and analytics, the agent is correct in practice on 48 of 50 manager questions. Its two hard failures are specific and well understood. The rest of its differences from a hand-written gold are either reasonable readings of unclear wording or count gaps below one percent.

Chapter 7. User's Manual

This chapter has two parts: Section 7.1 explains how to install and run the platform from scratch, and Section 7.2 is a user manual that walks through the interface, role by role.

7.1. Installation

The platform relies on modern, cross-platform technologies, so the steps below install and run it the same way on Windows, Linux, or macOS. It amounts to a Next.js frontend, a FastAPI backend, and one PostgreSQL database, with no container, message broker, or external worker to configure.

7.1.1. Software Requirements

The following software must be installed beforehand:

- **Python 3.11 or newer** for the backend.
- **Node.js 20 or newer** and `npm` for the frontend (Next.js 16 needs this version).
- **PostgreSQL 14 or newer**, which must allow the `pg_trgm` extension used for fast text search.
- **Java 11 or newer**, only if the synthetic dataset is generated locally with Synthea. It is not needed if the CSV files are already available.
- A valid **OpenAI API key** (for the SQL agent and the cluster/sequence interpretation).

7.1.2. Step-by-Step Installation

1. Get the source code. Copy the project onto the machine. It contains two top-level folders, `backend/` and `frontend/`.

2. Configure the backend environment. Create a file named `.env` inside `backend/` pointing to an existing PostgreSQL database and providing the LLM keys. These values are the only thing that changes between a local, testing, or production deployment:

```
POSTGRES_USERNAME=hospitai_user
POSTGRES_PASSWORD=your_password
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=hospitai
GPT_KEY=sk-...              # OpenAI key
```

The application tables, indexes, and materialized views are created automatically the first time the backend starts, so no schema script has to be run by hand.

3. Install the backend dependencies. From the `backend/` folder, create a virtual environment and install the pinned dependencies:

```
python -m venv .venv
.venv\Scripts\activate            # Windows  (use source .venv/bin/activate on
    Linux/macOS)
pip install -r requirements.txt
```

4. Get the dataset. The ETL stage reads the Synthea CSV export (`patients.csv`, `encounters.csv`, `observations.csv`, and the other clinical files) from the expected dataset directory. Either generate it with the open-source Synthea patient generator (which needs Java) or place a previously generated export at that location. This step is only needed for the very first data load.

5. Start the backend. Still inside `backend/`, launch the API server:

```
uvicorn app.main:app --reload
```

On its first start, the backend creates every table, applies the additive index and constraint migrations, builds the observation materialized views, seeds the pipeline state, and starts the monthly scheduler. The API is then served on `http://localhost:8000`.

6. Start the frontend. In a separate terminal, from the `frontend/` folder, install the packages and run the development server:

```
npm install
npm run dev
```

The interface becomes available at `http://localhost:3000`.

7. Run the analytics from the terminal (optional). Besides the Pipeline page described below, each analytical stage can also be launched directly, which is how the benchmarking in Chapter 6 was produced:

```
python app/clustering_algorithms/k_prototypes/k_prototypes.py

python app/clustering_algorithms/prefix_span/prefix_span.py
python app/clustering_algorithms/prefix_span/prefix_span.py --n-clusters 4
```

The first command runs K-Prototypes clustering. The other two run PrefixSpan sequence mining, globally and per cluster.

8. Load the data and compute the results. Open the interface, sign in as the Manager (see the next section), go to the Pipeline page, and start a run. On this first run the pipeline ingests the dataset, computes the clusters, and mines the sequential patterns. Once it finishes, every analytical view is populated and the platform is ready to use. On later runs the ingested data is reused and only the analytics are recomputed.

7.2. User Manual

This section describes the application from the user's point of view, one role at a time: Patient, Doctor, and Manager.

7.2.1. Signing In

The entry point is the sign-in page (Figure 7.1). The platform ships with three demonstration accounts, one per role:

- `manager@hospitai.com` - the Manager,
- `doctor@hospitai.com` - the Doctor,
- `patient@hospitai.com` - the Patient.

To sign in with a role, the user only has to enter a password of at least 8 characters, since the roles exist to illustrate how the interface changes per user rather than to enforce real security. After signing in, the user lands on the home page of their role and navigates through the menu to the pages that the role exposes.

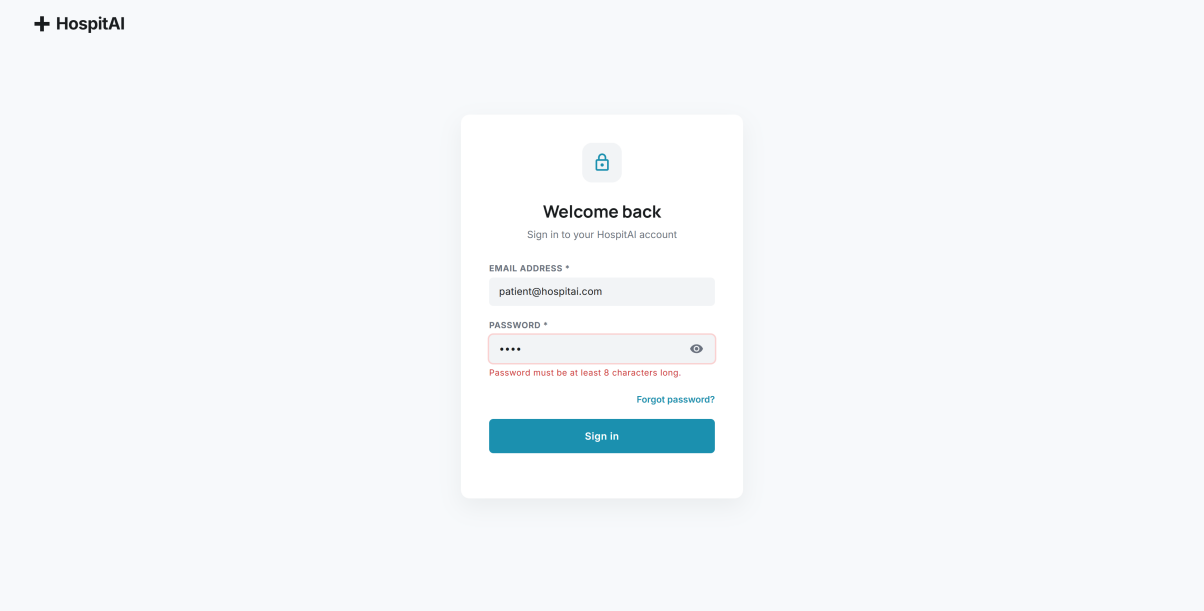


Figure 7.1: Sign-in page with the three demonstration accounts

7.2.2. Patient

The Patient role sees only its ownn data, read-only. On the Home page it can view its complete medical history: conditions, medications, procedures, vaccinations, allergies, and the other clinical categories recorded for it.

The Patient can also open its list of past visits and, for a particular visit, see the test results, conditions, medications, and procedures recorded during that encounter (Figures 7.2 and 7.3).

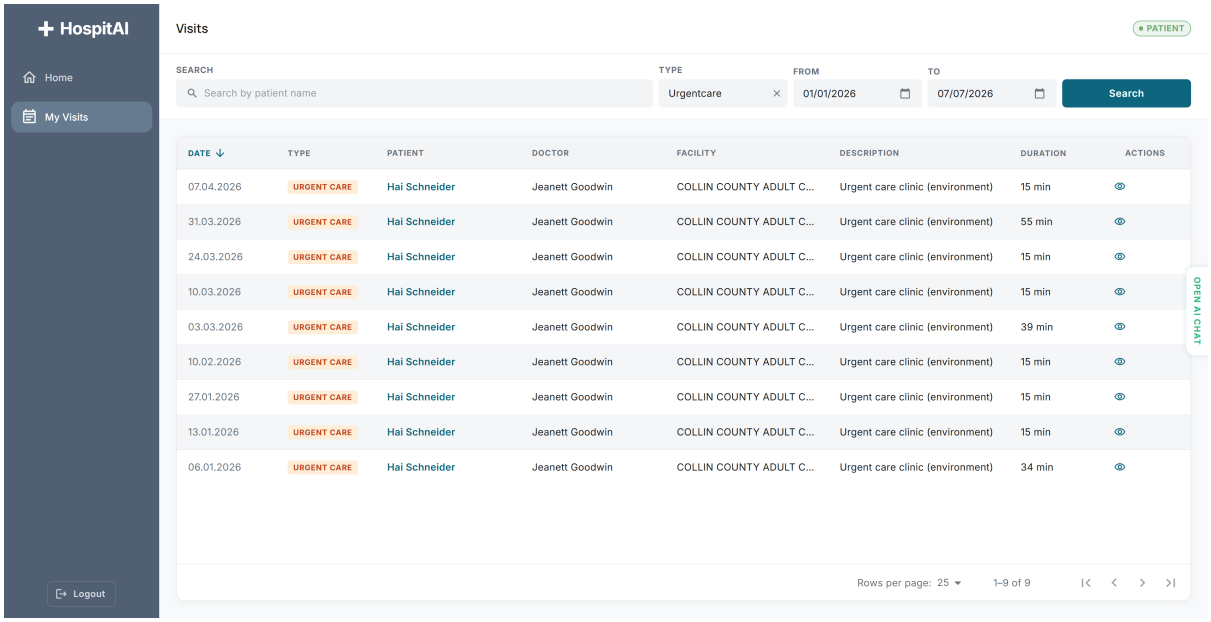


Figure 7.2: Patient view - list of past visits

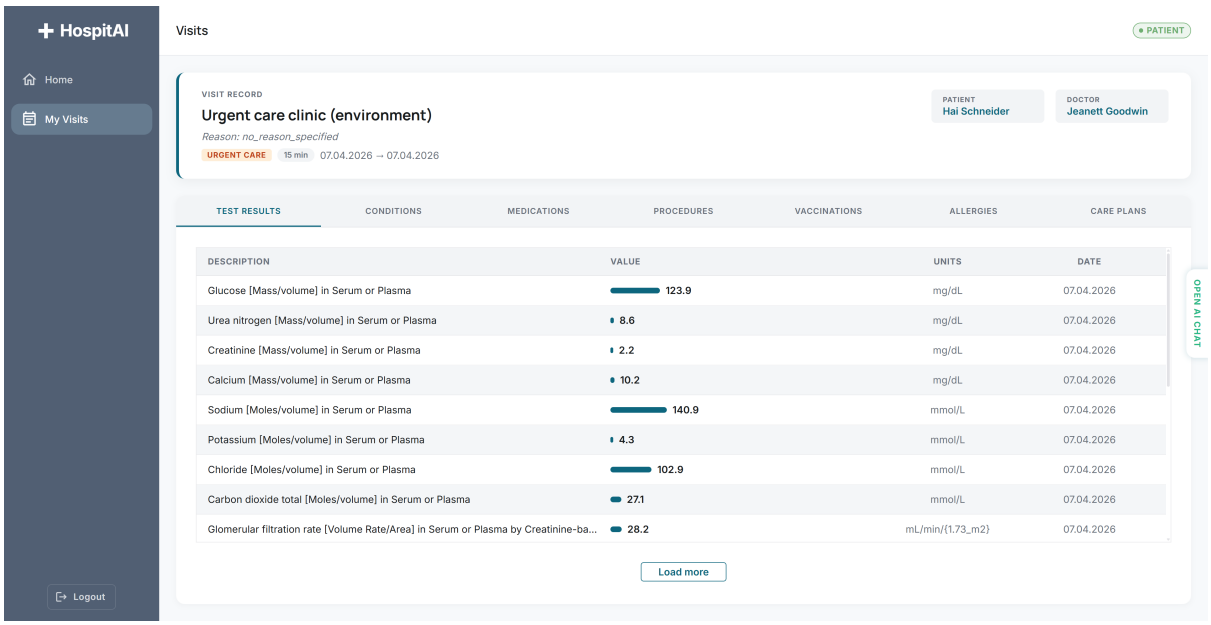


Figure 7.3: Patient view - detail of a particular visit

7.2.3. Doctor

On its Home page, the Doctor sees a dashboard with statistics over its own patients (Figure 7.4).

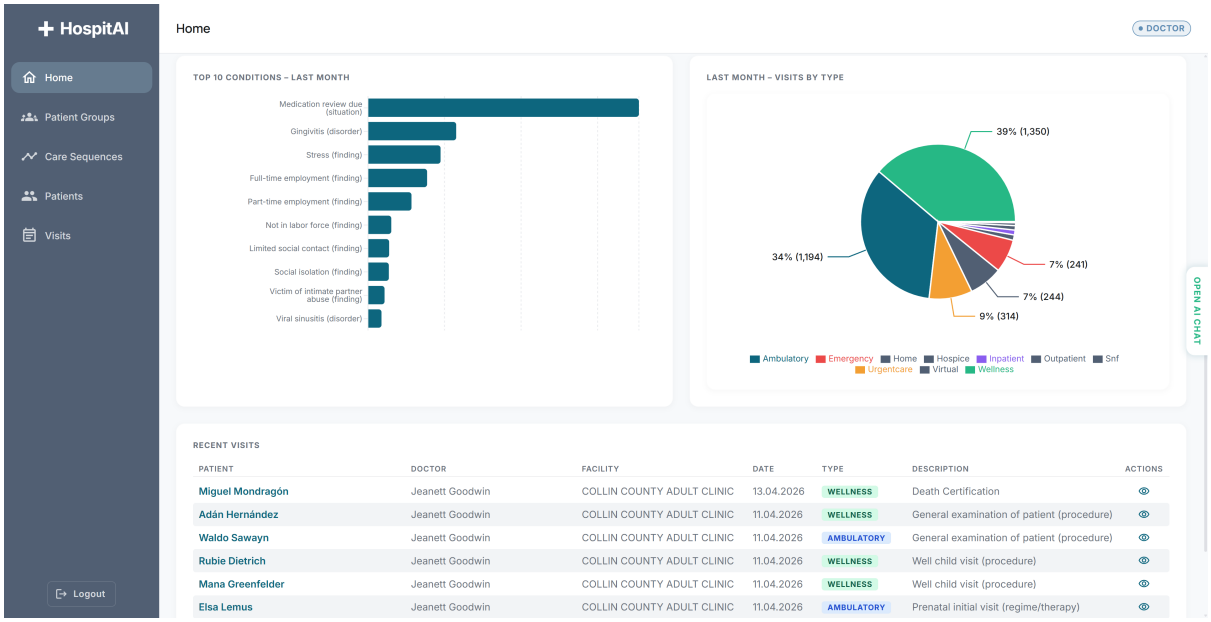


Figure 7.4: Doctor - home dashboard

The Doctor can browse the list of its patients, open a patient, and add, remove, or update entries in its record, such as conditions, medications, or procedures (Figure 7.5).

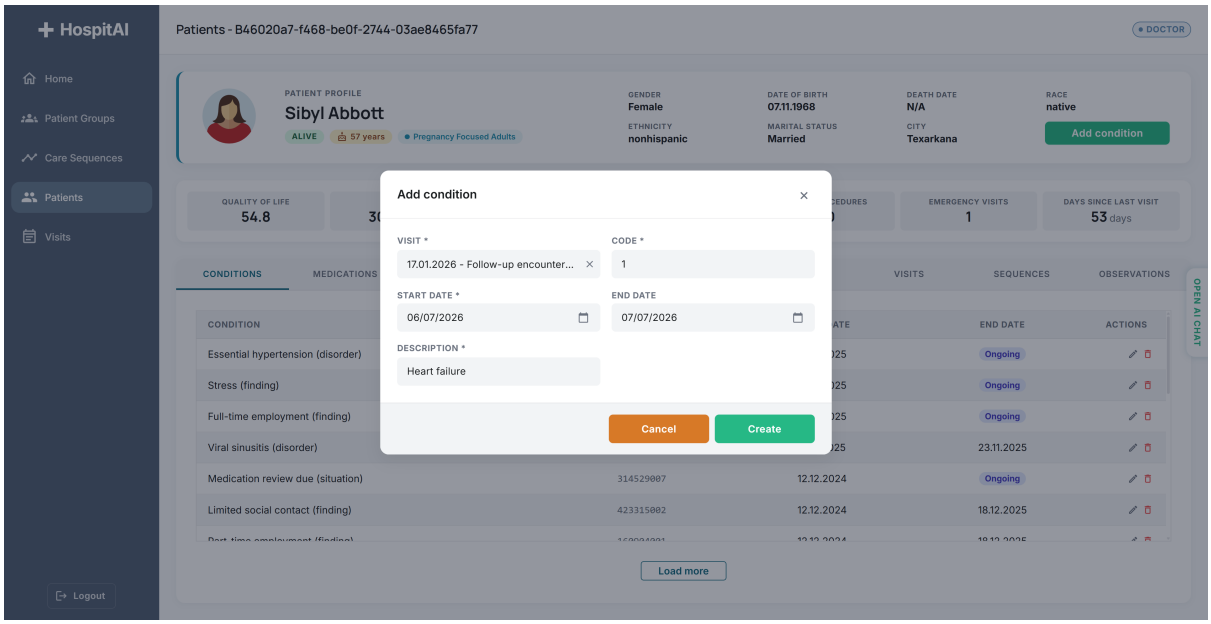


Figure 7.5: Doctor - adding a condition to a patient's record

The Doctor can also see all of a patient's visits and add, delete, or update a visit (Figure 7.6). Within a particular visit, the Doctor can add, edit, or delete its specific values, such as conditions, medications, procedures, and test results (Figure 7.7).

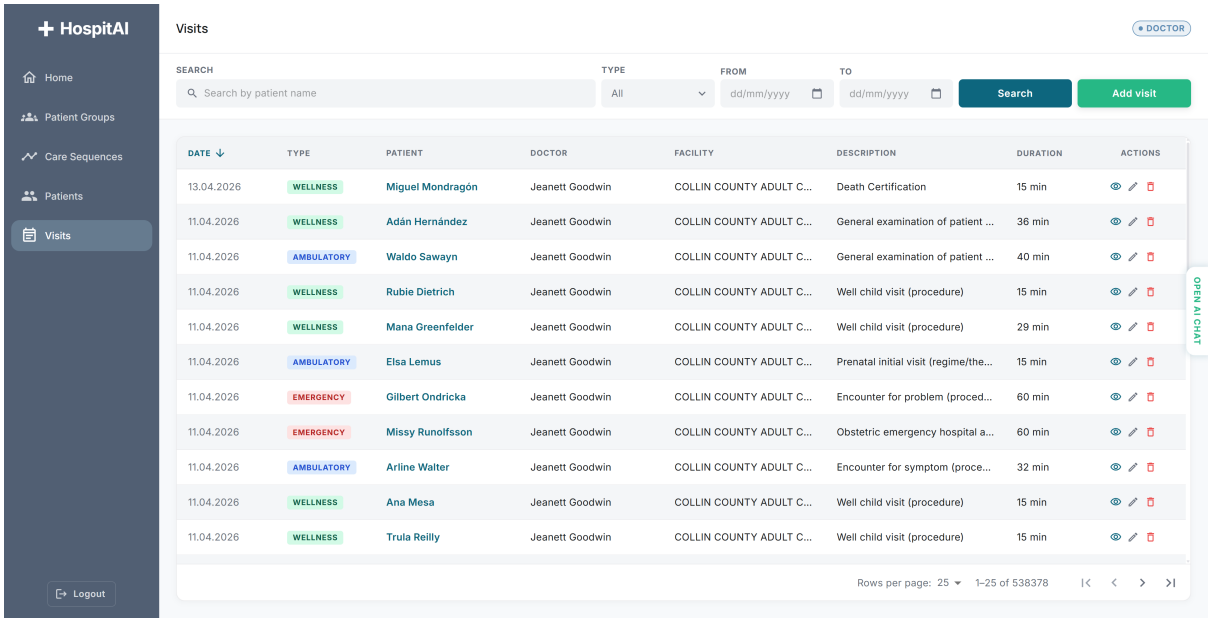


Figure 7.6: Doctor - managing a patient's visits

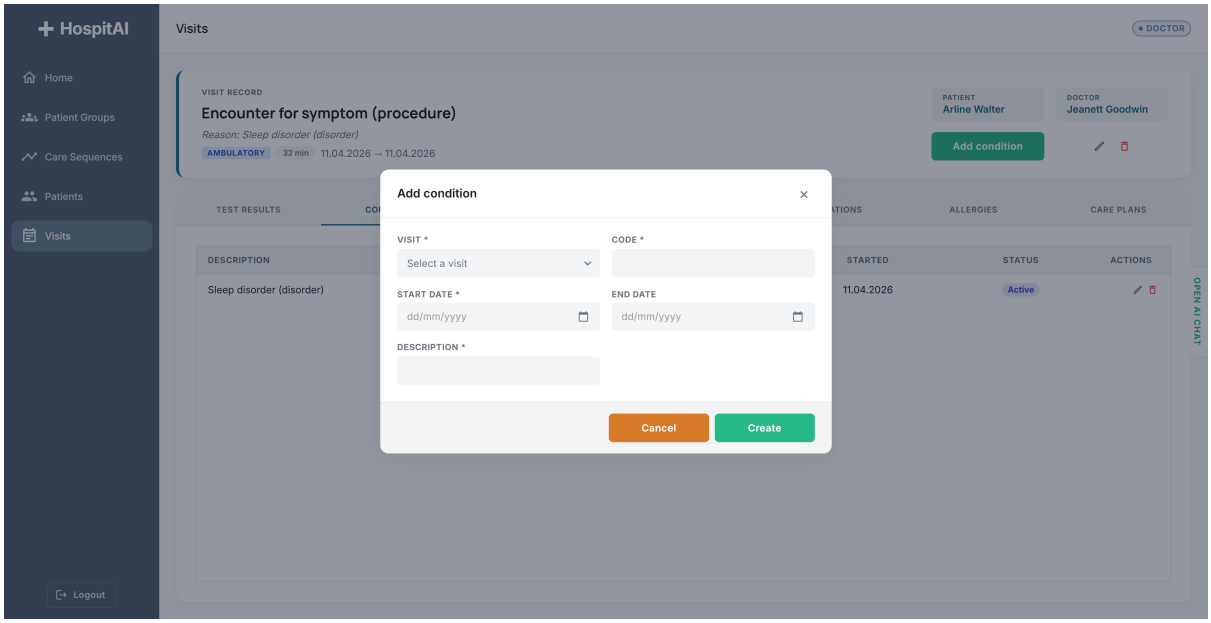


Figure 7.7: Doctor - editing the values of a particular visit

The Doctor can also inspect the patient groups (clusters) and the mined care sequences, either for the whole population or for a particular patient from that patient's profile. Unlike the Manager, though, the Doctor cannot start the analytics pipeline.

7.2.4. Manager

The Manager is the only role that can start and stop the analytics pipeline (Figure 7.8), which by default also runs automatically on the 1st of every month.

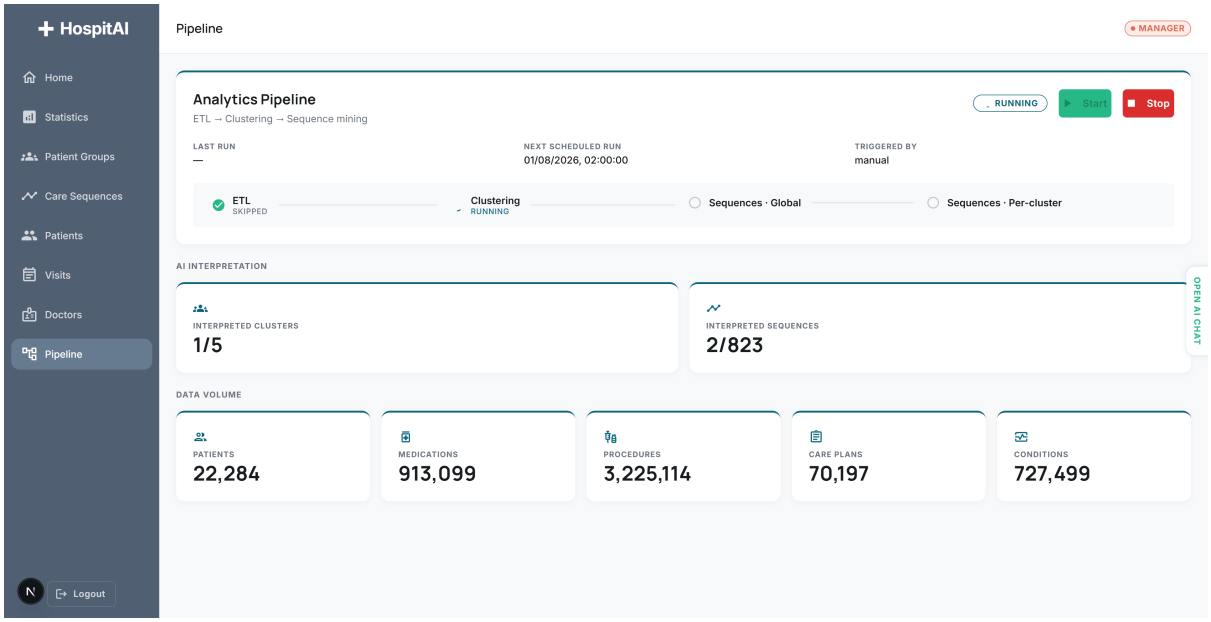


Figure 7.8: Manager - starting and stopping the analytics pipeline

Both the Manager and the Doctor can investigate the resulting clusers (Figure 7.9), request an AI interpretation of a specific cluster (Figures 7.10 and 7.11), and open a specific cluster to see its detail (Figures 7.12 and 7.13).

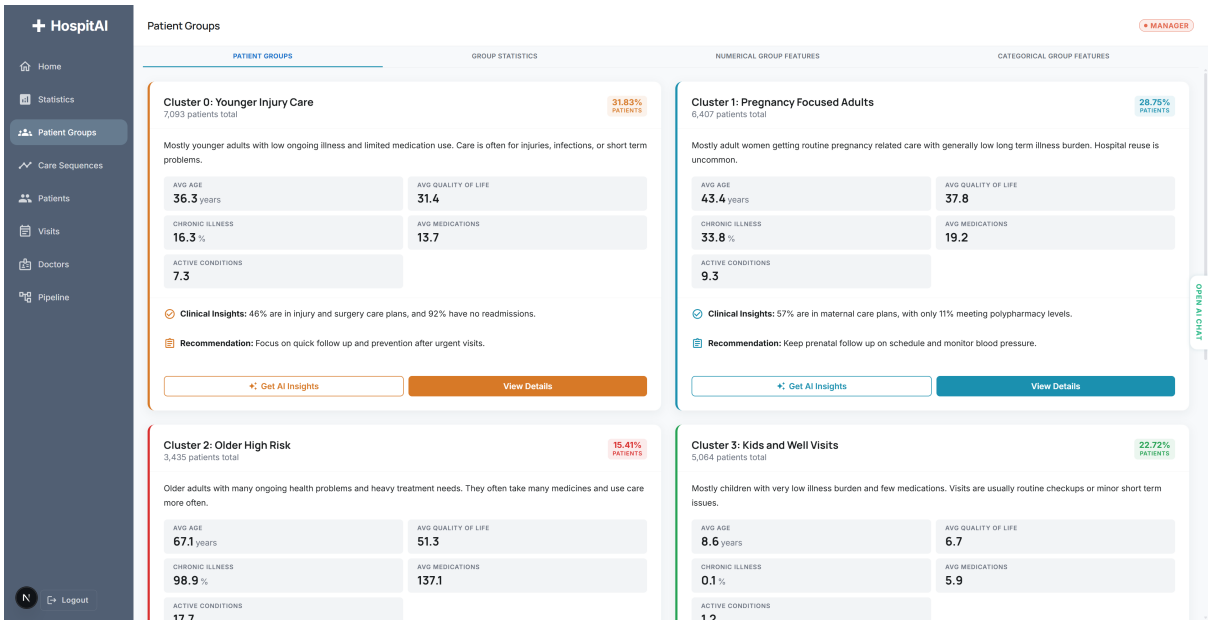


Figure 7.9: Cluster overview

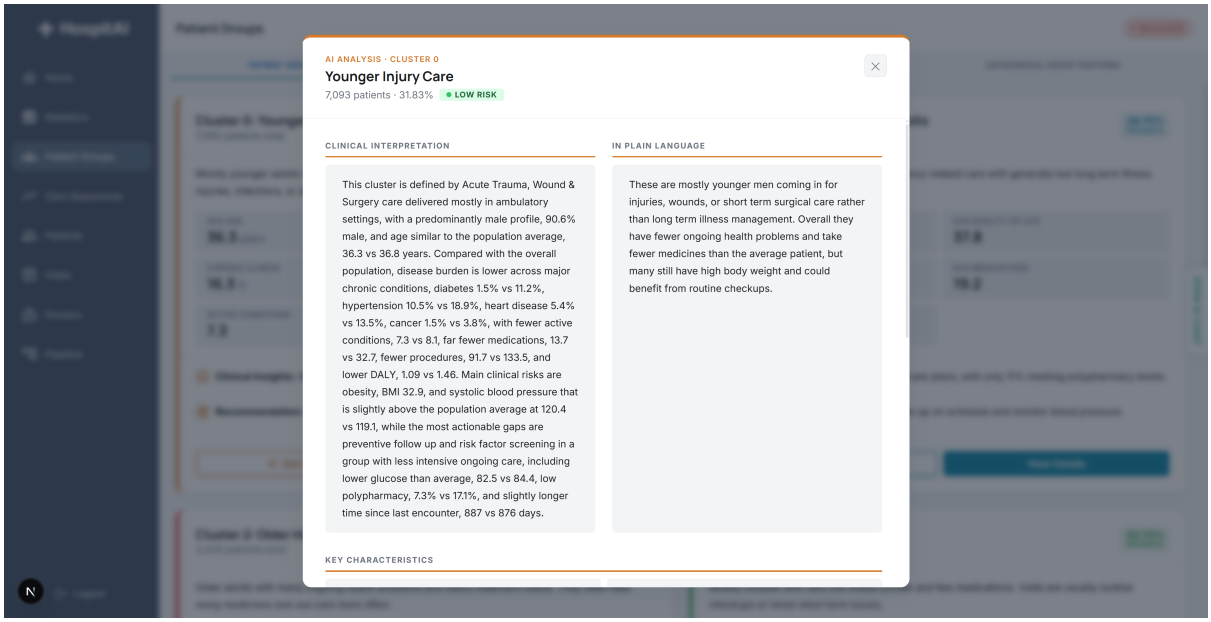


Figure 7.10: AI interpretation of a specific cluster (1)

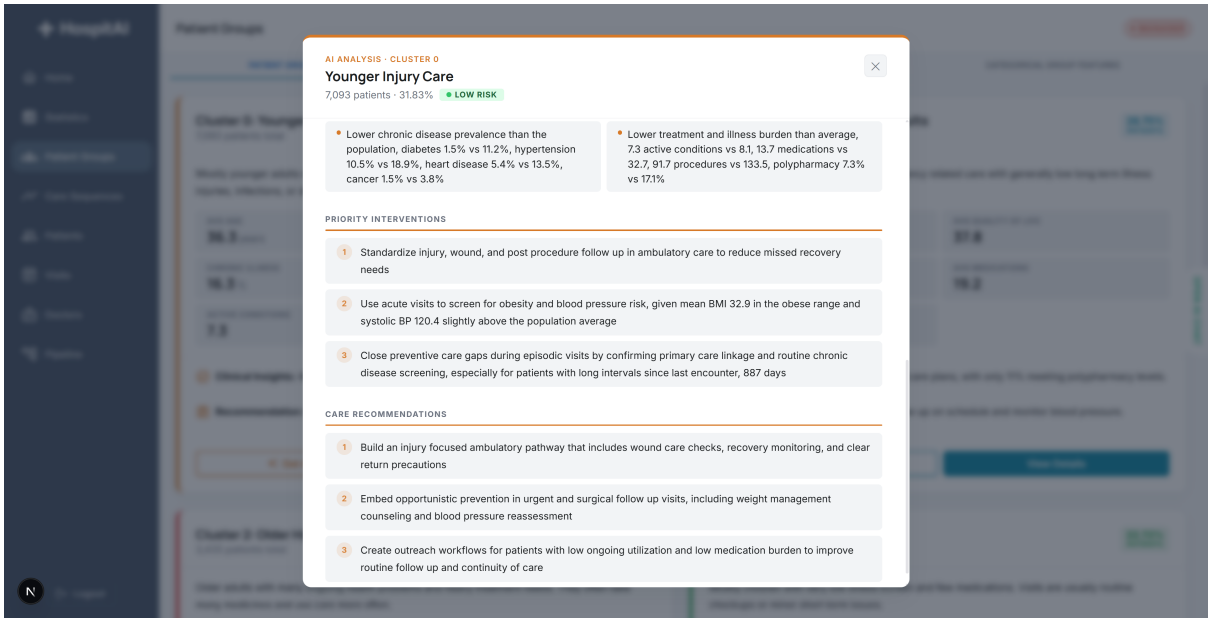


Figure 7.11: AI interpretation of a specific cluster (2)

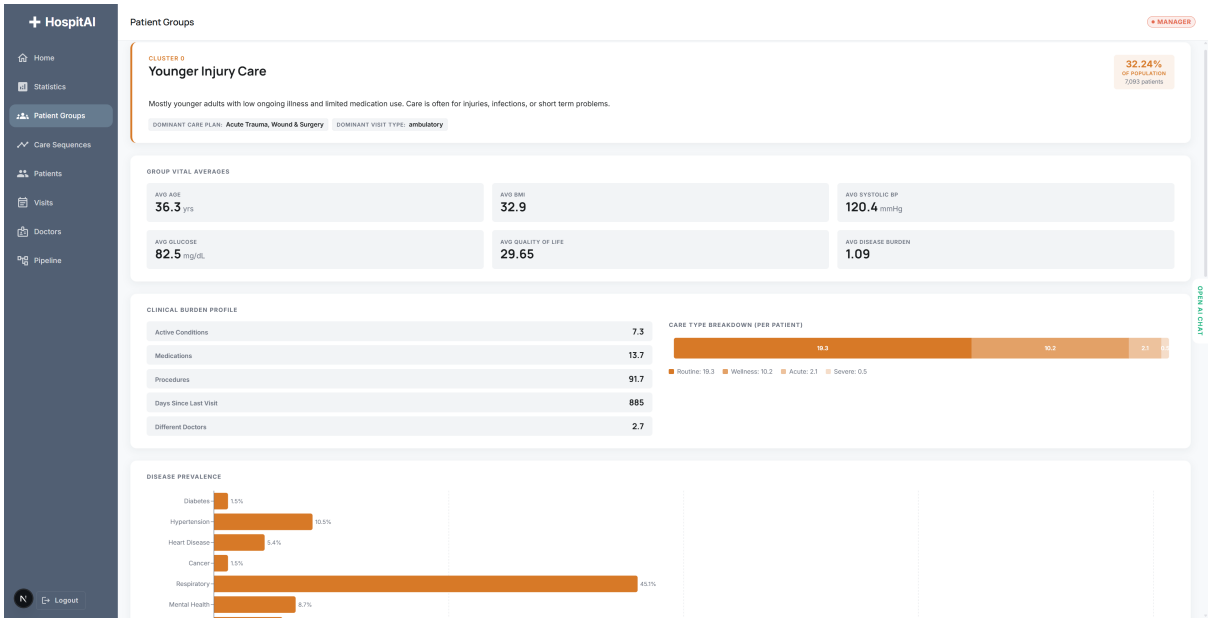


Figure 7.12: Detail of a specific cluster (1)

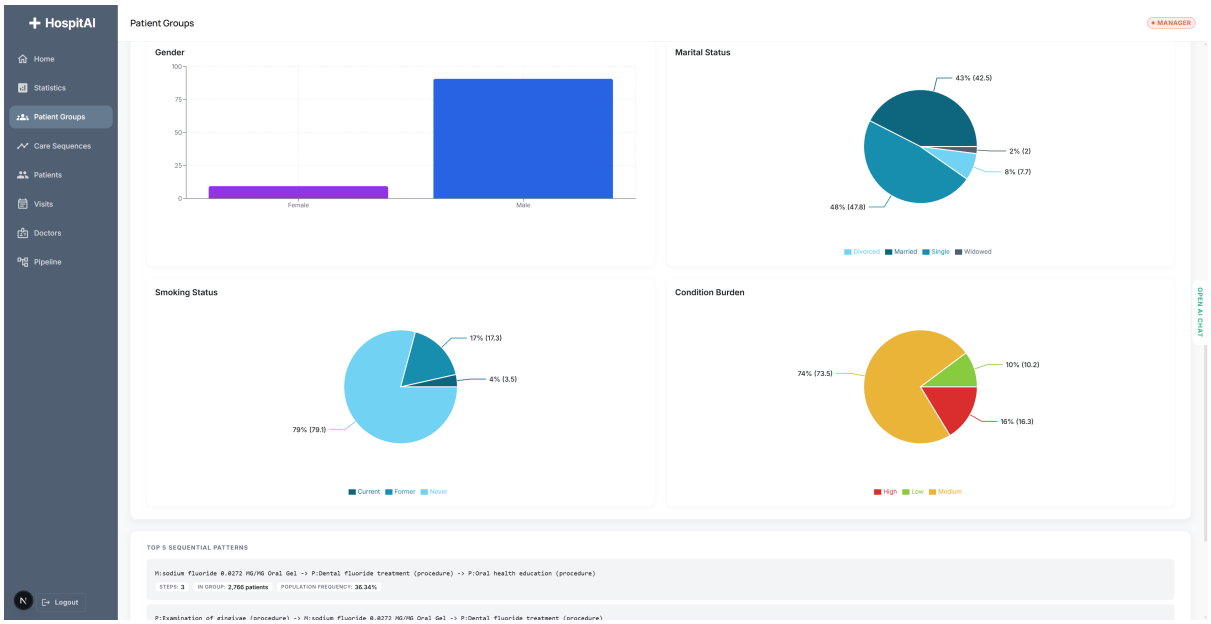


Figure 7.13: Detail of a specific cluster (2)

Both roles can likewise search the mined sequential patterns, either globally or per cluster (Figure 7.14), and see the AI interpretation of a specific sequence (Figures 7.15 and 7.16).

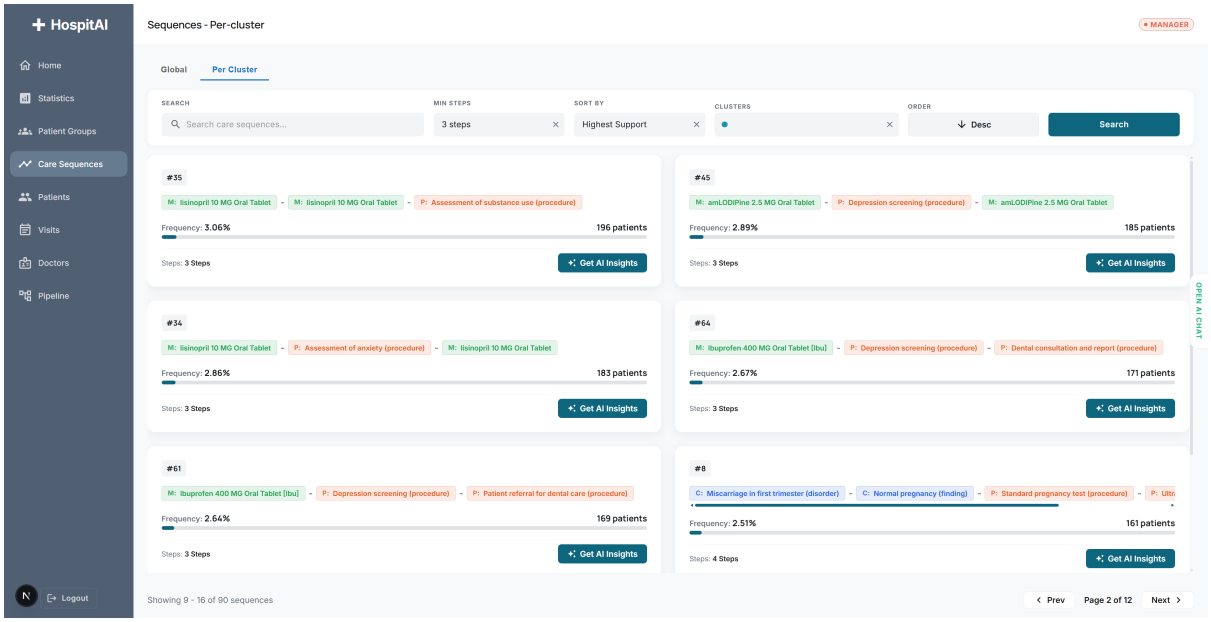


Figure 7.14: Sequential patterns, searchable globally or per cluster

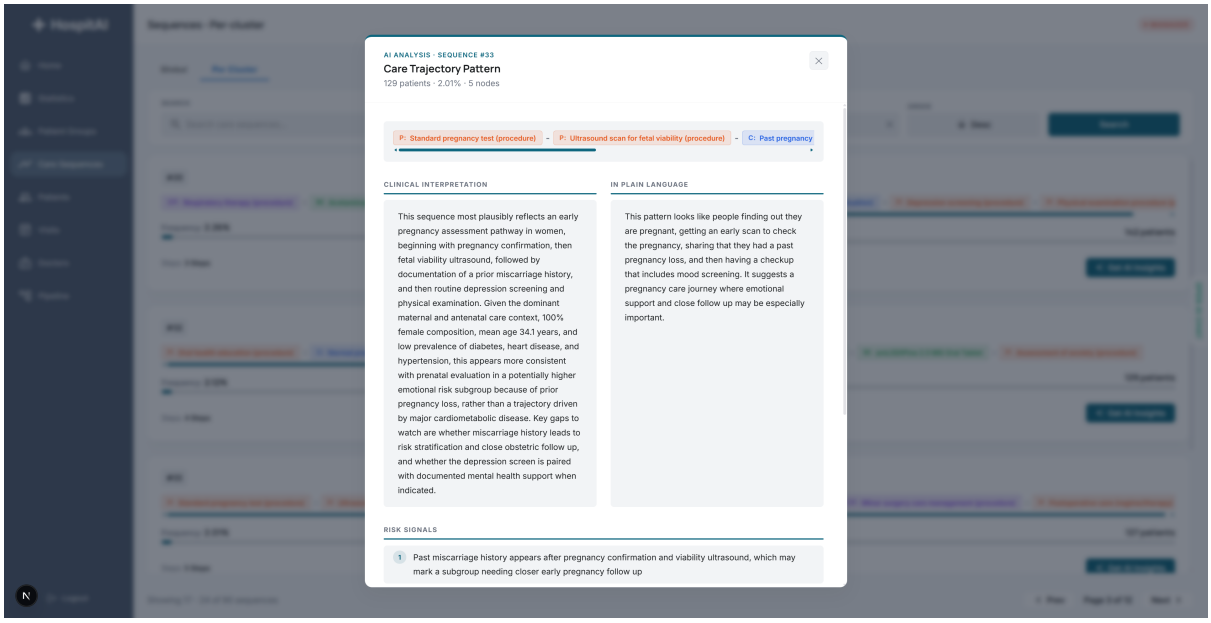


Figure 7.15: AI interpretation of a specific sequence (1)

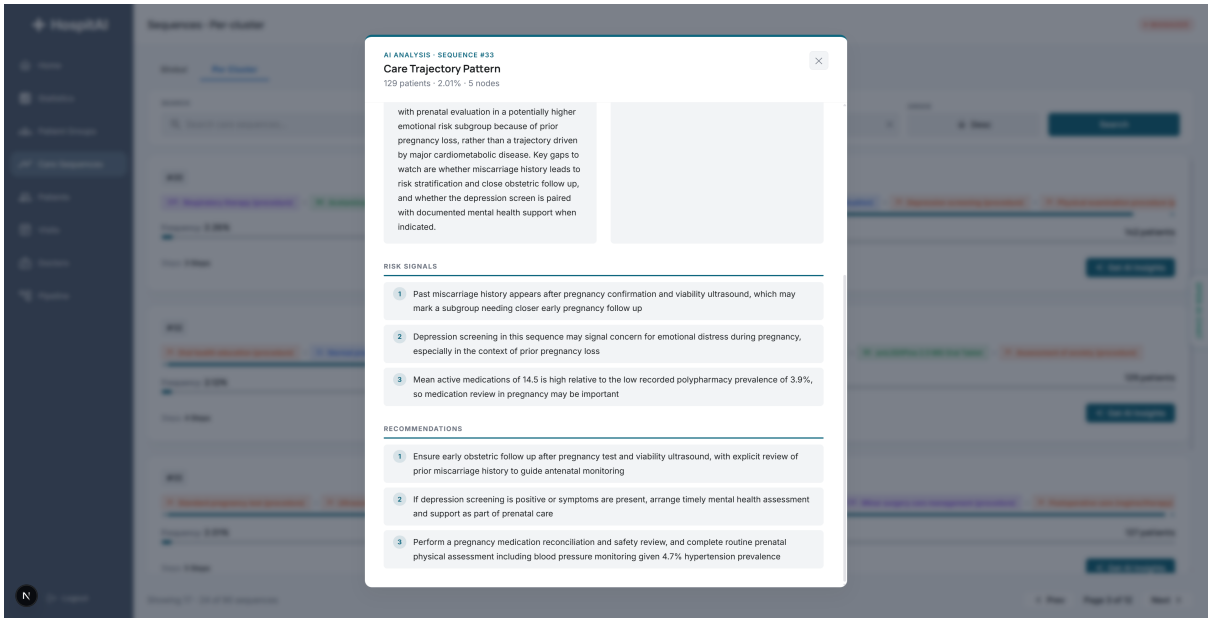


Figure 7.16: AI interpretation of a specific sequence (2)

Beyond the analytical views, the Manager can read (but not edit) the patients and their visits, and has full create, read, update, and delete access over the providers.

7.2.5. Natural-Language Query Agent

The Ask AI page lets the user query the database in everyday language instead of SQL. Figure 7.17 shows the agent answering a question grounded in the clinical data, as well as its response to a question that falls outside what the data can answer.

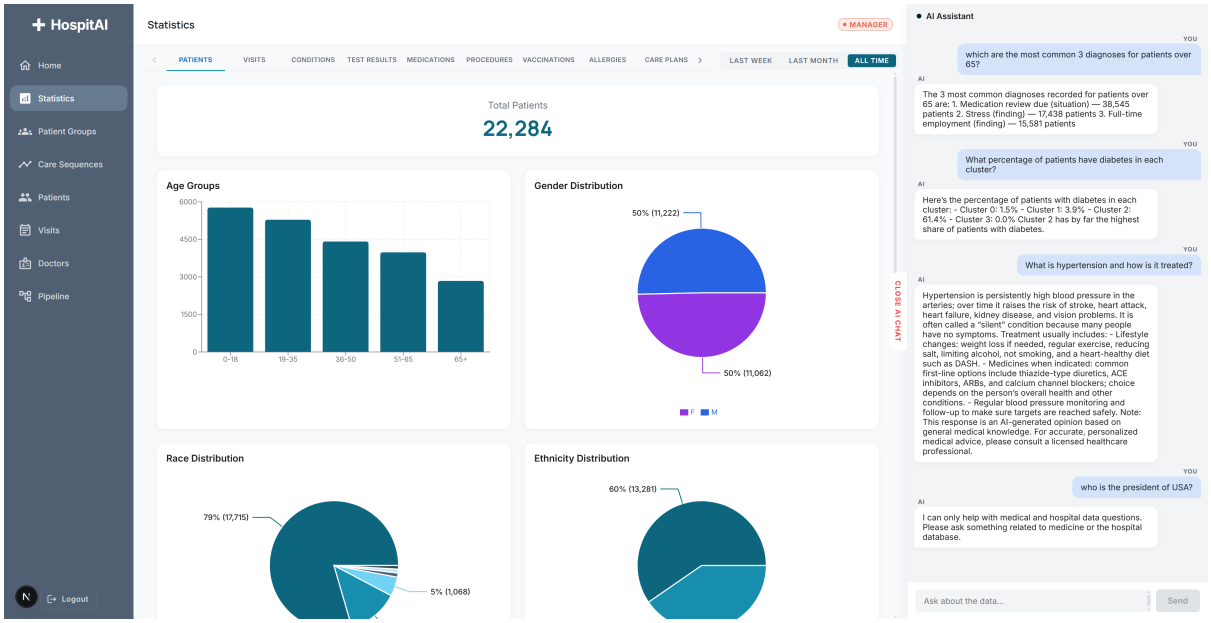


Figure 7.17: LLM agent answering a data-grounded question and an out-of-scope question

Chapter 8. Conclusions

Electronic Health Records hold analytical value that is hard to reach because the data is heterogeneous and longitudinal. This thesis addressed that problem with HospitAI, a web platform that ties patient segmentation, care-pathway mining, and large-language-model interpretation into a single pipeline. This chapter summarizes what was built, evaluates the results critically, and outlines future work.

Summary of Contributions

The main contribution is an end-to-end analytical pipeline over mixed-type clinical data, wrapped in a platform that someone without a data-science background can operate.

An ETL process takes a regional synthetic Synthea dataset, reduces it to a single hospital, and normalizes it into eleven relational tables in third normal form, computing the derived attributes the later analysis depends on along the way. Segmentation is handled by K-Prototypes, chosen because it works on numerical and categorical features at once. Around it we added feature engineering, outlier isolation, severity compression through PCA, and FAMD-based visualization, and selected the final model by benchmarking 63 feature configurations against six internal validation metrics.

Care-pathway discovery uses PrefixSpan, extended with seven clinically motivated customziations that turn its raw, noise-dominated output into a small set of distinctive closed pathways, mined both globally and within each cluster. A two-tier prompting scheme turns these numeric results into narratives a clinician can read, and a LangGraph-based SQL agent answers plain-language questions with safe, read-only, role-scoped queries. An in-process orchestration job chains all of this into one monitored flow, triggered by a manager or on a monthly schedule, and exposed through a role-aware Next.js frontend.

Critical Analysis of the Results

The three analytical layers do not carry the same evidentiary weight. Clustering quality was validated the most rigorously: K-Prototypes was scored against six internal validation metrics established in the clustering literature, so the choice of final configuration rests on a recognized, reproducible basis. Sequence mining was evaluated more loosely. PrefixSpan itself is a settled algorithm, but the six customizations on top of it were tuned by reranking and empirically checking that the surfaced pathways stayed relevant to the cluster they came from, not against a formal benchmark. The AI-generated interpretations sit on the weakest ground of the three: their soundness was likewise checked empirically, by inspecting whether the narratives matched the numbers they were given, but no doctor or other qualified clinician reviewed them for medical validity.

The data itself is a further limitation. The platform runs on synthetic Synthea records, which copy the shape of real EHRs but not the underlying epidemiological signal,

so the clusters and pathways found illustrate the method rather than establish clinical fact. The role separation is a demonstration of per-role behavior, not a real security boundary, and clustering sees only a static, frozen snapshot of each patient. The language-model interpretations remain a decision-support draft that a clinician would still have to check, not a diagnosis. Two further limitations follow from the design. The whole interpretation layer depends on a single external language model, which raises questions of cost, reproducibility, and governance that would need resolving before any hospital deployment. And the data-cleaning filters are tuned to this Synthea extract, so part of the preprocessing would have to be generalized before the pipeline transfers to another dataset.

Future Work

The step that matters most is validating the whole pipeline, clusters, sequences, and AI narratives alike, on real, de-identified clinical data together with domain experts, since that is the only way to close the evaluation gaps described above. That move would also justify replacing the demonstration roles with genuine authentication, role-based access control, and auditing.

On the analytical side, moving from a static snapshot to temporal or dynamic clustering, and adding time gaps and concurrency to the sequence model, would let the system show how a patient changes over time rather than only who they resemble at one point.The resulting clusters could also seed a supervised outcome-prediction model. From an engineering angle, an incremental pipeline and some distributed computation would let the platform grow past a single hospital, and a side-by-side comparison of several language models would help pin down how far the automated interpretations can be trusted.

Bibliography

[1] P. Yadav, M. Steinbach, V. Kumar, and G. Simon, "Mining electronic health records (EHRs): A survey," *ACM Computing Surveys*, vol. 50, no. 6, pp. 1–40, 2018.

[2] J. Walonoski, M. Kramer, J. Nichols, A. Quina, C. Moesel, D. Hall, C. Duffett, K. Dube, T. Gallagher, and S. McLachlan, "Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record," *Journal of the American Medical Informatics Association*, vol. 25, no. 3, pp. 230–238, 2018.

[3] Z. Huang, "Extensions to the k-means algorithm for clustering large data sets with categorical values," *Data Mining and Knowledge Discovery*, vol. 2, no. 3, pp. 283–304, 1998.

[4] J. Pei, J. Han, B. Mortazavi-Asl, H. Pinto, Q. Chen, U. Dayal, and M. Hsu, "PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth," in *Proceedings of the 17th International Conference on Data Engineering (ICDE '01)*, 2001, pp. 215–224.

[5] A. Aljohani, "Optimizing patient stratification in healthcare: A comparative analysis of clustering algorithms for EHR data," *International Journal of Computational Intelligence Systems*, vol. 17, no. 1, p. 173, 2024.

[6] W.-C. Yang, J.-P. Lai, Y.-H. Liu, Y.-L. Lin, H.-P. Hou, and P.-F. Pai, "Using medical data and clustering techniques for a smart healthcare system," *Electronics*, vol. 13, no. 1, p. 140, 2024.

[7] R. Novidianto, H. Wibowo, and D. R. Chandranegara, "ClusterMix k-prototypes algorithm to capture variable characteristics of patient mortality with heart failure," *Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control*, vol. 6, no. 2, pp. 109–116, 2021.

[8] A. Najjar, C. Gagné, and D. Reinharz, "A novel mixed values k-prototypes algorithm with application to health care databases mining," in *Proceedings of the 2014 IEEE Symposium on Computational Intelligence in Healthcare and e-health (CICARE)*, 2014, pp. 159–166.

[9] A. José-García, J. Jacques, A. Filiot, J. Handl, D. Launay, V. Sobanski, and C. Dhaenens, "Multi-view clustering of heterogeneous health data: Application to systemic sclerosis," in *Parallel Problem Solving from Nature – PPSN XVII*, ser. Lecture Notes in Computer Science, vol. 13399. Springer, 2022, pp. 352–367.

[10] A. Perer, F. Wang, and J. Hu, "Mining and exploring care pathways from electronic medical records with visual analytics," *Journal of Biomedical Informatics*, vol. 56, pp. 369–378, 2015.

[11] H. Edman, "Sequential pattern mining on electronic medical records for finding optimal clinical pathways," Master's thesis, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2018, dA222X.

[12] D. F. A. Botas, "Mining association rules and sequential patterns from electronic prescription databases," Master's thesis, Instituto Superior Técnico, Universidade de Lisboa, 2019.

[13] C. Baudrier, Y. Tran, N. Delanoy, S. Katsahian, B. Sabatier, and G. Perrin, "Identifying homogeneous healthcare use profiles and treatment sequences by combining sequence pattern mining with care trajectory clustering in kidney cancer patients on oral anticancer drugs: A case study," *Health Informatics Journal*, vol. 28, no. 2, pp. 1–18, 2022.

[14] A. Najjar, D. Reinharz, C. Girouard, and C. Gagné, "A two-step approach for mining patient treatment pathways in administrative healthcare databases," *Artificial Intelligence in Medicine*, vol. 87, pp. 34–48, 2018.

[15] J. Liu, F. Liu, C. Wang, and S. Liu, "Prompt engineering in clinical practice: Tutorial for clinicians," *Journal of Medical Internet Research*, vol. 27, p. e72644, 2025.

[16] M. Al-Garadi, T. Mungle, A. Ahmed, A. Sarker, Z. Miao, and M. E. Matheny, "Large language models in healthcare," *arXiv preprint*, 2025, arXiv:2503.04748.

[17] J. C. Gower, "A general coefficient of similarity and some of its properties," *Biometrics*, vol. 27, no. 4, pp. 857–871, 1971.

[18] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1. University of California Press, 1967, pp. 281–297.

[19] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons, 1990.

[20] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96)*. AAAI Press, 1996, pp. 226–231.

[21] F. Cao, J. Liang, and L. Bai, "A new initialization method for categorical data clustering," *Expert Systems with Applications*, vol. 36, no. 7, pp. 10 223–10 228, 2009.

[22] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[23] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.

[24] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[25] H. Cramér, *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[26] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.

Appendix A. Relevant Code sections

Sesiunea de Comunicări Științifice ale Studenților CSSC 2026

Diploma de Participare

Ioan Iulian Todor

" HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI-Generated Interpretation"

Universitatea Tehnică Cluj-Napoca

26 iunie 2026

HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI-Generated Interpretation

Ioan-Iulian Todor
Faculty of Automation and Computer Science
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
todor.io.ioan@student.utcluj.ro

***Abstract*—Many electronic health records (EHR) are generated in the healthcare environment. Unfortunately, a vast majority of this data remains underutilized, as clustering patient data to analyze longitudinal care histories is highly complex.**

In this paper, we present HospitAI, a platform structured around a three-stage pipeline: ETL for data ingestion, the K-Prototypes algorithm for mixed-type patient clustering, and a PrefixSpan sequence-mining framework tailored for the medical context. Additionally, an AI agent allows users to query the database using natural language, without requiring any technical knowledge. The AI is also used for interpreting our results (clusters and sequences), making them easily understandable for both medical professionals and patients.

Analyzing each patient cluster separately and identifying anomalies allows for the detection of unique care pathways, such as oncology, dialysis, and heart failure treatments. Testing the system on a synthetic dataset of 22,000 patients yielded four stable clusters with distinct sequence patterns, successfully representing actual patient categories.

Consequently, HospitAI converts complex numerical data into accessible textual information, using LLMs to provide actionable, easily interpretable insights into patient categories and care patterns.

I. Introduction

Nowadays hospitals contain large amounts of data that are stored in electronic health records (EHRs). With each patient encounter, data about diagnoses, prescribed medications, care plans and other procedures are recorded. On the theoretical level, it would be possible to use such data to find out meaningful patterns about patients and care. In reality, most information remains unused since the tables are too large, untidy, and complicated for an untrained eye.

There are three types of difficulties associated with this kind of data analysis. The first one is caused by the diversity of patients. One patient may be young with a single condition, while another may be older with multiple comorbidities. As a result, calculating a simple mean for the whole set of data hides the differences between the individuals included in it. Another challenge related to patient care is its temporal character. Clinical care is always represented by a series of actions performed during several weeks or months, and their order matters. The last difficulty is interpretation of the results. Even when a model produces a good result, the output is usually a list of numbers or codes that a clinician or a patient cannot interpret quickly.

In this paper, we introduce HospitAI, a web application designed to solve all three of these problems at once. We combined the data transformation, clustering process, sequence mining and the language processing in such a way that the same data of a patient can be viewed through different lenses with the help of one tool.

To solve the first problem, we apply K-Prototypes [**?**], an algorithm designed for mixed datasets. The data has both numeric variables (e.g., age of the patient and the number of drugs) and categorical variables (e.g., gender and major care plan), each having a different weight during the clustering process. To solve the second problem, we create a pipeline on top of PrefixSpan algorithm [**?**] and modify it to make sure it provides useful patient care sequences rather than repetitive ones. To solve the third problem, we integrate LLMs that summarize statistics in human-understandable terms and also help clinicians to query the database faster.

The contributions of this paper are as follows:

- Creation of an end-to-end solution that uses mixed data clustering, clinical sequence mining, and generation of AI-assisted explanations, with a clear differentiation between backend analysis and frontend implementation of the web-application.
- Creation of a feature-engineering routine that transforms raw data from multiple EHR tables into numeric and categorical features while taking into account the problem of skewed values and missing vitals.
- Adaptation of the PrefixSpan algorithm for domain-specific sequence mining with ranking of patterns according to geometric mean lift score and diversity rules. This sharply reduces trivial output and brings clinically relevant pathways to the front.
- Use of an LLM as an assisted "verbalizer" based on statistical information and a Natural-Language-to-SQL agent to translate natural language queries to SQL.

The rest of the paper is organised as follows. Section II reviews related work on care-pathway mining and patient clus-

tering. Section III provides the system overview. Section IV explains the three analytics components. Section V reports the main results, and Section VI discusses limitations and future work.

II. Related Work

Surveys of patient pathway mining. There exist two recent surveys that provide a good overview of this area. Rosa et al. [1] provide a survey of the models and techniques used for patient pathways mining. They distinguish between the sequence mining and process mining approaches. Flothow et al. [2] provide a survey of various techniques applied in the search of patient sequences of healthcare use, distinguishing clustering approaches, which uncover entire patient trajectories, from pattern mining approaches, which are applied to subsequences.

Combining clustering with sequence mining. The most relevant research hybridizes the two concepts. Najjar et al. [**?**] suggest a two-step process, whereby they cluster single healthcare visits using K-Prototypes and derive treatment pathways afterwards. It is an important predecessor of our research because it also uses the mixed clustering technique that we employ. Baudrier et al. [3] hybridize their care trajectory clustering on patients with kidney cancer with a sequential pattern discovery algorithm and conclude that their clusters differ from each other by actual results like mortality rate. Vogt et al. [**?**] discover care pathway clusters for heart-failure patients from sequences in insurance claims with ambulatory healthcare pathways. Our approach is conceptually similar, yet it differs from theirs in the order of steps involved.

Sequence mining from EMR with interpretation. Perer et al. [**?**] conduct sequence mining from electronic medical records based on frequent event sequences to patient outcomes, with the major emphasis on visual analysis. Their results show that frequent sequences can detect clinically meaningful care patterns unknown to the physician, which motivates our own attention to pattern quality, since frequency alone is not enough.

Clustering of patients on EHR data. Some studies concentrate on analyzing methods of proper patients' clustering. Yang et al. [4] develop a smart-healthcare solution aimed at clustering patients based on different kinds of information contained in their medical records, to reveal potential health risks and disease patterns across patient groups. Aljohani [5] evaluates eight different clustering methods for patients' stratification based on EHR and analyzes them using a multi-criteria approach.

Process mining as an alternative. An alternative set of methods analyzes care processes from event logs. Specifically, Murazzano et al. [6] use process mining to analyze chronic respiratory patients and observe a large variability in their care processes, pointing to another direction of analysis that addresses the same objective in a different way.

Clustering and sequence mining are both mature domains in healthcare analytics. There have been some research works which try to link them together. However, there remains the need for a unified framework that integrates cluster-scope mining with a rank of clinically meaningful patterns followed by an AI-based explanation component. That's where HospitAI comes in.

III. System Overview

A. Architecture

The HospitAI architecture consists of two components, one that runs online via a web application, and another one which is offline analytics. In the online component, we use a frontend built with Next.js and a backend with FastAPI. The backend implements the layered approach without any bypasses between layers. The layers consist of routers which use API services, service layer which communicates with a repository layer, and repository layer which directly connects to the database. The entire persistence storage consists of a single PostgreSQL database. Figure 1 illustrates the overall architecture.

The logic for analytics is separated from the online requests and represents a three-step offline pipeline implemented in a set of Python scripts: ETL cleaning of the initial export data, a clustering step and sequence mining. Each step reads its input from the database, performs its computations and stores the results back into special tables, which the web app then only reads to provide a snappy experience. Above all this sits a single large language model running on top of the OpenAI API, enabling each of the text-generation tasks, including cluster/sequence interpretation and question-answering.

B. Data

For training and experimenting, we will use a simulated database created using the open source patient simulator Synthea. Simulated data fits well here because it avoids privacy and legal concerns while keeping a realistic database schema. Our database contains approximately 22,284 patients and standard clinical data tables such as encounters, conditions, medications, procedures, observations and care plans. The ETL process described in Section IV filters our database down to a single hospital and fixed set of providers as well as fixing the dates and calculating some auxiliary information such as patient age and treatment duration.

Both the clinical database and the analysis results are stored in the same database. The clustering algorithm creates a single row for each patient with its clustering result and a list of selected features cached in the row, while sequence mining stores patterns in a separate table along with a mapping of patterns to the patients matching them.

IV. Methods

The analytics component of HospitAI consists of an offline pipeline with three stages plus the AI component. Stage one consumes the exports and generates a clean relational database (ETL process). Stage two computes the patient features and clusters patients via the K-Prototypes algorithm. Stage three computes the clinical sequences in each patient group. Above all this, the AI component will take in the preprocessed

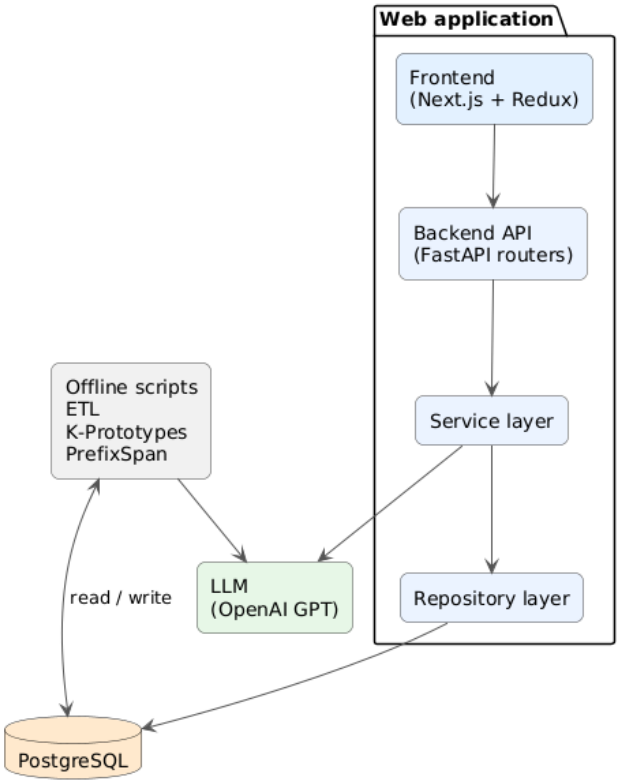


Fig. 1. High-level architecture of the application

information and generate readable text output, either as automatically generated cluster/sequence descriptions or as a question-answer session for humans. We explain the details below.

A. Stage 1: Data Ingestion and ETL

The data flow in HospitAI begins with the comma-separated exports generated by Synthea and ends with a clean PostgreSQL database.

Three key decisions related to reduction and data cleansing influence the results of our analysis. First, we fix one hospital. In particular, the organization which was selected has the highest utilization. Only busy providers are kept. Three hundred providers are ranked by decreasing number of encounters. Then all encounters of the excluded providers are redistributed between other providers with a fixed random seed so that the number of encounters and the provider load remain unchanged. Second, we fix the dates. Some events created by Synthea may have the timestamp set to some date in the future. Therefore, we restrict the timestamp to the current date. We obtain the patient's age by subtracting the birth date from the death date (or from the current date for patients who are still alive). The length of an encounter is estimated using the beginning and end of the encounter time and limited by some reasonable value depending on the encounter's class (for instance, up to two hours for the wellness visit and thirty days for the inpatient encounter). Third, missing values are not mean-imputed, since that would distort the numerical centroids K-Prototypes treats as real measurements. Sparse categorical and vital-sign features instead fall into an explicit "Unknown" level, and count features are filled with zero, which correctly represents an absence of events.

The number of records in the observation table is in millions, so the ingestion is done in parts. This step produces normalized data that includes patients, encounters, conditions, medications, procedures, observations, care plans and providers.

TABLE I
ENGINEERED PATIENT FEATURES, GROUPED BY KIND. THE FULL CATALOGUE IS COMPUTED OFFLINE. CLUSTERING USES A NON-REDUNDANT SUBSET.

Group	Features
Numerical	age, active conditions, total medications, total procedures, distinct providers seen, lifetime medication days, encounter duration hours, preventive-to-reactive ratio
Demographic	gender, marital status, race, ethnicity
Disease flags	diabetes, hypertension, heart disease, cancer, kidney disease, mental health, respiratory profile
Binned vitals	BMI class, blood-pressure risk, glucose risk (each with an explicit "Unknown" level)
Care-pattern tiers	dominant care plan (severity-ranked), condition burden, medication intensity, polypharmacy, 30-day readmission tier, out-of-hours flyer
Risk profiles	substance-use profile, pain-management tier, cholesterol risk, pregnancy history, smoking status

B. Stage 2: Feature Engineering and Clustering

For clustering, we need a table in which every row represents a patient and every column represents a feature. Most effort here goes into creating such a table, since the clinically relevant information is scattered across the database tables and not directly available for modeling. For each feature, a separate SQL aggregation query is run.

We start off with a large collection of potential features and end up selecting a few non-overlapping ones later. These features are grouped according to type in Table I. The numeric features are relatively simple (counts, lengths and ratios). On the other hand, there is more domain knowledge embedded within the categorical features because we do not rely on raw codes, which are derived into clinically meaningful tiers through rule-based SQL. Disease features such as diabetes, hypertension, heart disease, cancer and kidney disease are defined based on keyword matching against condition descriptions, specifically excluding "prediabetes", "suspected" or "family history" when the feature would imply otherwise. Care behavior features include a tier of thirty-day readmissions generated using window functions on consecutive visits, a flag for those who make visits to the ED in out-of-office hours, their drug use pattern which integrates their diagnostic information with the results of the AUDIT-C and DAST-10 tests, and a tier for different pain management approaches.

The selected engineering methods include the following. Some features are heavily skewed, so we apply a natural log

transform and then min-max scaling. For the physiological vital sign features such as BP, Glucose, we assume that they may not be available, hence categorizing them along with unknowns instead of replacing the missing values with mean values as part of retaining the knowledge that the particular feature was never recorded. The most critical care plan is not the most common care plan but the care plan that is most severe. We arrange the care plans per patients according to specific severity rankings (oncology and palliative, maternal, trauma, respiratory and cardio-metabolic), then choose the most severe care plan. So a cancer patient is assigned the cancer care plan even though their most common care plan can be a checkup. We do not transform categorical features through one hot encoding because K-Prototypes can work with them directly. There is also the clinical severity score (first principal component of log-transformed counts of procedures/medications/conditions).

K-Prototypes algorithm [**?**] is an extension of K-Means algorithm for mixed data. Distance is defined by the sum of squares of the difference of numerical features and the number of mismatches in categorical features. The weighted sum using weight γ helps to control the relative significance of the categorical features. We use the Huang initialisation and select the value of γ automatically using the library.

Prior to the model fitting, we look for any statistical outliers based on a z-score criterion for the numerical variables after log-transforming the heavily skewed counts, such that the tail distribution itself does not lead to the detection of an outlier. If a z-score greater than six is detected for any of the variables, the corresponding patient is considered an outlier and there are 289 such patients that constitute around 1% of the dataset. None of the outliers are removed from the analysis. Instead, they receive a unique label of -1 and are included in the analysis with the other patients without being used in the K-Prototypes cluster fitting, i.e., none of the clusters are generated for them. This is done because their values would skew the cluster generation, but they are clinically important since they represent the most severe cases like cancer, dialysis, and heart disease.

C. Stage 3: Clinical Sequence Mining

In the third phase, we study the order of clinical events. One ordering sequence is defined for each patient, i.e., the ordering sequence of all the events occurring in a certain timeline, arranged according to the dates. Each event is prefixed with a type character such that the source table for the particular event is known (Condition is labeled C, Medication M, Procedure P, and Care plan CP). If more than one event occurs on a particular day, then the ordering is based on a clinical order (C-M-P-CP).

Cleaning comes before mining, where we remove all events except for those occurring during the last two years per patient, exclude some purely administrative events and psychosocial "status" events added to almost any encounter by Synthea, merge immediate duplicates of events, and restrict the length of the event timeline, allowing us to perform mining very

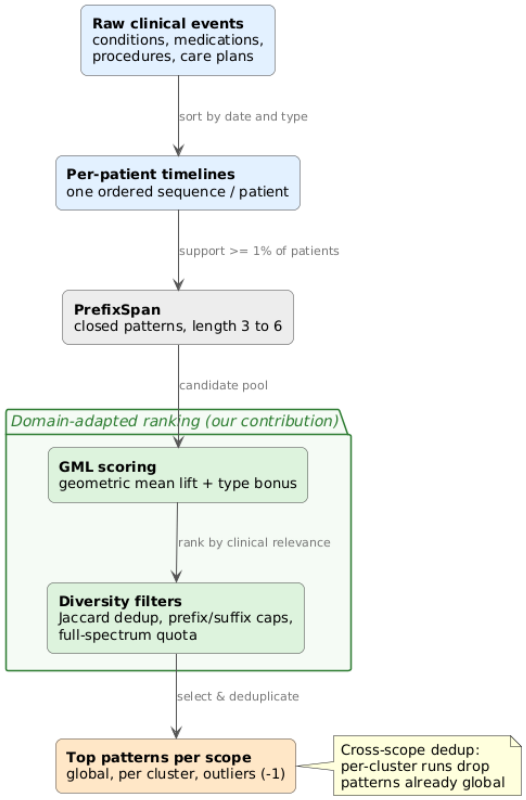


Fig. 2. The sequence-mining pipeline

efficiently. Mining of sequences is done using the PrefixSpan algorithm [**?**], a classical approach that builds on finding closed frequent sequences by projecting the data onto all the already discovered prefixes. We ask PrefixSpan to generate closed sequences of length ranging from three to six events having a support of at least one percent of patients, and we mine more than needed for selection purposes.

The key step happens around PrefixSpan in Figure 2. When ranking the patterns by their supports only, the best patterns are going to be very frequent but not useful sequences, such as regular dentistry cleaning actions which everyone undergoes. To solve this, we rank the patterns using Geometric Mean Lift (GML). Given a pattern p containing k elements with its support being $s(p)$ among n patients and item supports being $s(i)$, we compute

$$\mathrm{GML}(p) \;=\; \frac{s(p)/n}{\left(\prod_{i\in p} s(i)/n\right)^{1/k}}. \qquad (1)$$

It is just another step in applying the notion of lift from two items to k items based on the geometric mean calculation. It can be calculated in log scale to prevent overflow. The higher GML value is, the more probable it is that the pattern would appear in practice more often than its component frequencies' product.

Two other concepts have been proposed. Firstly, an additional bonus is assigned to the overall score as multiplication

by a multiplier of $(1+0.3\,(t-1))$ where t stands for the total number of different types of events in the pattern. In this way, we give preference to the actual trajectory of a patient which includes diseases, drugs, and surgeries to a technical pattern of three drug applications. Secondly, certain diversity criteria are used. We filter out patterns including triplets of the same type of events, patterns with very similar items according to the Jaccard distance, and limit the number of patterns including the same beginning or ending events or dominated by dentistry events. Also, some slots are reserved for fully representative patterns that include all four types of events.

Finally, all these processes are run for each scope once. Mining is performed both on the global scope and on the individual clusters, including the outlier group denoted -1. Patterns discovered in the global scope are ignored while mining the individual clusters so as to keep only those patterns that define the clusters. Further, each of the patterns retained has their patient ID attached to them using the intersecting posting list of the least frequent items.

D. AI Interpretation Layer

Finally, this layer makes the output understandable. We use just one LLM using the API of OpenAI. However, we do this wisely: the statistics are never computed by the machine alone. The programming code computes everything before providing these static values to the machine in its prompt, asking it to verbalize about no other data but the provided numbers. Such an approach is known as "verbalizer" since it guarantees that facts are described in the text.

The key point is that clinical features are given as auxiliary data within the prompt. Clusters' prompts are constructed out of actual figures: population averages, cluster-specific values with an annotation "compared to population average", prevalence rates of various conditions, averages for vital indicators, dominant care plan category, and prevalent diseases, therapies, and interventions. In turn, the model is supposed to generate strict JSON including the risk level, clinical reasoning for doctors, statement for patients, key characteristics of a certain cohort or sequence, and suggestions as to what actions should be taken. Sequences interpretation is done analogically by providing mined patterns accompanied by aggregated patient profile (age, gender, BMI, disease prevalence, dominant care plan), and further decoding the prefixes and temporal arrows into reasonable care pathway.

All AI outputs are cached as a JSON value together with their corresponding rows, so the model is run only once per row.

E. Natural-Language-to-SQL Agent

In addition to strict interpretations, HospitAI allows users to ask questions on top of the available data. A question-answer agent was developed, which is a state machine based on LangGraph, shown in Figure 3. Each state accomplishes one task alone.

In the initial phase the question is classified and rephrased. The agent determines whether the question requires the database, whether it is a generic medical question that can be answered directly, or whether it is irrelevant and must be rejected.

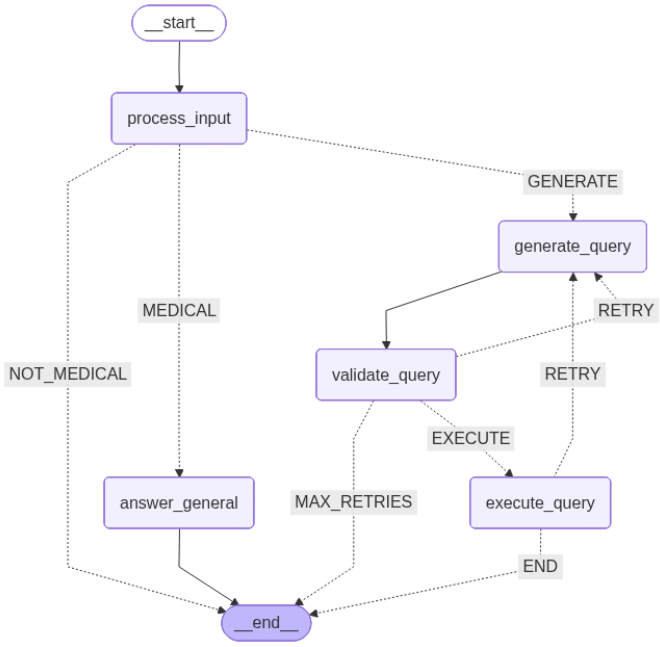


Fig. 3. The natural-language-to-SQL agent

In case the database needs to be accessed, the query generation procedure is performed. Generation is based on the embedding of the database schema and a set of rules. Generated queries are validated in two layers. The first, deterministic layer ensures the query is a single read-only statement and blocks potentially harmful keywords via regex validation. The second layer validates the query semantics, i.e. its ability to answer the question with the appropriate tables and filters. If the query fails either layer, the error message is passed back to the agent to regenerate the query until an attempt limit is reached. Database execution errors are treated the same way.

The agent is also role-aware. If the request comes from a doctor, an additional scope condition restricts every search to the records of that doctor's own patients via their provider id, and any request without this condition is rejected, preventing a doctor from seeing other doctors' patients.

V. Results

The effectiveness of the proposed approach is illustrated using the Synthea dataset, which is introduced in Section III. Out of 22,284 patients, only 21,995 patients have been clustered, and all outliers have been retained in a different class labeled as -1.

A. Clustering

To discover the optimal configuration, we performed a large benchmarking procedure on 628 configurations. Configuration

means feature subset selection (we tried more than sixty feature subsets), cluster number k, scaler used, categorical weight γ, random state and outlier threshold. For benchmarking, we calculated a score based on three metrics. First of all, the metrics are min-max normalized individually within the scope of all 628 configurations, and the score is then their linear combination with weights that reflect our confidence in them (0.5 for Gower silhouette, 0.3 for Calinski-Harabasz index and 0.2 for Davies-Bouldin index, inverted because the higher the score the worse the partition is). We chose the best clinically interesting configuration per k value (we tried four, five and six), and from these three configurations we tried once again to optimize the hyperparameters and feature subset used to choose the best local configuration.

Table II reports a representative configuration for each k. The choice of k is made to be 4, with feature combination v55. It scores best overall (0.82) among the clinically meaningful runs, and does so across all individual metrics: the largest Gower silhouette, the largest Calinski-Harabasz score, and the smallest Davies-Bouldin score. We still eliminate one family of runs, which scores even better on paper. It achieves a score of 0.87 by segregating patients according to gender, but since the separation is meaningless clinically, we exclude them. Among the four clusters in the optimal run, three get a Gower silhouette per cluster greater than 0.30, and only one gets close to 0.20, compared with the results of five and six clusters where one and two clusters respectively have values less than 0.20. Since clusters will be used for the scope of the sequence mining analysis, we prefer to have the smallest k possible while still having all the clusters be cohesive. The important features are the number of active conditions and patient age numerically, and condition burden, gender, diabetes, and heart disease categorically.

Figure 4 shows the per-cluster means of the numeric variables represented as a heatmap. There is a clear morbidity gradient. That is, cluster 0 represents young people with low morbidity, while the rest of the clusters contain a progressively greater number of conditions. By interpreting all four clusters in conjunction with the categorical features, we see four distinct patient profiles, which the AI platform automatically labels as: Younger Injury Care (young, low morbidity), Pregnancy Focused Adults (reproductive women), Older High Risk (hypertensive polypharmacy adults), and Kids and Well Visits (pediatric/acute patients). These titles, and the description, insight, and recommendation attached to each cluster, are produced by the LLM at each run and are therefore not deterministic.

B. Sequence Mining

The clearest sign of our pipeline's impact is the change in the mined patterns' properties as each new version is introduced. In Table III, we present two sample patterns of at least four events from every one of these versions. The initial versions (v0-v2) rank patterns solely based on support, and therefore, the beginning of the ranking consists of common yet meaningless sequences, which consist mainly of similar

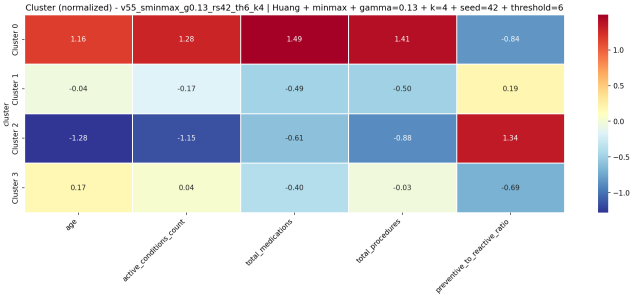


Fig. 4. Z-scored numerical feature means per cluster ($k = 4$). Warm colours are above the population average, cool colours below it.

dental cleanings that about 93% of the patients undergo. With v3 and onwards, geometric mean lift-based ranking along with diversity filters were applied, and at the top of the ranking, clinically consistent care pathways can be seen. Version v4 also filters out consecutive occurrence of similar events, which is why the long sequence of procedure events related to a diabetic eye exam no longer appears from v4 onwards. At v6, when social status events are removed and reserved space for full spectrum patterns was set aside, diverse combinations of conditions, care plans, medicines and procedures appear despite their low support scores.

Frequent pattern mining from each scope individually yields 175-252 frequent patterns per cluster. Examples of representative frequent patterns are listed in Table IV. Each cluster has a unique signal dominating its set of frequent patterns. For example, in Cluster 0, the dominant signal relates to acute care episodes. In Cluster 2, the chronic disease management includes antihypertensive drugs combined with screenings, or furosemide along with oxygen therapy to manage fluid overload. The most valuable benefit is obtained from the outlier scope. Since we mine each individual scope independently, high-cost combinations like chemotherapy and cisplatin leading to hospice care, or dialysis and epoetin, will dominate the results. Otherwise, these pathways fall under the global support threshold and would be missed when mining all cases at once.

C. AI Layer

Upon running the K-Prototypes script, the AI creates a short card per cluster based on the statistics generated beforehand and determines the cluster title, its colour theme that will be used in all mentions of the cluster on the platform, its description, an insight and a recommendation for each. Figure 5 shows what these cards look like in the dashboard. Since the model uses only the precomputed values, the card matches the heatmap and the prevalence rates by assigning high risk and a cardiology-related recommendation to the high-burden cluster, and a healthy, preventive profile otherwise.

Beyond the card information, the platform generates a more detailed report about every identified cluster or mined pattern. It serves two purposes: a clinical explanation for clinicians, and a simpler, layman's summary for someone without a

TABLE II

REPRESENTATIVE CLUSTERING CONFIGURATION FOR EACH CLUSTER NUMBER (FEATURE VECTOR BASED ON THE CORRESPONDING VERSION ID). CH INDEX AND DB INDEX ARE COMPUTED FROM THE FAMD EMBEDDING IN THE MIXED SCORE FRAMEWORK. THE HIGHEST VALUE OBTAINED FOR EACH CRITERION IS MARKED IN BOLD FONT. THE SELECTED CLUSTER NUMBER IS $k = 4$ (V55).

Config	k	Gower	Per-cluster Gower	DB	CH	Score
v55 (final)	4	**0.350**	0.42, 0.20, 0.32, 0.40	**0.72**	**26813**	**0.82**
v40	5	0.324	0.28, 0.22, 0.31, 0.19, 0.48	0.87	22547	0.74
v60	6	0.287	0.17, 0.13, 0.47, 0.21, 0.38, 0.27	0.95	22273	0.70

TABLE III

HOW THE MINED PATTERNS EVOLVE ACROSS MINER VERSIONS

Ver.	Example pattern (length ≥ 4)	Sup.
v0	dental consultation → dental care → supragingival scaling → subgingival scaling	93%
v0	supragingival scaling → subgingival scaling → gingivae exam → oral health education	93%
v1	supragingival scaling → subgingival scaling → gingivae exam → oral health education	63%
v1	dental care → supragingival scaling → subgingival scaling → oral health education	63%
v2	dental consultation → dental care → subgingival scaling → gingivae exam → oral health education	62%
v2	dental referral → dental consultation → dental care → subgingival scaling → oral health education	57%
v3	tropicamide eye drops → gonioscopy → fundoscopy → diabetic eye exam	3%
v3	furosemide → oxygen by mask → prone position → furosemide	2%
v4	acute bronchitis → respiratory therapy → respiratory function test → acetaminophen	3%
v4	salmeterol-fluticasone inhaler → albuterol → spirometry → depression screening	2%
v5	subgingival scaling → gingivae exam → sodium fluoride gel → fluoride treatment	35%
v5	dental bitewing X-ray → gingivae exam → sodium fluoride gel → fluoride treatment	19%
v6	acute bronchitis → respiratory therapy → respiratory function test → acetaminophen	3%
v6	chest X-ray → furosemide → oxygen by mask → prone position → furosemide	1%

TABLE IV

REPRESENTATIVE MINED PATHWAYS BY SCOPE (RANKED BY GML).

Scope	Pattern
Cluster 0	Fracture of bone → Fracture care → Bone immobilization
Cluster 1	Normal pregnancy → Pregnancy test → Ultrasound for fetal viability
Cluster 2	Furosemide → Oxygen by mask → Prone position → Furosemide
Cluster 3	Acute allergic reaction → Epinephrine injection → Prednisone
Complex High Need (−1)	Chemotherapy + radiation → Cisplatin → Hospice care

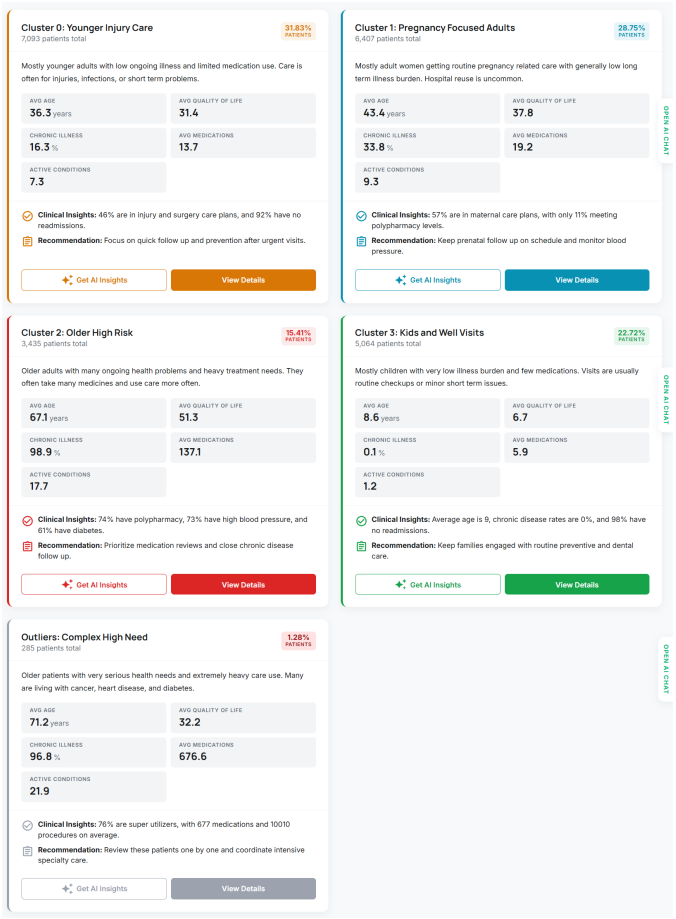


Fig. 5. The clusters as shown in the platform. The title, colour, description, insight and recommendation of each card are produced automatically by the AI when the clustering script runs.

medical background. This kind of explanation is depicted in Figure 6.

Finally, the natural-language-to-SQL agent lets a user ask free questions. We tested it on questions of different difficulty, including non-trivial ones that need several joins. Table V lists two such questions: the first joins several clinical tables, the second joins the cluster and the sequence-mining tables. The answers are correct almost every time, and the guard layer blocked every attempt to run a non-`SELECT` statement. When the agent cannot build a valid query or finds no matching data, it plainly says it could not complete the task instead of inventing an answer.

A full quantitative evaluation of the agent and of the generated summaries is left for future work, as discussed next.

VI. DISCUSSION AND CONCLUSION

We have introduced HospitAI, which brings together three analytical stages in hospitals based on data analysis: clustering

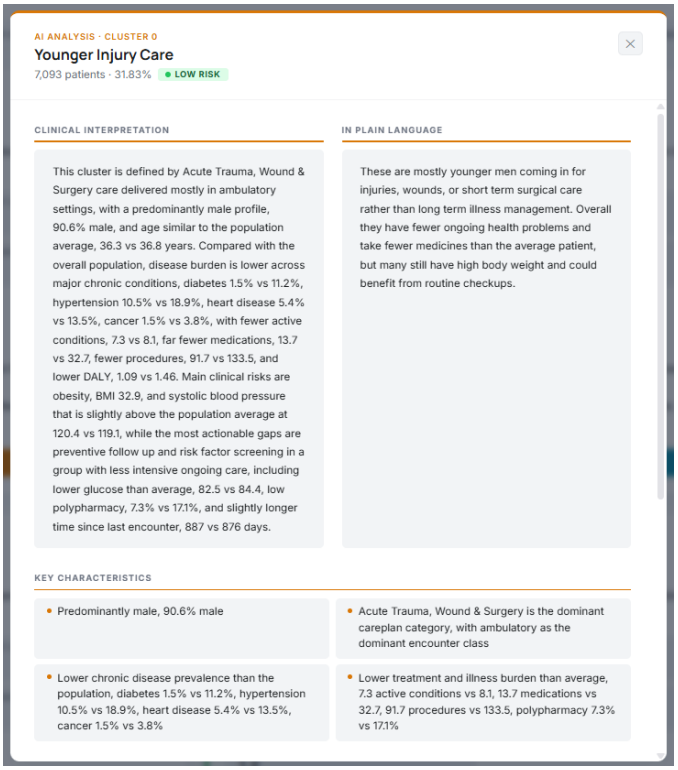


Fig. 6. A cluster AI interpretation in the front end, written for both a clinician and a reader without medical training.

TABLE V
TWO EXAMPLE EXCHANGES WITH THE NATURAL-LANGUAGE-TO-SQL AGENT. THE FIRST NEEDS SEVERAL JOINS ACROSS THE CLINICAL TABLES, WHILE THE SECOND JOINS THE CLUSTER AND THE SEQUENCE-MINING TABLES.

Question	Agent answer
How many distinct female patients over 60 with a diabetes diagnosis were also prescribed metformin and had at least one emergency encounter?	There were 144 distinct female patients over 60 with a diabetes diagnosis who were prescribed metformin and had at least one emergency encounter.
Which mined care pathway is followed by the most patients in cluster 2, and how many of them follow it?	In cluster 2, the most common mined pathway is "M:sodium fluoride 0.0272 MG/MG Oral Gel → P:Dental fluoride treatment → P:Oral health education", followed by 1,265 patients.

patients using mixed-type clustering, sequence mining, and AI interpretation. The clustering produces a small set of understandable clusters of patients. Sequence mining, enhanced by geometric mean lift scoring and diverse filtering, provides clinically relevant paths rather than statistically frequent ones. Language generation converts numerical representations into short summaries and allows natural language querying.

There are also several limitations. The data is artificial, meaning the trends represent modeling in Synthea and do not depict an actual hospital's processes. We also have not conducted any form of clinical validation with medical practitioners, and the mined trends were evaluated based on qualitative evaluation rather than quantitatively compared to other miners with a benchmark for the same data set. The system depends on one external language model, which introduces challenges of expense and repeatability and poses a problem of governance that needs to be resolved before deployment at a hospital. The filter we used is tuned for the data, implying that some of the cleaning process is unique to this dataset.

Several next steps follow. First, we need to validate our approach on a real de-identified dataset and have clinicians evaluate our mined pathways. Second, a better evaluation of the sequence patterns would be beneficial, for instance by comparison with existing clinical guidelines or against other mining algorithms. Finally, a detailed analysis of the language layer is needed, particularly the quality of the SQL agent and of the produced summaries. Replacing the dataset-specific filtering with a more generic one would also make the pipeline more transferable. With appropriate preprocessing and postprocessing, off-the-shelf algorithms already give a hospital an affordable platform built on data it should already have.

REFERENCES

[1] C. d. O. C. S. Rosa, M. Ito, A. B. Vieira, and A. T. A. Gomes, "Modelling and mining of patient pathways: A scoping review," *arXiv preprint arXiv:2206.01980*, 2022.
[2] A. Flothow, A. Novelli, and L. Sundmacher, "Analytical methods for identifying sequences of utilization in health data: a scoping review," *BMC Medical Research Methodology*, vol. 23, p. 212, 2023.
[3] C. Baudrier, Y. Tran, N. Delanoy, S. Katsahian, B. Sabatier, and G. Perrin, "Identifying homogeneous healthcare use profiles and treatment sequences by combining sequence pattern mining with care trajectory clustering in kidney cancer patients on oral anticancer drugs: A case study," *Health Informatics Journal*, vol. 28, pp. 1–18, 2022.
[4] W.-C. Yang, J.-P. Lai, Y.-H. Liu, Y.-L. Lin, H.-P. Hou, and P.-F. Pai, "Using medical data and clustering techniques for a smart healthcare system," *Electronics*, vol. 13, no. 1, p. 140, 2024.
[5] A. Aljohani, "Optimizing patient stratification in healthcare: A comparative analysis of clustering algorithms for ehr data," *International Journal of Computational Intelligence Systems*, vol. 17, p. 173, 2024.
[6] L. Murazzano, P. Landa, J.-B. Gartner, M. H. Raki, and A. Côté, "Clinical pathways discovery for long-term and chronic patients: A process mining approach," *Computers in Biology and Medicine*, vol. 204, p. 111539, 2026.

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF AUTOMATION AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

„BABEŞ-BOLYAI" UNIVERSITY
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
COMPUTER SCIENCE DEPARTMENT

Computer Science Students Conference 2026

June 26, 2026

CLUJ-NAPOCA, ROMANIA

9:00 – 9:15 – Deschidere Sala Aula – Cladirea HUB UTCN / Et. 3

14.15 - Premiere (SCSS-2026) Sala Aula - Cladirea HUB UTCN / Et.3

Panel 1, Sala BT5_01 **Gheorghe Sebestyen, Ioan Mircea-Gabriel** **UTCN, Str. George Baritiu, Nr. 8, Cluj Napoca**			**Panel 2, Sala BT5_02** **Calin Cenan, Iulian Benta** **UTCN, Str. George Baritiu, Nr. 8, Cluj Napoca**	
9:15 – 9:30	*Abonoub Masoud Nageh Youssef*, Comprehensive Comparison of Machine Learning and Deep Learning Models for Network Intrusion Detection with Hierarchical Classification, **UTCN**		**9:15 – 9:30**	*Adina-Ionela Jarda*, Bone Fracture Detection in X-Ray Images Using Deep Learning Methods, **UTCN**
9:30 – 9:45	*Mihai-Bogdan Deaconu*, TopoCurve: Geometry-Aware Topology Reasoning via Bézier Curves in Autonomous Driving, UBB		**9:30 – 9:45**	*Diana-Georgiana Drăgușin*, A Comparative Analysis of Visual Imperceptibility and Statistical Detectability in AI-Generated vs. Natural Covers, UBB
9:45 – 10:00	*Alexandrina-Ruxandra Sebes*, LLM-based Personalized Nutrition Management and Human in-the-Loop Validation, **UTCN**		**9:45 – 10:00**	*Alexandru Craciun*, WmiTrace: Attributing WMI Based Operations to the Originating Process, **UTCN**
10:00 – 10:15	*Ana-Ștefania Kogălniceanu*, From Pixels to Predictions: Estimating 3D Reconstruction Quality in Under-Ice Environments, UBB		**10:00 – 10:15**	*Emanuela-Coralia Toda-Puiuleț*, RoAD: A Romanian Accounting Document Dataset and a Multi-SLM Pipeline for Privacy-Preserving Information Extraction, UBB
10:15 – 10:30	*Alexia Shirazi*, Detection and anatomic classification of intracranial aneurysm using deep learning, **UTCN**		**10:15 – 10:30**	*Alina Maria Macavei*, Dimensionality Reduction of EEG Data for Cognitive State Classification, **UTCN**

10:30 – 10:45	*Klára Orbán*, Benchmarking 3D Reconstruction for Aerial Freeboard Mapping in Arctic Environments, UBB		**10:30 – 10:45**	*Mihai Neagoi*, Entity linking for biomedical domain using Romanian documents, UBB
10:45 – 11:00	*Ana-Maria Bizga*, Astronomical Image Processing for Automatic Star Detection and Constellation Visualization, **UTCN**		**10:45 – 11:00**	*Stan Ariana-Maria*, Simple Yet Effective: Rule-Based Late Fusion for Multimodal Emotion Analysis, **UTCN**
11:00 – 11:15	*Tudor-Ştefan Ivancea*, Local Generation and Visual Validation of Technical Diagrams with QLoRA: A Multi-DSL Benchmark and Proposed Pipeline, UBB		**11:00 – 11:15**	*Denis-Raul Birău*, RiftFX: Design and Implementation of a Custom Interpreted Language for Declarative Desktop UI Development, UBB
11:15 – 11:30	*Andrei Truta*, Vulnerability Detection in Windows RPC Interfaces via Fuzzing, **UTCN**		**11:15 – 11:30**	*Raul Flavius Blotor*, 2D and 3D Brain Tumor Segmentation with nnU-Net and SwinUNETR, and Radiomics Survival Classification, **UTCN**
11:30 – 11:45	*Andrei-Eugen Bratu*, Serverless function benchmarking across cloud industry leaders, UBB		**11:30 – 11:45**	*Andreea-Ioana Gheorghe*, 3D Object Reassembly of Broken Objects using Deep Learning Methods, UBB
11:45 – 12:00	*Bogdan Cristian Marcu*, Explainable Transformer Models for Discovering Epilepsy-Specific Connectivity Patterns in EEG Signals, **UTCN**		**11:45 – 12:00**	*Diana Elena Boitor*, A Modular Segmentation-to-Language Pipeline for Traffic Monitoring Scene Understanding: Model Comparision and VLM Grounding on TSP6K, **UTCN**
12:00 – 12:15	*Camelia-Denisa Sandor*, Kalyra: A Voice-Based Emotion-Aware Journaling Application for Personalized Mental Well-Being Support, UBB		**12:00 – 12:15**	*Răzvan-Andrei Bocra*, MIDAS: Multi-Layer Invariant Domain Adaptation for Cross-Patient Seizure Detection, UBB
12:15 – 12:30	*Darius Cornoc & Vasile Vlad Moca*, Epileptic Seizures Forecasting Through Dynamic Graph Structure Learning and Functional Connectivity, **UTCN**		**12:15 – 12:30**	*Cristian Ioan Tirziu & Horatiu Andrei Bologa*, A Distributed IoT Architecture for Autonomous Bottle Classification: Connectivity-Energy Characterization and Unsupervised Network-Anomaly Detection, **UTCN**

12:30 – 12:45	*Andreea Negoiţescu*, X-cDiT: Explainable Latent Conditional Diffusion Inpainting with Transformers for Iris Occlusion Removal, UBB		**12:30 – 12:45**	*Sorina-Maria Mane*, Multi-Sensor Hardware Entropy Harvester for Cryptographic Password Generation, UBB
12:45 – 13:00	*Dragos Crisan*, RansomOpDFA: Behavioral Ransomware Detection Using Deterministic Finite Automata over File-Operation Traces, **UTCN**		**12:45 – 13:00**	*Alexandru Felix Dumitru*, Performance comparison of classic and transfomer-based dimensionality reduction techniques on an EEG dataset, **UTCN**
13:00 – 13:15	*Alexandru-Nicolae Grancea*, Fine-tuning Whisper for Romanian ASR and Audio-Based Fact-Checking, UBB		**13:00 – 13:15**	*Tudor-Costin Paraschiv*, Automated Urban Road Damage Detection, Segmentation, and Severity Prioritisation from Dashcam Footage, UBB
13:15 – 13:30	*Flavia Adriana Moldovan*, Simulation and Analysis of Video over IP Transmission in Virtual Environments Using the EVE-NG Platform, **UTCN**		**13:15 – 13:30**	*Ioan-Rares Fleseriu*, nnU-Net-Based Multi-Organ and Tumor Segmentation from Abdominal CT Images, **UTCN**
13:30 – 13:45	*Dragoş Trandafir*, Exploring Preprocessing and Training Configurations for Lightweight CNN Mammographic Lesion Classification, UBB		**13:30 – 13:45**	*Anca-Elena Magui*, EndoXAI: An Explainable Machine Learning Pipeline for Endometriosis Risk Prediction, UBB

9:00 – 9:15 – Deschidere Sala Aula – Cladirea HUB UTCN / Et. 3

14.15 - Premiere (SCSS-2026) Sala Aula - Cladirea HUB UTCN / Et.3

Panel 3, Sala BT5_03 **Cristina Pop, Ioan Daniel Pop** **UTCN, Str. George Baritiu, Nr. 8, Cluj Napoca**			**Panel 4, Sala BT5_04** **Mihai Daian, Imre Mali** **UTCN, Str. George Baritiu, Nr. 8, Cluj Napoca**	
9:15 – 9:30	*George Octavian Ungurean* , A Hardware USB Security Gateway for Authenticated HID Access and Wireless File Mediation, **UTCN**		**9:15 – 9:30**	*George Ursu*, Ontology-guided multi-step reasoning for explainable differential diagnosis, **UTCN**
9:30 – 9:45	*Clementina-Ioana Moisă*, Ablation Study on the Pre-Disaster Context in Building Damage Assessment, UBB		**9:30 – 9:45**	*Anisia-Veronica Oltean*, Integrating retrieval augmented generation (RAG) with machine learning-based, UBB
9:45 – 10:00	*Dan Stefan Homeniuc & Mircea Gabriel Antonesi*, Frequency-decomposed Koopman operator for energy time series forecasting, **UTCN**		**9:45 – 10:00**	*Ioana-stefania Risteiu*, Profile-Based Privacy-Preserving Extension for Internet Browsers, **UTCN**
10:00 – 10:15	*Ștefan-George Alexa*, SafeShare: A Zero-Knowledge File Sharing Platform with Privacy-Preserving Anomaly Detection, UBB		**10:00 – 10:15**	*Rareş-Andrei Cotoi*, Optimization of Urban Road Infrastructure Using Graph Theory and Evolutionary Algorithms, UBB
10:15 – 10:30	*Maria-Eliza Gozman-Pop*, Reinterpreting DIPRE for the Modern Web: Structural Extraction of Multilingual Person Records, **UTCN**		**10:15 – 10:30**	*Flavia-Maria Marincau*, Silhouette-Based Coordination for Multi-Agent Deep Reinforcement Learning in Gas Source Localization, **UTCN**
10:30 – 10:45	*Eric Moghioroș*, Privacy-Preserving Federated Anomaly Detection for Distributed Microservice Architectures, UBB		**10:30 – 10:45**	*Teodora Schiau*, Artificial Intelligence Copilot for Breast Cancer Pathology: Integrating Diffusion - Based Spatial Transcriptomics Inference with Large Language Models, UBB

10:45 – 11:00	*Ioana Melisa Barna &Raluca-Delia Bozdog*, Guide Me: An Accessibility-Aware Crowdsensing Navigation System for Urban Pedestrian Routing, **UTCN**		**10:45 – 11:00**	*Mihai Panduru*, DotMal: A Methodology for Discovering Runtime Telemetry Signals in .NET Malware Detection, **UTCN**
11:00 – 11:15	*Andreea-Bianca Croitoru*, Affective Computing in Low-Resource Languages: A Romanian Emotion Detection Model and Empathetic Conversational Agent for Emotional Support, UBB		**11:00 – 11:15**	*Anamaria Ordean*, Multi-Task Toxicity Prediction on Tox21: A Comparative Study from Fingerprints to Graph Neural Networks and Stacking, UBB
11:15 – 11:30	*Miruna Roxana Bacisor & Maria Alexandra Buzila*, Detection and Analysis of Neuronal Avalanches from Local Field Potential Signals under Different Anesthesis Levels, **UTCN**		**11:15 – 11:30**	*Monica Nicoleta Juravle*, Automated Dendrite Analysis in Brightfield Microscopy Using a Classical Image Processing Pipeline, **UTCN**
11:30 – 11:45	*Andrei-Paul Dobrescu*, Deliberation Over Prediction: A Multi-Agent LLM Framework for Post-Earnings Drift, UBB		**11:30 – 11:45**	*Sebastian Șoptelea*, Real-Time Quantum-Inspired 5D K-Means Image Segmentation, UBB
11:45 – 12:00	*Robert-Virgiliu Oltean*, A Secure Low-Cost ESP32 VANET with OBD-II Telemetry and Air Quality Sensing, **UTCN**		**11:45 – 12:00**	*Alexandra-Elena Pitaru & Marian Sandor*, A Study of Static PE Malware Detection on SOREL-20M, **UTCN**
12:00 – 12:15	*Emilia-Andreea Alexandrescu*, Muretto Code: A Multi-tiered Machine Learning Decision Support System for Formula 1 Race Engineering, UBB		**12:00 – 12:15**	*David Obreja*, AR Smart Glasses for Real-Time Queen Bee Detection Using Deep Learning, UBB
12:15 – 12:30	*Raluca-Ioana Mercas*, A Modular Deep Learning Pipeline for Interpretable Glaucoma Screening from Fundus Photographs, **UTCN**		**12:15 – 12:30**	*Razvan-Mihai Bolundut*, On The Detection Of Hand-Marshaled COM Activation And Communication Threats, **UTCN**
12:30 – 12:45	*Elena-Tamara Ivanof*, Monte Carlo Dropout for Out-of-Distribution Detection: A Comparative Study on MNIST-like Benchmarks, UBB		**12:30 – 12:45**	*Alexandru-Dan Balas*, Evaluation Pitfalls in Deep Learning for Small Medical Imaging Datasets, UBB
12:45 – 13:00	*Robert-Emanuel Ardelean*, Text Embedding Cross Fusion for Overcoming Token Limits in Vision Language Models, **UTCN**		**12:45 – 13:00**	*Bogdan Cristian Sipos*, Deep Learning Peak Detection and Quantification in Proton Nuclear Magnetic Resonance Spectra, **UTCN**

13:00 – 13:15	*Delia Dumitru*, Database Testing Using Differential Query Execution and an Adaptive Greedy Extension, UBB		**13:00 – 13:15**	*Iulia-Antonia Bartic*, Intelligent Shipping Cost Estimation Through Packing Optimization and AI Integration, UBB
13:15 – 13:30	*Stefan Bia*, Compiling in the Quantum Computing Era: A Circuit Transpiler for NISQ Devices, **UTCN**		**13:15 – 13:30**	*Dan-Stefan Strungar*, Hardware-Accelerated Semi-Global Matching: Evaluating CUDA & HALIDE Implementations for Neural Network Penalty Prediction, **UTCN**
13:30 – 13:45	*Andra-Riana Fecske*, ArtAICoach: CNN Ensemble Classification and LLM-Generated Feedback for Artistic Style Analysis, UBB		**13:30 – 13:45**	*Alexandra-Ioana Moroz*, Enhanced Screening for Polycystic Ovary Syndrome, UBB
			13:45 – 14:00	*Alexandra-Isabela Trifan*, A Hybrid Phishing-Page Detector with a Region-Adapted Visual Knowledge Base: A Romanian Case Study, UBB

9:00 – 9:15 – Deschidere Sala Aula – Cladirea HUB UTCN / Et. 3 **14.15 - Premiere (SCSS-2026) Sala Aula - Cladirea HUB UTCN / Et.3**	
Panel 5, Sala BT5_05 **Cristian Lelutiu, Alexandru-Gabriel Sirbu** **UTCN, Str. George Baritiu, Nr. 8, Cluj Napoca**	
9:15 – 9:30	*Tudor-Alexandru Bogdan*, QRProtect: A Secure-by-Design Multi-Agent System for Detecting QR-Code Phishing on Fresh URLs, **UTCN**
9:30 – 9:45	*Victor Boar*, OptiStereo: A Marker-Based Sparse Stereo Pipeline for Traceable Optical Measurement, UBB
9:45 – 10:00	*Tudor Sebastian Gădălean, Tudor Sîntoma, Vasile Vlad Moca, Adriana Nagy Dabacan*, Real Time EEG-Driven Emotional Music Generation, **UTCN**
10:00 – 10:15	*Geanina Clipa*, Routing Failures to the Right Agent: A Controlled Study of Agentic AI for Java Unit Test Generation, UBB
10:15 – 10:30	*Daria Ungureanu*, OCT-DINO: Domain-Adapted Vision Transformers for Cross-Dataset Retinal OCT, **UTCN**
10:30 – 10:45	*Ștefania-Ioana Bunaciu*, Automated Detection of Dark Patterns in Web Interfaces, UBB
10:45 – 11:00	*Maria Vasilache*, Brain Tumor Analysis using Integrated YOLO and Segment Anything Model Architectures, **UTCN**
11:00 – 11:15	*Tudor-Luca Crețu*, Topological Deep Learning for Knot Diagrams, UBB

11:15 – 11:30	*Darius-Stefan Vlad,* CourtVision: An Intelligent System for Basketball Action Recognition and Kinematic Analysis Using Monocular Video, **UTCN**
11:30 – 11:45	*Emanuel-Vasile Puțura*, Data Redundancy in Encrypted Traffic Datasets for Network Traffic Classification, UBB
11:45 – 12:00	*Vlad-Ioan Durdeu,* A Multi-Agent Desktop Automation Architecture with Structured Evidence Review via Model Context Protocol, **UTCN**
12:00 – 12:15	*Alexandru-Mihai Toma*, CodeGraph: Repository-Level Structural Retrieval for AI Coding Agents via Personalized PageRank, UBB
12:15 – 12:30	*Bianca Mitroi & Vlad Ursache*, Securing the Aggregation Pipeline: Server and Client-Side Defenses Against Multi-Vector Threats in Federated Learning, **UTCN**
12:30 – 12:45	*Miruna-Elena Mărginean*, TravelAssist: A Clustering-Based Approach to Automated Travel Itinerary Generation, UBB
12:45 – 13:00	*Ioan-Iulian Todor*, HospitAI: A Hospital Analytics Platform Combining Patient Clustering, Clinical Sequence Mining and AI-Generated Interpretation, **UTCN**
13:00 – 13:15	*Mălina-Teodora Dumitrescu*, When VPN Detectors Learn the Capture: Structural Dataset Shift in Flow-Based Encrypted VPN Traffic Classification, UBB
13:15 – 13:30	*Aurora-Malina Bondre***,** Multi-label ECG classification using deep learning architectures with multimodal metadata fusion**, UTCN**
13:30 – 13:45	*Alina Buliga*, ClujHub: A Data-Driven Civic Engagement Platform with Automated Hotspot Detection and Petition Generation, UBB
13:45 – 14:00	*Alexandru-Răzvan Gorgan*, Explainable Fake News Detection in Romanian Media Using a RoBERT Ensemble, UBB